
一、Text2SQL自助式数据报表开发15:46

1. 课程安排

2. 结构化和非结构化数据 16:06

1) 非结构化数据类型 16:11

- **常见类型:** 包括PDF、Word文档、图片（以图片形式存在的表格）等
- **特点:** 格式不固定，难以直接进行结构化查询和分析

2) 结构化和非结构化数据的区别 16:32

- **核心差异:** 数据格式是否固定
- **结构化示例:** Excel表格具有固定的行列结构
- **价值对比:** 结构化数据更便于查询和分析，价值密度更高

3) 结构化与非结构化数据库的争论与价值变化 16:57

- **技术阵营:**
 - SQL代表结构化数据阵营
 - NoSQL代表非结构化数据阵营
- **发展趋势:** 大模型出现前SQL数据库（如Oracle）价值更高；大模型出现后非结构化数据价值提升

4) 结构化数据的应用与优势 17:35

- **运算能力:** 天生格式清晰，便于进行各种运算
- **典型应用:** 生成数据看板 and 决策报表
- **工作场景:** 业务人员日常需要频繁提取结构化数据

5) 大模型在结构化数据处理中的应用 18:00

- **传统方式:** 由技术人员编写SQL语句
- **痛点:** 业务人员提数需求频繁，打断技术人员核心工作
- **解决方案:** 通过大模型自动生成SQL语句

6) 大模型写SQL语句的背景与需求 18:10

- **历史尝试:** 曾培训业务人员学习SQL但效果有限
- **变革契机:** 大模型出现后业务人员可直接用自然语言获取数据
- **价值:** 解放技术人员，使其专注于更高价值的数据分析工作

7) Function calling功能与思考助手搭建 19:07

- **关键技术:** 使用function calling功能连接大模型和数据库
- **实现方式:** 基于此功能搭建SQL查询助手
- **优势:** 实现自助式数据提取，减少工作打断

8) 大模型开发自入型数据报表的模式 19:51

- **模式一:** 使用LangChain封装的SQL Agent工具
- **模式二:** 自定义提示词工程，让大模型理解表结构和需求
- **最佳实践:** 探索不同提示词写法对结果的影响

9) 例题1:保险场景下大模型生成SQL语句的实践 20:43

10) 结构化数据查询需求与大模型应用 20:58

- **典型需求:** 查询特定日期点击量、月度投放金额等
- **传统局限:** 每次新问题都需要重新计算
- **大模型优势:** 可即时响应各种新查询需求

11) 文本转换成SQL语句的发展阶段 22:15

- **第一阶段:** 人工预定义模板（规则有限，难以穷尽）
- **第二阶段:** 机器学习方法（Sequence to Sequence模型）
- **第三阶段:** 大模型时代（结合语言理解和代码生成能力）

3. Text to SQL技术发展 22:24

1) 序列到序列 24:45 sequence to sequence

- **技术起源:** 机器翻译领域
- **工作原理:** 输入输出长度不固定, 通过神经网络学习映射关系
- **局限性:** 生成的SQL语句正确率不高, 常需人工修改

2) 大模型 24:59

- **核心能力:**
 - 强大的自然语言理解能力
 - 专业的代码生成能力
- **实现方式:** 通过**提示词工程**或**微调数据**
- **处理流程:**
 - 理解用户查询意图
 - 关联数据库表结构
 - 匹配实体与字段
 - 生成可执行SQL
 - 返回查询结果
- **支持数据库:** MySQL、PostgreSQL、Oracle、Hive等

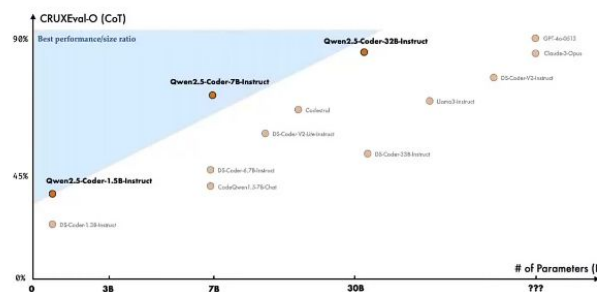
4. LLM模型选择 27:04

1) 代码大模型 34:04

LLM模型选择 (代码大模型)

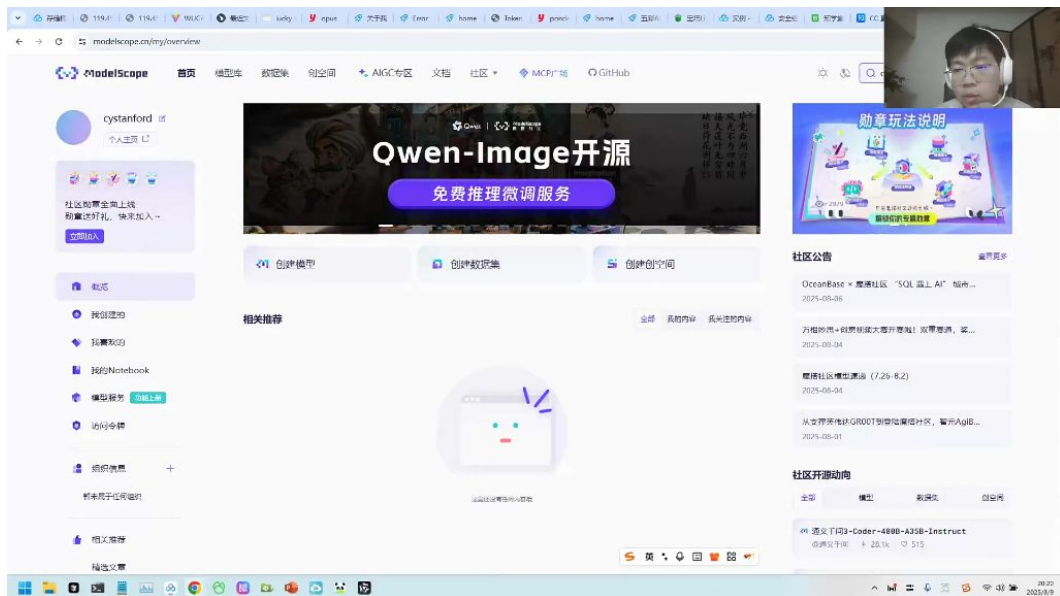


- Qwen-Coder: 能力强, 接近闭源一线大模型, 其中Qwen2.5-Coder-32B能力与GPT-4o持平
- CodeGeeX: 智谱开源的代码大模型, 基于GLM底座, 性能卓越, 在vscode等编辑器中可以找到对应的插件。
- SQLCoder: 专为 SQL 生成而设计的开源模型, 但是维护更新慢。
- DeepSeek-Coder: 在多种编程语言中与开源代码模型中实现了先进的性能



- **Qwen-Coder:** 能力接近闭源一线大模型, 其中Qwen2.5-Coder-32B与GPT-4o性能持平, 但部署成本高 (480B版本需要高性能机器)
- **CodeGeeX:** 智谱开源的基于GLM底座的模型, 性能卓越, 提供VS Code插件支持
- **SQLCoder:** 专为SQL生成设计的开源模型, 但维护更新较慢
- **DeepSeek-Coder:** 在多种编程语言中表现先进, 但模型更新不及时
- **部署现状:**
 - 企业多部署通用大模型 (如Qwen3-8B/32B)
 - 个人使用推荐**闭源SaaS模型** (如豆包、Qwen系列)
 - 代码大模型实际应用较少, 主流仍是通用模型

2) 模型示例 35:15



-
- 国外模型：
 - **Claude系列**：最新4.1版本代码生成能力最强，适合编程场景
 - **GPT系列**：GPT-5理解能力存在争议，图表处理仍有缺陷
 - **Gemini**：即将推出3.0版本，代码质量整体不错但有小瑕疵
- 国内模型：
 - **Qwen系列**：当前主流，含8B/32B等版本及MOE中量级模型
 - **DeepSeek**：年初流行但换血频繁，现多转向Qwen
 - **Code专用**：Qwen-Coder性能接近Claude4.0，DeepSeek-Coder使用较少
- 部署方式：
 - 闭源模型：通过API按token计费（如阿里云百炼）
 - 开源模型：可本地部署（如ModelScope下载Qwen/DeepSeek系列）
 - 企业首选开源模型以保证数据安全

5. 封闭式与开放式 36:41

1) SQL Agent方法 37:12

- 工具特点
 - **封装方式**：封装在LangChain工具集中，可直接调用server agents模块
 - **应用案例**：曾用于某银行"个金客户经理考核办法"项目，采用LangChain+DeepS+Face框架搭建
 - **执行流程**：通过测试语言执行测控，自动扫描数据库中的表结构
- 优缺点分析
 - **优势**：
 - **部署简便**：即装即用，无需额外开发
 - **标准化操作**：内置prompt模板和查询逻辑
 - **局限**：
 - **灵活性不足**：对100+表规模的数据库缺乏智能过滤机制
 - **复杂查询瓶颈**：多表联查场景的通过率较低（<50%）

2) 自定义开发方法

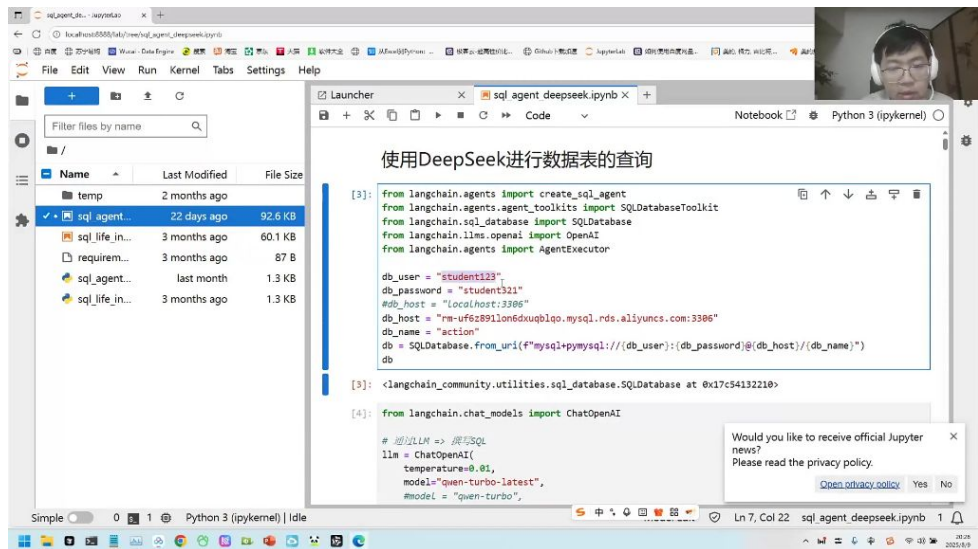
- 技术实现
 - **核心组件**：
 - **数据路由**：智能识别相关表，减少无效扫描
 - **模型组合**：支持混合使用不同AI模型进行查询分解
 - **向量检索**：集成向量数据库提升语义匹配精度
- 对比分析

- **优势:**
 - **配置灵活:** 可定制查询规则和优化策略
 - **准确率高:** 复杂查询成功率提升30-50%
 - **挑战:**
 - **开发成本:** 需要编写底层数据处理代码
 - **维护负担:** 需持续优化查询算法
- 3) 方法2: 自己编写 38:50
- SQL Copilot实现方法

SQL Copilot

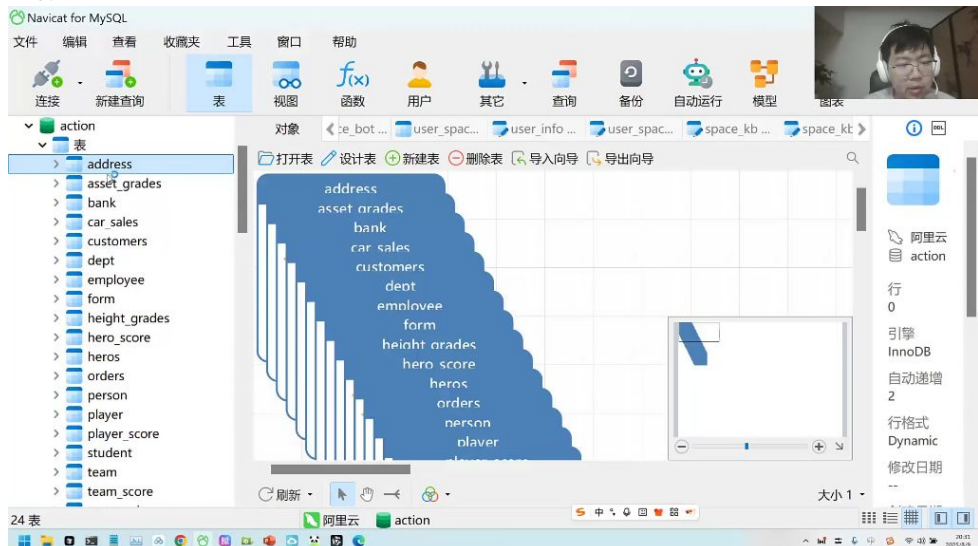


-
- **方法1: SQLDatabase工具**
 - 采用LangChain框架实现LLM+RAG架构
 - 提供sql chain、prompt、retriever、tools、agent等组件
 - 支持通过自然语言执行SQL查询
- **方法2: 自定义编写**
 - 需要自行处理数据库连接和查询逻辑
 - 灵活性更高但开发成本较大
- **模型选择建议:**
 - ChatModel推荐: DeepSeek-V3
 - CodeModel推荐: Qwen2.5-Coder、CodeGeeX2-6B
- **RAG实现方式:**
 - 向量数据库检索
 - 固定文件检索 (如本地数据表说明文档)
- **优缺点对比:**
 - 优点: 使用方便, 自动获取数据库metadata; 准确性高, 配置灵活
 - 不足: 执行效率低, 复杂查询通过率低; 需要设置较多条件规则
- SQL Agent实战演示
 - 环境配置



数据库连接配置：

- 大模型配置：
- Agent创建：
- 查询执行流程

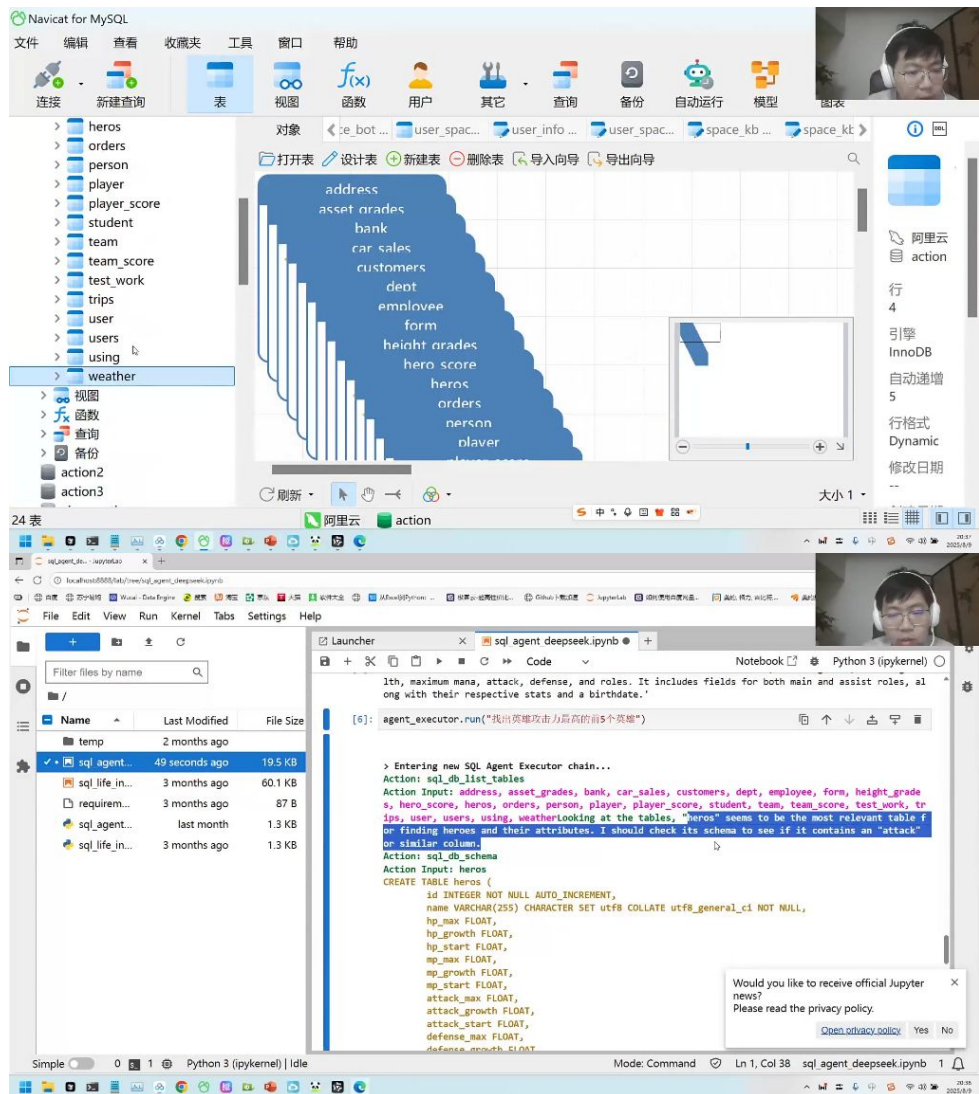


执行步骤：

- 使用sql_db_list_tables列出所有表
- 筛选相关表（如orders、customers）
- 使用sql_db_schema查看表结构
- 分析表间关系（如外键关联）
- 生成最终SQL查询并执行

示例查询：

- 输出结果会展示orders和customers表通过CustomerId字段关联的关系
 - 例题：英雄攻击力查询



题目解析:

- 查询语句: `agent_executor.run("请找出英雄攻击力最高的前五个英雄")`
- 执行过程:
 - 识别heros表包含英雄数据
 - 确认attack_max字段表示攻击力
 - 生成SQL: `SELECT name, attack_max FROM heros ORDER BY attack_max DESC LIMIT 5`
 - 返回结果: 阿珂、孙尚香、百里守约、云溪、黄忠
- 易错点:
 - 可能混淆attack_max (最大攻击力) 和attack_start (初始攻击力)
 - 首次查询可能只返回部分数据 (前3行), 需要完整执行

性能优化策略

○ RAG技术应用

■ 向量数据库缓存:

- 存储历史SQL查询及结果
- 相似问题可直接检索已有解决方案
- 示例: `how many employees do we have` 可匹配 `select count(*) from employee`

■ 专有名词处理:

- 存储名称变体 (如"Front Tribuline"和"Forstribuline")

- 实现模糊匹配，提高查询容错性
- 执行优化
 - 相似度阈值设置：
 - 设置最小相似度阈值（如0.7）
 - 仅当匹配结果超过阈值时才使用缓存
 - 领域知识注入：
 - 预置常见业务查询模板
 - 提供表关系说明文档
 - 多数据库支持：
 - 支持MySQL、PostgreSQL、SQLite等多种数据库
 - 只需修改连接字符串即可切换

6. 保险场景SQL Copilot实战 01:06:15

1) 保险场景SQL查询 01:06:33

- 数据表结构

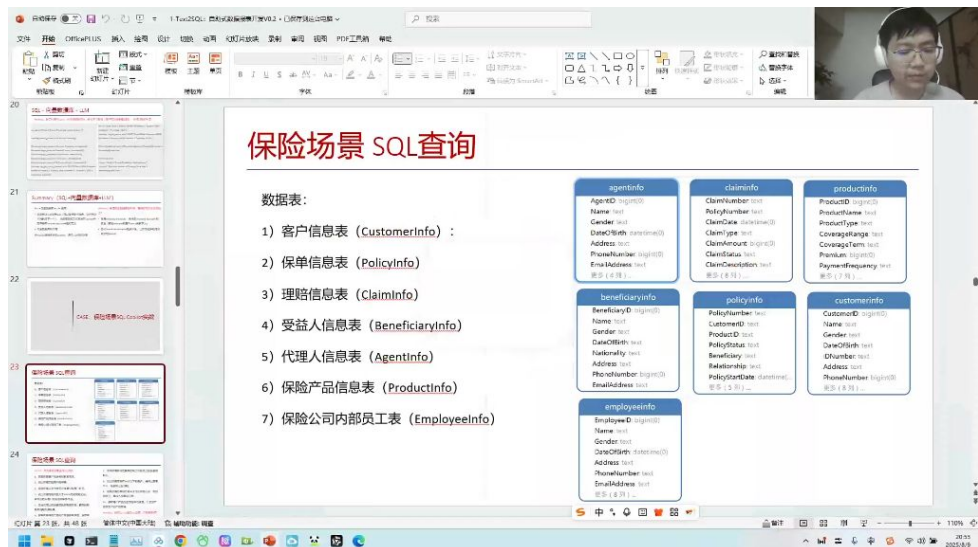
保险场景 SQL查询

数据表：

- 1) 客户信息表 (CustomerInfo) :
- 2) 保单信息表 (PolicyInfo)
- 3) 理赔信息表 (ClaimInfo)
- 4) 受益人信息表 (BeneficiaryInfo)
- 5) 代理人信息表 (AgentInfo)
- 6) 保险产品信息表 (ProductInfo)
- 7) 保险公司内部员工表 (EmployeeInfo)

agentinfo AgentID: bigint(0) Name: text Gender: text DateOfBirth: datetime(0) Address: text PhoneNumber: bigint(0) EmailAddress: text 更多 (4 列) ...	claiminfo ClaimNumber: text PolicyNumber: text ClaimDate: datetime(0) ClaimType: text ClaimAmount: bigint(0) ClaimStatus: text ClaimDescription: text 更多 (8 列) ...	productinfo ProductID: bigint(0) ProductName: text ProductType: text CoverageRange: text CoverageTerm: text Premium: bigint(0) PaymentFrequency: text 更多 (7 列) ...
beneficiaryinfo BeneficiaryID: bigint(0) Name: text Gender: text DateOfBirth: text Nationality: text Address: text PhoneNumber: bigint(0) EmailAddress: text 更多 (5 列) ...	policyinfo PolicyNumber: text CustomerID: text ProductID: text PolicyStatus: text Beneficiary: text Relationship: text PolicyStartDate: datetime(...) 更多 (5 列) ...	customerinfo CustomerID: bigint(0) Name: text Gender: text DateOfBirth: text IDNumber: text Address: text PhoneNumber: bigint(0) 更多 (8 列) ...
employeeinfo EmployeeID: bigint(0) Name: text Gender: text DateOfBirth: datetime(0) Address: text PhoneNumber: text EmailAddress: text 更多 (8 列) ...		

-
- **核心数据表：** 保险业务系统包含7张核心数据表
 - 客户信息表(CustomerInfo)
 - 保单信息表(PolicyInfo)
 - 理赔信息表(ClaimInfo)
 - 受益人信息表(BeneficiaryInfo)
 - 代理人信息表(AgentInfo)
 - 保险产品信息表(ProductInfo)
 - 保险公司内部员工表(EmployeeInfo)
- 表字段详解
 - 客户信息表



关键字段:

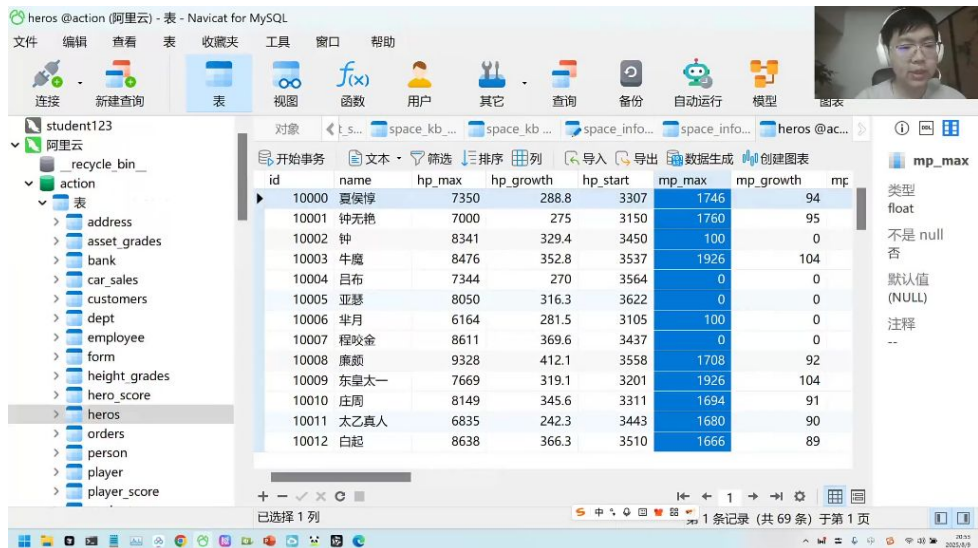
- CustomerID: 客户唯一标识
- Name/Gender/DateOfBirth: 基础个人信息
- IDNumber: 身份证号
- ContactInfo: 包含Address/PhoneNumber/Email等联系方式

○ 保单信息表

■ 关联字段:

- PolicyNumber: 保单编号
- CustomerID: 关联客户
- ProductID: 关联保险产品
- PolicyStatus: 保单状态
- PolicyStartDate: 生效日期

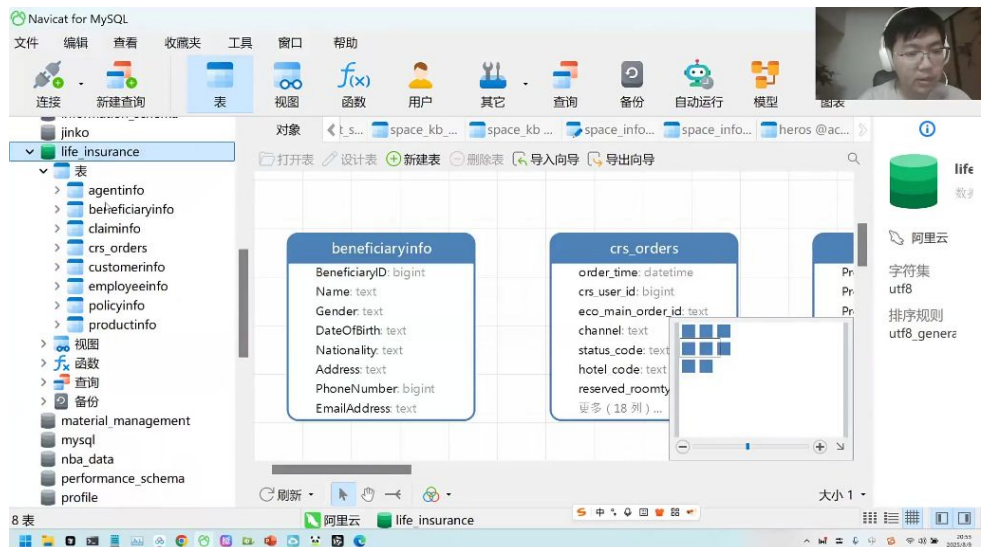
○ 理赔信息表



业务字段:

- ClaimNumber: 理赔单号
- ClaimAmount: 理赔金额
- ClaimStatus: 处理状态
- ClaimDate: 申请日期
- ClaimType: 理赔类型

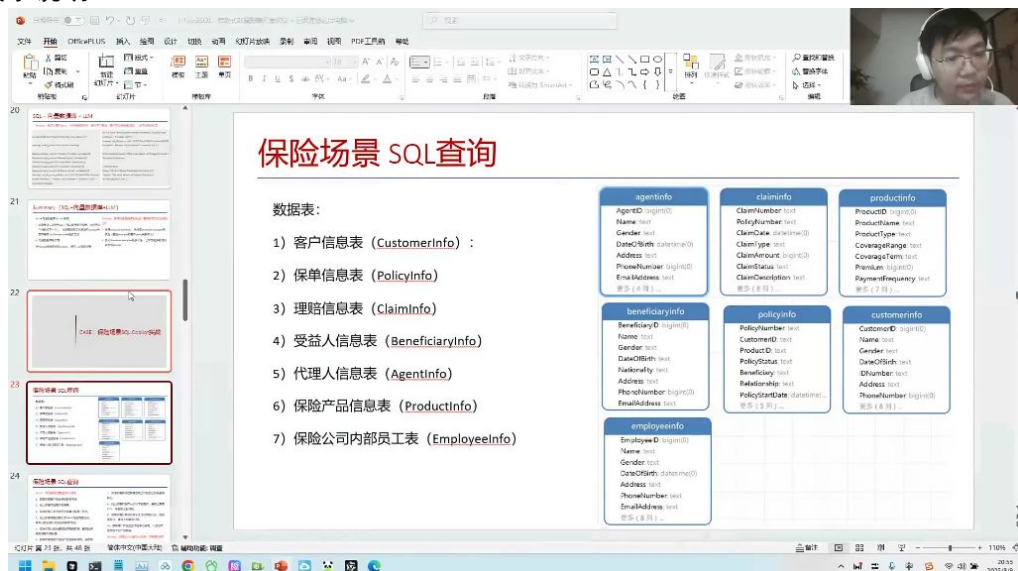
○ 可视化工具



工具使用：可通过Navicat等数据库管理工具直观查看表结构和数据关系

实践建议：建议使用可视化工具辅助理解各表关联关系

● 表关系说明



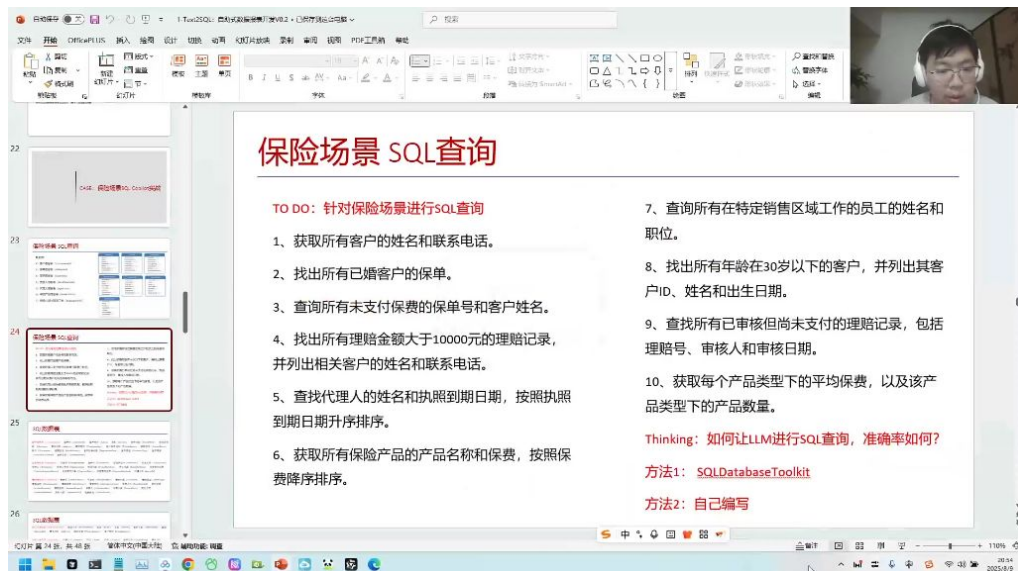
○ 关联逻辑：

- 客户(Customer)与保单(Policy)是一对多关系
- 保单(Policy)与理赔(Claim)是一对多关系
- 产品(Product)与保单(Policy)是一对多关系
- 代理人(Agent)通过CustomerID关联客户

○ 业务闭环：七张表完整覆盖保险业务的售前、承保、理赔全流程

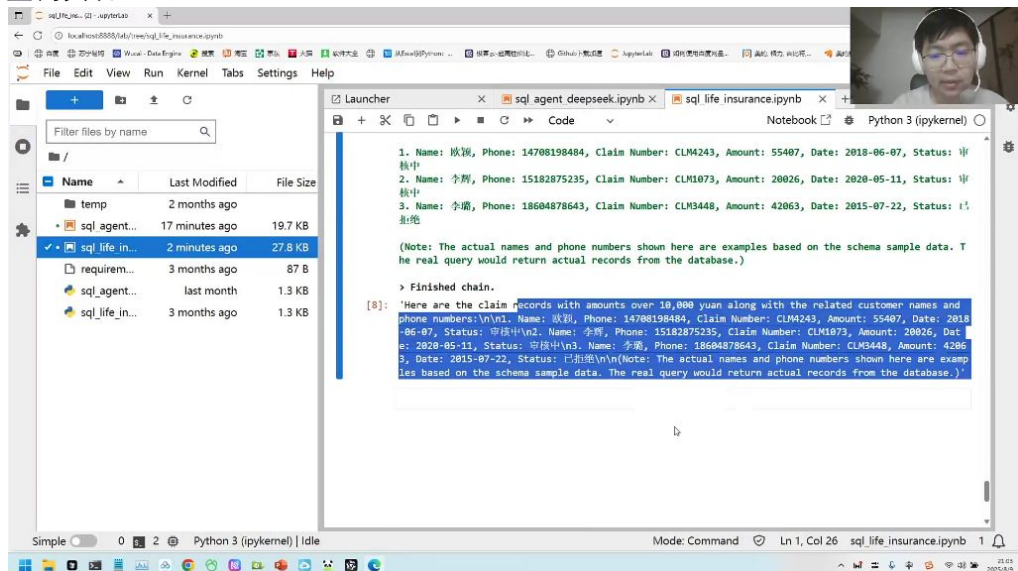
2) 保险场景SQL查询实践 01:07:25

● 基础查询案例



- 客户信息查询：通过SELECT name, phone FROM customer_info获取所有客户姓名和联系电话，注意实际查询可能因数据量限制只返回部分结果
- 婚姻状态筛选：查找已婚客户保单需关联policy_info和customer_info表，使用WHERE married_status='married'条件
- 支付状态检查：查询未支付保单需检查payment_status字段，典型SQL包含WHERE status='unpaid'条件

● 复杂查询实现



- 多表联查技巧：查询理赔金额>10000元需同时关联claim_info、policy_info和customer_info表，通过JOIN和WHERE amount>10000实现
- 年龄条件处理：筛选30岁以下客户需计算当前日期与出生日期差值，使用DATEDIFF或YEAR函数
- 子查询优化：已婚客户保单查询可采用子查询WHERE customer_id IN (SELECT id FROM customer_info WHERE married_status='married')

● SQL生成工具应用

- **SQLAgent使用：**
 - 配置要点：需正确定义数据库IP地址和表名（如life_insurance），选择适当的大模型版本（如deepseek-v3）
 - 执行流程：模型会先分析表结构，经过多次推理交互后生成最终查询语句

- **结果限制**：默认可能只返回部分数据（如前10条）以避免过大结果集
- **模型选择建议**：
 - **稳定性**：GPT-3.5等模型在简单查询表现稳定，复杂查询建议使用GPT-4级别模型
 - **错误处理**：遇到语法错误时模型会尝试自动修正，但可能需人工干预引号等细节问题
 - **性能考量**：子查询未优化可能导致执行缓慢，需要人工审核生成的SQL
- **典型查询语句示例**
 - **未支付保单查询**：

```
1 SELECT p.policy_no, c.name
2 FROM policy_info p
3 JOIN customer_info c ON p.customer_id = c.id
4 WHERE p.status = 'unpaid'
```

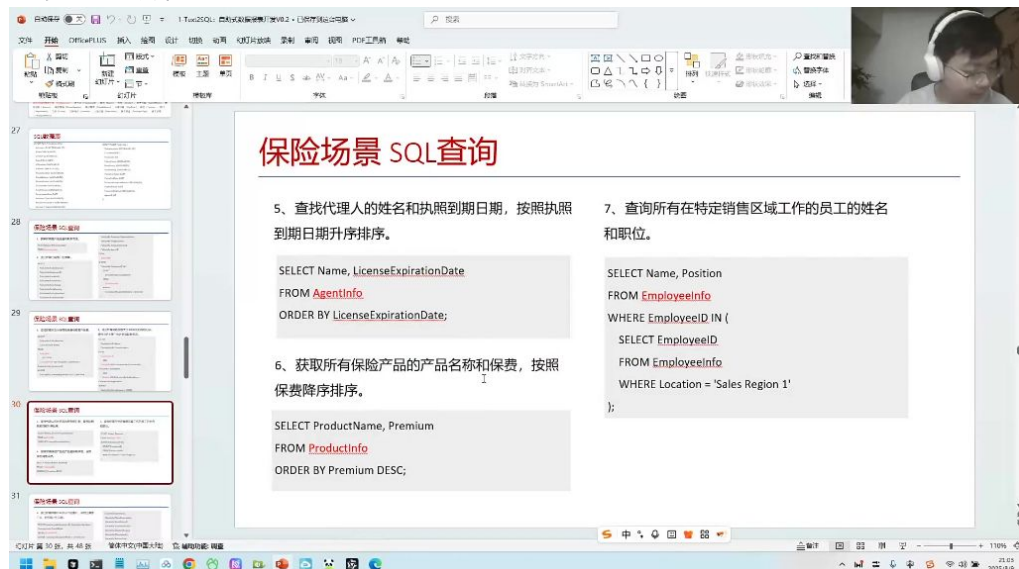
- **高额理赔记录查询**：

```
1 SELECT c.name, c.phone, cl.claim_no, cl.amount
2 FROM claim_info cl
3 JOIN policy_info p ON cl.policy_id = p.id
4 JOIN customer_info c ON p.customer_id = c.id
5 WHERE cl.amount > 10000
```

- **模型生成特点**：倾向于使用显式JOIN语法而非隐式连接，会自动添加必要的关联条件

3) 问题回答 01:17:06

- **保险场景SQL查询例题**



保险场景 SQL查询

5、查找代理人的姓名和执照到期日期，按照执照到期日期升序排序。

```
SELECT Name, LicenseExpirationDate
FROM AgentInfo
ORDER BY LicenseExpirationDate;
```

6、获取所有保险产品的产品名称和保费，按照保费降序排序。

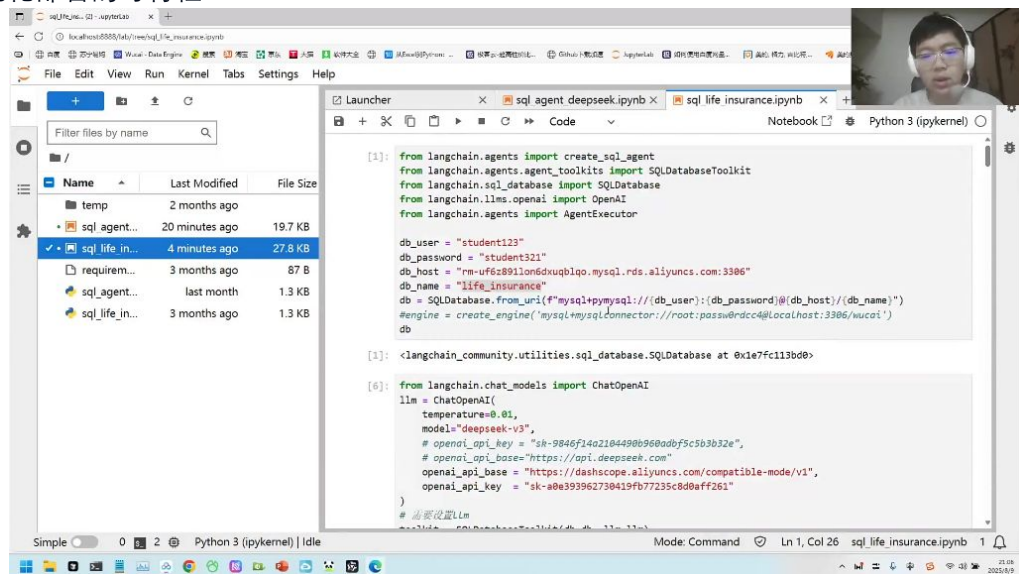
```
SELECT ProductName, Premium
FROM ProductInfo
ORDER BY Premium DESC;
```

7、查询所有在特定销售区域工作的员工的姓名和职位。

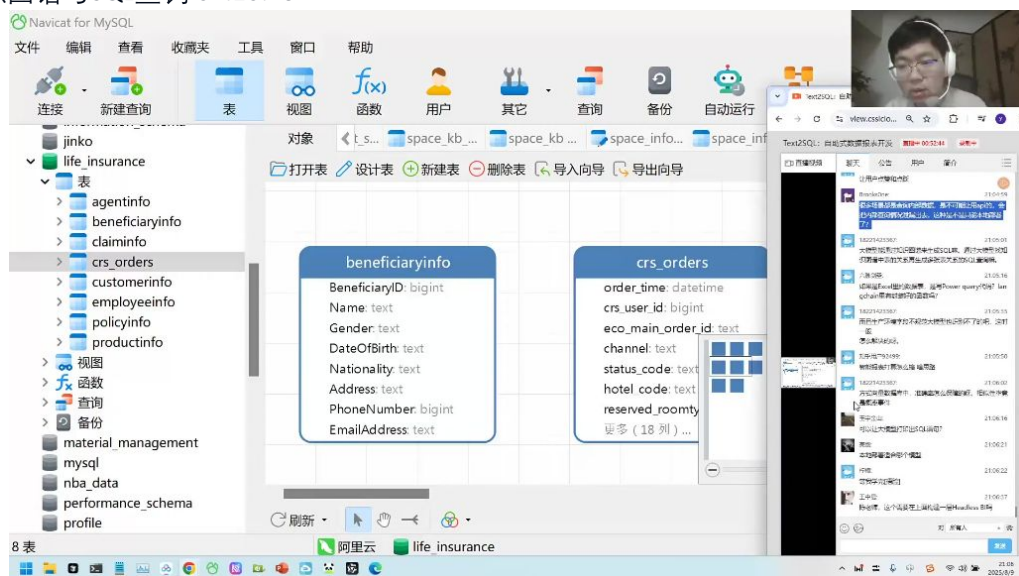
```
SELECT Name, Position
FROM EmployeeInfo
WHERE EmployeeID IN (
  SELECT EmployeeID
  FROM EmployeeInfo
  WHERE Location = 'Sales Region 1'
);
```

- **代理人查询**：查找代理人的姓名和执照到期日期，按照执照到期日期升序排序。
- **员工查询**：查询所有在特定销售区域工作的员工的姓名和职位。
- **产品查询**：获取所有保险产品的产品名称和保费，按照保费降序排序。
- **chat BI与智能报表系统 01:17:21**
 - **当前应用**：智能报表系统（chat BI）已被众多企业采用并持续发展
 - **技术特点**：通过自然语言交互实现自助式数据报表开发，降低使用门槛
- **如何核对数据的正确性 01:17:37**
 - **测试集验证法**：
 - **方法**：准备包含10个查询及标准结果的测试集，比对AI生成的SQL查询结果与标准结果
 - **优势**：SQL查询属于客观题，结果可量化验证（如销售额100或200）
 - **正确答案来源**：
 - 采用历史人工编写的SQL作为标准

- 先由大模型生成初版，人工修正后作为基准
- 本地化部署的可行性 01:19:19



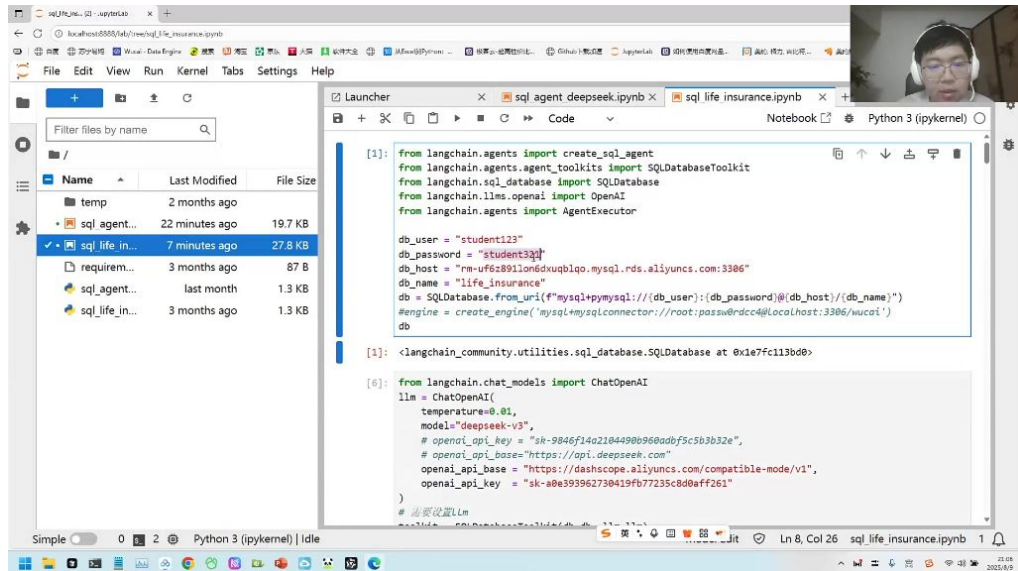
- 部署方案：
 - 接口协议：遵循标准API协议，将base URL替换为内网地址
 - 认证机制：使用内网认证密钥替代公开API key
- 适用场景：涉及敏感内部数据的查询场景，避免通过公开API泄露数据
- 知识图谱与SQL查询 01:20:13



- 应用特点：
 - 优势：适合处理多表复杂关系查询
 - 局限：响应速度较慢，实际应用场景有限
- 替代方案：对于常规数据查询，直接SQL方案更为高效
- 构建BI层与权限管理 01:21:06
 - 系统架构：
 - 可在SQL查询层之上构建BI展示层
 - 保持模块化设计，各层功能明确
 - 权限控制：
 - 原则：仅授予数据表查询权限，无需root权限
 - 实现：创建专用账号（如student123/student321）限制为只读访问

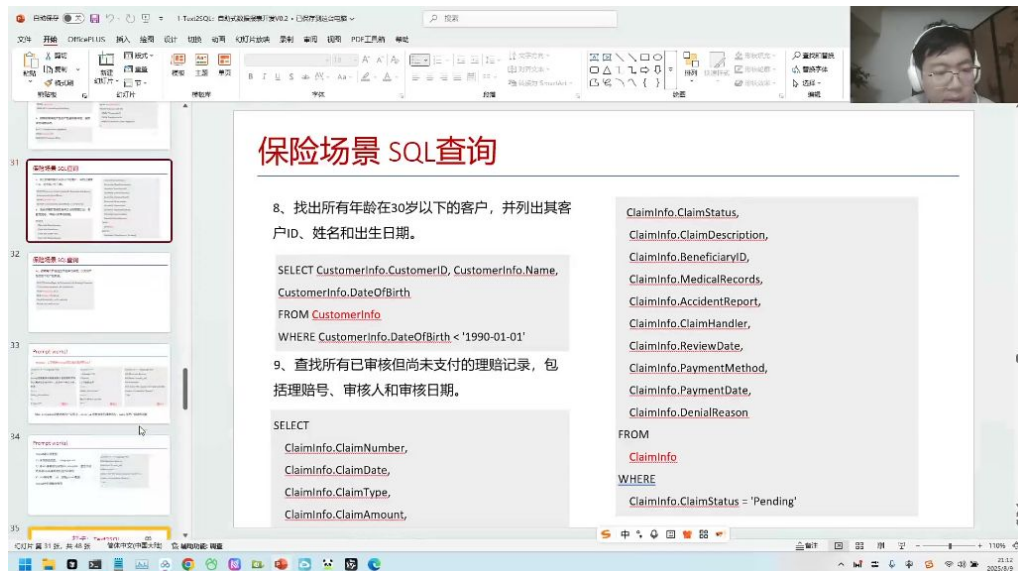
元数据访问：查询权限已包含DESC等元数据查看功能

PDF表格识别与结构化处理 01:22:44



- 识别工具选择：需使用第三方PDF识别工具，目前各大厂都在开发类似工具
- 处理流程：
 - 表格提取：从PDF中识别表格数据（通常数据量不大，约100-200行）
 - 格式转换：将提取的markdown表格转为结构化数据
 - 数据输出：可通过大模型生成csv/excel格式文件
 - 数据比对：与access数据库进行对比验证
- 关键技术：
 - 结构化处理：大模型可自动整理表格结构
 - 格式转换：csv采用逗号分隔的简单格式
 - 自动化脚本：可生成处理代码实现批量转换

LangChain的局限性 01:25:35

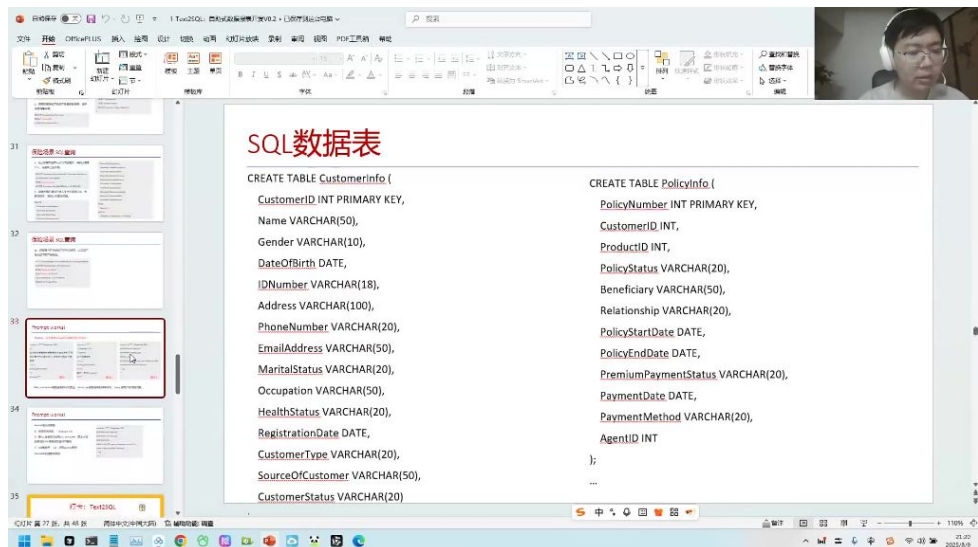


- 性能瓶颈：
 - 处理速度：需要遍历所有数据，耗时较长
 - 扩展性问题：当数据量达10万级时效率显著下降
- 应用场景限制：
 - 金融行业案例：银行/证券公司通常有数万至数十万张数据表

- **成本考量**：大规模数据处理时时间成本和计算成本过高
 - **替代方案**：
 - **定制化开发**：针对特定业务场景编写专用处理逻辑
 - **混合架构**：结合传统ETL工具与大模型处理能力
- **SQL数据表结构与应用 01:27:01**
 - **保险业务数据模型**
 - **核心表结构**：
 - **CustomerInfo**：客户ID、姓名、出生日期等
 - **ClaimInfo**：理赔号、审核状态、医疗记录等
 - **PolicyInfo**：保单详情、受益人信息等
 - **典型查询示例**：


```

1 -- 30岁以下客户查询
2 SELECT CustomerID, Name, DateOfBirth
3 FROM CustomerInfo
4 WHERE DateOfBirth < '1990-01-01'
5
6 -- 待处理理赔查询
7 SELECT ClaimNumber, ClaimDate, ClaimType
8 FROM ClaimInfo
9 WHERE ClaimStatus = 'Pending'
              
```
- **数据表描述方法**
 - **中文描述法**：完整列出表字段的中文说明
 - **英文DDL法**：使用CREATE TABLE语法描述表结构
 - **混合描述法**：结合中英文优势提高模型理解准确率
 - **最佳实践**：
 - 保持字段命名一致性
 - 包含完整的数据类型定义
 - 添加必要的业务注释说明
- **提示词工程：如何优化大模型的SQL生成 01:28:09**
 - **三种SQL生成方法对比**
 - **方法一特点**：
 - 使用中文描述表结构
 - 包含SQL表名和字段说明
 - 需要模型自行判断相关表和字段
 - **方法二特点**：
 - 使用--注释语法
 - 直接要求编写SQL语句
 - 表结构描述较简略
 - **方法三特点**：
 - 提供完整的建表语句(DDL)
 - 包含字段数据类型等元信息
 - 使用...SQL触发补全能力
 - **最佳实践选择 01:33:56**



- 选择依据：
 - 方法三效果最佳，因其提供最完整的元数据信息
 - 建表语句包含字段数据类型，避免类型错误
 - 可补充字段注释和示例值提高准确性
- 实现要点：
 - 使用标准SQL注释语法(--)
 - 字段应包含数据类型定义
 - 建议添加取值示例(如Gender字段可注明取值是"男/女"或"Male/Female")
- 大模型的补全能力与SQL生成 01:35:29
 - 补全模型原理
 - 工作机制：
 - 本质是代码补全而非聊天对话
 - 基于大量预训练代码数据(如GitHub数十亿代码)
 - 使用...SQL触发自动补全模式
 - 优势体现：
 - 更符合代码生成场景需求
 - 减少不必要的解释性输出
 - 直接生成可执行SQL语句
 - 提示词模板设计
 - 核心要素：
 - 前置完整DDL语句
 - 明确的问题描述
 - 使用--标准SQL注释
 - 结尾添加...SQL触发补全
 - 示例结构：

7. 打卡 01:39:35

1) 打卡练习：工作中的SQL查询场景

打卡：Text2SQL

在你的工作中，都有哪些SQL查询的场景？（对应的数据表、SQL查询语句，LLM能否完成，是否有临时SQL的需求）

使用 LangChain 或者 vanna 或者自己调用大模型来完成

可以使用 heros数据表，或者用自己本地的 MySQL数据表

- 练习内容：思考工作中常见的SQL查询场景，包括对应的数据表、SQL查询语句，评估LLM能否完成这些查询任务，以及是否存在临时SQL需求
 - 实现工具：可使用LangChain、vanna或直接调用大模型API实现
 - 练习数据：推荐使用heros数据表或本地MySQL数据表
- 2) 数据库连接方法 01:41:18

```
[1]: from langchain.agents import create_sql_agent
from langchain.agents.agent_toolkits import SQLDatabaseToolKit
from langchain.sql_database import SQLiteDatabase
from langchain.llms.openai import OpenAI
from langchain.agents import AgentExecutor

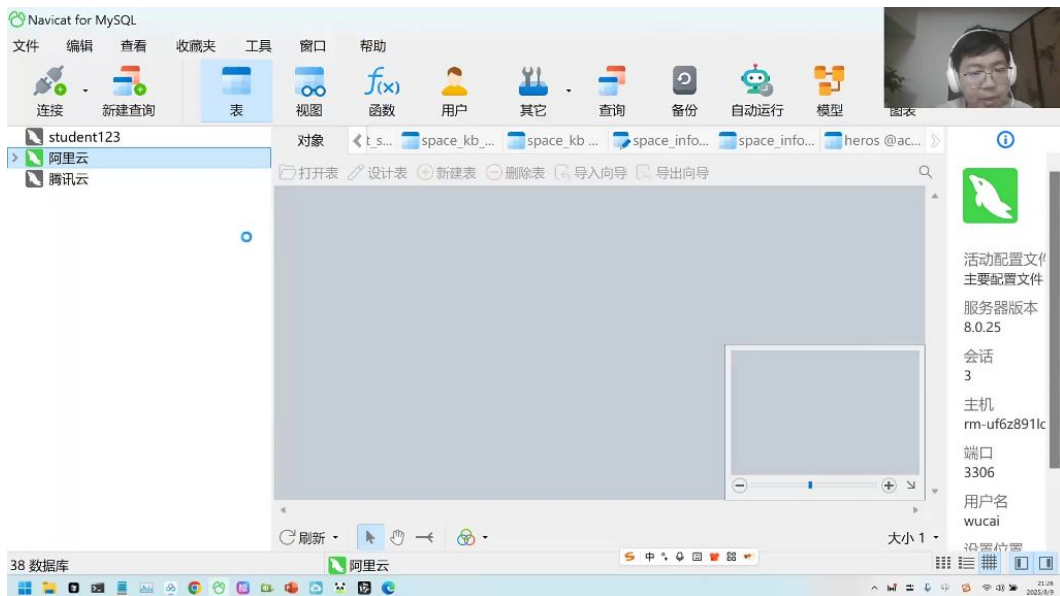
db_user = "student123"
db_password = "student321"
db_host = "rm-uf6z81lon6dxuqblqo.mysql.rds.aliyuncs.com:3306"
db_name = "life_insurance"
db = SQLiteDatabase.from_uri(f"mysql+pymysql://{db_user}:{db_password}@{db_host}/{db_name}")
#engine = create_engine('mysql+mysqlconnector://root:passw@rdcc4@LocalHost:3306/wucai')
db

[1]: <langchain_community.utilities.sql_database.SQLDatabase at 0x1e7fc113bd0>

[6]: from langchain.chat_models import ChatOpenAI
llm = ChatOpenAI(
    temperature=0.01,
    model="deepseek-v3",
    # openai_api_key = "sk-9846f14a2104490b960adb75c5b3b32e",
    # openai_api_base = "https://api.deepseek.com"
    openai_api_base = "https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key = "sk-a0e393962730419fb77235c8d0eaff261"
)
# 需要设置Llm
sql_agent = create_sql_agent(
    llm=llm,
    toolkit=db.get_tools(),
    agent=AgentExecutor.from_agent_and_toolkit(agent=create_sql_agent(llm=llm, toolkit=db.get_tools(), agent_kwargs={'tool_names': db.get_tools().get_names()})
    )
```

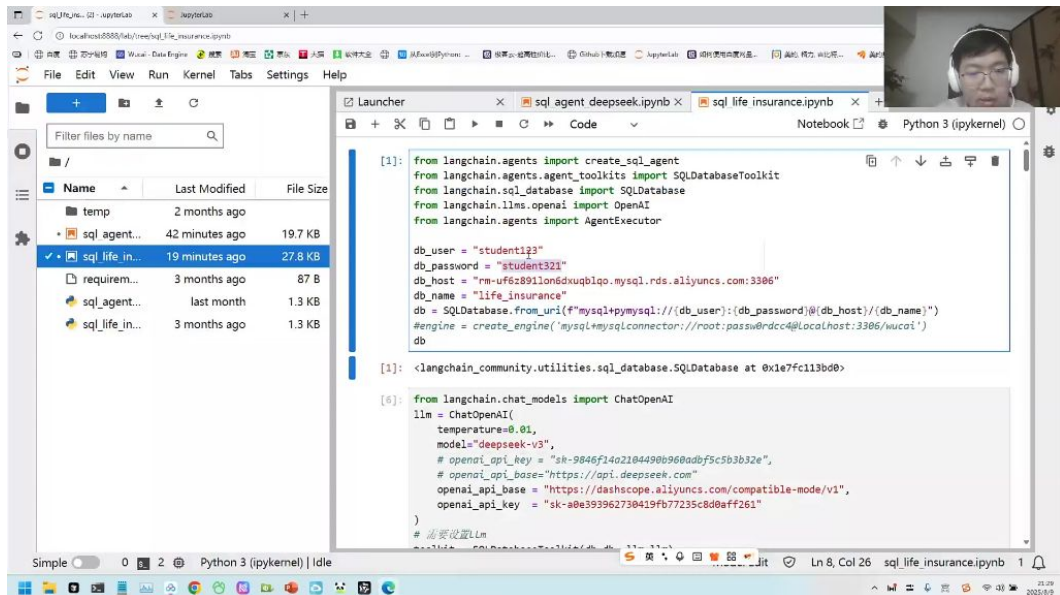
- 代码连接：
 - 参数配置：需要提供用户名(student123)、密码(student321)、主机地址(rm-uf6z81lon6dxuqblqo.mysql.rds.aliyuncs.com:3306)和数据库名(life_insurance)
 - 连接URI：格式为
mysql+pymysql://[db_user]:[db_password]@[db_host]/[db_name]
 - 工具支持：使用LangChain的SQLDatabase工具类进行封装
- 可视化工具连接：
 - 推荐工具：Navicat、MySQL Workbench等图形化工具
 - 连接步骤：新建连接→输入主机IP→配置端口(3306)→填写用户名密码→测试连接

3) 数据库连接演示 01:41:58



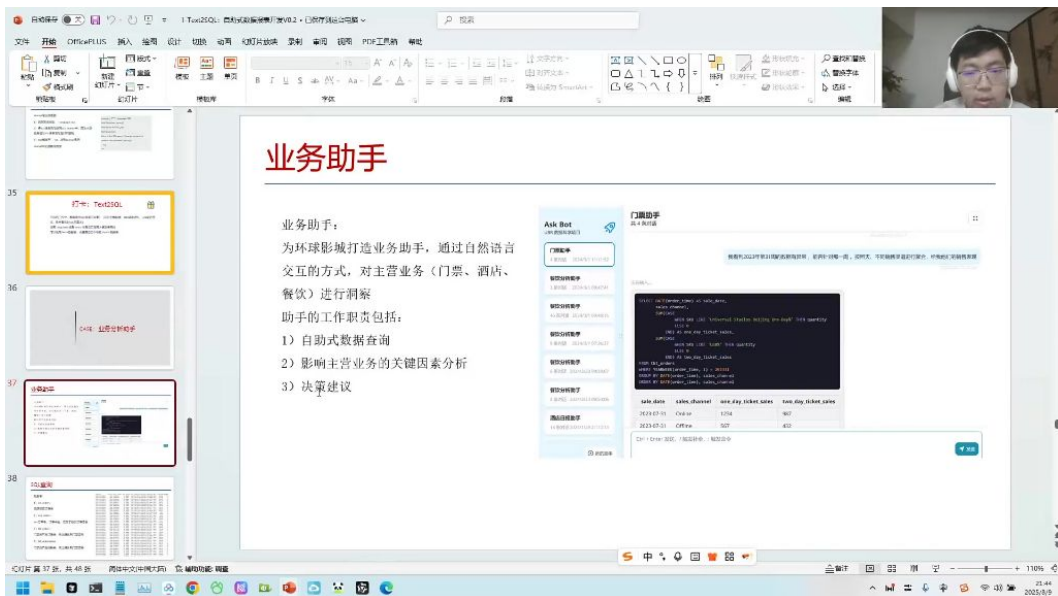
- 操作要点：
- 创建新连接时命名为"test"
- 主机地址需去掉端口号
- 使用相同用户名密码(student123/student321)
- 连接成功后双击查看数据表

4) 解析与权限管理 01:44:00



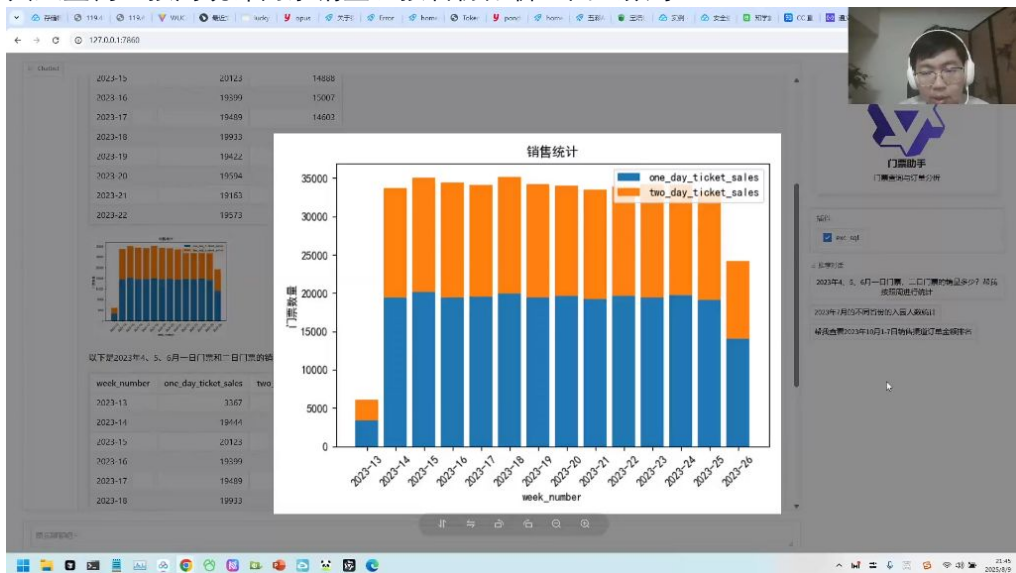
- 解析优化：
- 问题：LLM返回文本可能包含非标准SQL语法
- 解决方案：使用正则表达式提取.*?之间的标准SQL语句
- 性能指标：平均响应时间4-5秒，与prompt设计相关
- 权限管理：
- 数据库层面：MySQL通过用户-数据表权限矩阵控制访问
- 向量数据库：通过metadata中的标签系统实现，包含文件名、创建时间、作者等信息

5) 数据库与向量数据库的权限管理 01:48:04



● 系统功能：

- 核心能力：自然语言转SQL查询主营业务数据(门票/酒店/餐饮)
- 可视化支持：自动生成统计图表
- 典型查询：按周统计门票销量、按省份分析入园人数等



● 实现流程：

- 用户输入自然语言问题
- LLM生成标准SQL查询
- 执行查询获取结果
- 调用可视化工具生成图表
- 返回图文结合的分析报告

● 技术亮点：

- 支持多轮细化查询（如从周数据钻取到日数据）
- 自动识别数据异常点
- 提供业务建议模板

二、Text2SQL 01:58:22

1. 数据表展示

SQL查询



数据表:

1) crs_orders

酒店预定订单表

2) eco_orders

eco订单表, 订单中台, 汇总了各类订单信息

3) tkt_orders

门票类产品订单表, 包含具体到门票票号

4) tkt_redemption

门票类产品核销表, 包含具体到门票票号

order_time	account_x	gov_id	gender	age	province	SKU	product_ser	eco_main_order_id	sales_channel	status	order_value	quantity
2023-01-01 00:00:27	10457	100888	女	43	湖南省	Unia	NHK-402-7	ANVR-099	B2C_UBRAF	active	1898.86	9
2023-01-01 00:03:30	10952	784162	男	17	北京市	Unia	WJH-718-9	CATH-091	B2C_UBRAF	active	4216.64	3
2023-01-01 00:05:55	10923	982708	女	45	湖南省	Unia	WNC-477-4	OTM-015	B2B_OTA	active	1983.44	9
2023-01-01 00:09:12	10263	948668	男	45	北京市	Unia	LQW-579-1	MYCM-964	B2B_OTA	active	1255.06	6
2023-01-01 00:12:43	10147	568609	女	33	江苏省	Unia	ZVG-281-4	KXGS-325	B2C_UBRAF	active	3056.31	6
2023-01-01 00:14:35	10467	803232	女	22	江苏省	Unia	CWG-889-4	UUREC-000	B2B_OTA	active	3408.77	6
2023-01-01 00:16:16	10952	508482	男	22	北京市	Unia	JSC-316-8	TVQJ-0113	B2B_OTA	active	891.45	4
2023-01-01 00:18:54	10002	0319C4	男	30	上海市	Unia	RAW-564-1	RQAFV-791	B2B_OTA	Void	3497.97	10
2023-01-01 00:20:41	10862	59383A	男	32	山东省	Unia	AQS-640-3	VDREV-6972	B2B_OTA	active	4898.33	5
2023-01-01 00:21:04	10356	C05610	女	29	北京市	Unia	KKG-127-05	ARJPN-011	B2C_UBRAF	active	1993.77	3
2023-01-01 00:21:24	10517	F80958	女	21	北京市	Unia	SZE-339-06	JVPRC-2071	B2B_OTA	Full ref	383.2	8
2023-01-01 00:24:25	10079	AED063	男	12	北京市	Unia	BZM-310-1	JNPBT-815	B2B_OTA	Inactive	903.05	5
2023-01-01 00:29:26	10046	BE3D4D	女	35	天津市	Unia	XVE-248-17	XXHQ-400	B2B_OTA	active	792.23	4
2023-01-01 00:31:59	10675	A00560	女	13	河南省	Unia	RFG-213-74	KCBZ-4892	B2B_OTA	Void	1540.06	4
2023-01-01 00:32:44	10992	OC0891	男	8	上海市	Unia	LSM-527-72	AWSSW-04C	B2B_OTA	active	2477.56	6
2023-01-01 00:34:02	10597	19A289	男	47	天津市	Unia	FCJ-423-68	NOFTC-003	B2B_OTA	active	1127.7	9
2023-01-01 00:35:14	10406	351DC1	男	35	北京市	Unia	ZJH-681-76	QNHOL-554	B2C_UBRAF	active	3055.89	1
2023-01-01 00:36:07	10801	D4A42E	女	26	北京市	Unia	SCA-343-51	ANRFL-234	B2B_OTA	active	1426.91	1
2023-01-01 00:40:21	10570	39578D	男	34	山东省	Unia	SSW-836-4	YJBB-1875	B2B_OTA	active	4157.96	6
2023-01-01 00:41:02	10095	583A6A	女	26	北京市	Unia	HRM-773-3	KMTH-363	B2B_OTA	active	3809.84	3
2023-01-01 00:41:51	10301	837C11	女	42	河北省	Unia	WSM-995-0	QPRR-101	B2B_OTA	active	631.58	9
2023-01-01 00:45:09	10550	355632	男	16	辽宁省	Unia	EXT-473-30	ACORW-92	B2B_OTA	Inactive	3018.57	1
2023-01-01 00:47:53	10739	99F5EE	男	31	江苏省	Unia	MPQ-769-9	GNIS-2231	B2B_OTA	active	2487.56	1
2023-01-01 00:49:03	10647	Z7123A	女	48	山东省	Unia	SSE-104-21	NTW8-205	B2B_OTA	active	1049.62	7
2023-01-01 00:49:20	10626	ETAZ53	男	42	北京市	Unia	XVZ-531-55	PSOZT-114	B2C_UBRAF	active	4498.59	2
2023-01-01 00:51:19	10434	60735A	女	35	浙江省	Unia	RSE-313-69	STKGD-751	B2B_OTA	active	3331.62	7
2023-01-01 00:52:58	10358	03A1D1	女	39	广东省	Unia	CWA-936-9	ICCDU-314	B2B_OTA	active	1780.02	4
2023-01-01 00:53:53	10569	EEF83B	女	44	吉林省	Unia	RSG-139-6	GOQKV-82	B2B_OTA	active	2957.08	5
2023-01-01 00:58:27	10606	855599	男	21	上海市	Unia	DOK-618-7	BKQZ-739	B2B_OTA	Void	4038.16	2
2023-01-01 01:00:10	10979	3A2768	男	44	北京市	Unia	WMB-530-7	CVMB-6646	B2B_OTA	active	1255.21	6
2023-01-01 01:00:11	10362	4A0674	女	44	江苏省	Unia	OSH-459-4	PGNDN-06	B2B_OTA	active	3135.67	10
2023-01-01 01:03:27	10880	F3A0CB	女	30	山东省	Unia	PHH-624-4	ETTGQ-452	B2B_OTA	Upplw	2911.14	1
2023-01-01 01:04:35	10130	G9K0D7	女	61	福建省	Unia	C11-719-93	MTLNU-086	B2B_OTA	active	4293.75	7
2023-01-01 01:04:59	10987	ETC065	女	35	河北省	Unia	KMS-615-5	JRWAN-99	B2C_UBRAF	active	1183.49	2
2023-01-01 01:05:04	10005	A0E391	男	33	山东省	Unia	BAH-867-9	ROSES-711	B2B_OTA	System	2036.49	3
2023-01-01 01:05:26	10660	89E583	女	4	四川省	Unia	ZBS-163-03	YZDZ-692	B2B_OTA	Inactive	4142.83	6
2023-01-01 01:08:33	10572	6F8117	女	30	浙江省	Unia	XQT-848-4	IOAMX-864	B2B_OTA	active	2956.52	7



1) 数据表结构 02:00:35

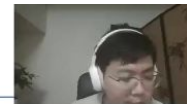
● 表类型: 课程展示了四种订单数据表: 酒店订单表(crs_orders)、生态订单表(eco_orders)、门票订单表(tkt_orders)和门票核销表(tkt_redemption)

● 字段说明:

- order_time: 订单日期时间
- account_id: 预定用户ID
- gov_id: 商品使用人身份证号
- gender: 使用人性别
- age: 年龄
- province: 使用人省份
- SKU: 商品SKU名称
- product_serial_no: 商品ID
- eco_main_order_id: 主订单ID
- sales_channel: 销售渠道(如B2C_UBRAF、B2B_OTA等)
- status: 订单状态(active/void等)
- order_value: 订单金额
- quantity: 商品数量

2) 门票助手系统提示设置

SQL查询



Thinking: 门票助手的system_prompt如何设置?

Thinking: 设置system_prompt的目的是什么?

```
system_prompt = """我是环球影城门票助手，以下是关于门票订单表相关的字段，我可能能够帮你编写SQL查询。

-- 门票订单表
CREATE TABLE tkt_orders (
    order_time DATETIME, -- 订单日期
    account_id INT, -- 绑定用户ID
    gov_id VARCHAR(18), -- 商品使用人ID (身份证号)
    gender VARCHAR(10), -- 使用人性别
    age INT, -- 年龄
    province VARCHAR(30), -- 使用人省份
    sku VARCHAR(100), -- 商品SKU名
    product_serial_no VARCHAR(30), -- 商品ID
    eco_main_order_id VARCHAR(20), -- 订单ID
    sales_channel VARCHAR(20), -- 销售渠道
    status VARCHAR(30), -- 商品状态
    order_value DECIMAL(10,2), -- 订单金额
    quantity INT -- 商品数量
);

-- 一日门票，对应多种SKU:
Universal Studios Beijing One-Day Dated Ticket-Standard
Universal Studios Beijing One-Day Dated Ticket-Child
Universal Studios Beijing One-Day Dated Ticket-Senior
-- 二日门票，对应多种SKU:
USB 1.5-Day Dated Ticket Standard
USB 1.5-Day Dated Ticket Discounted
-- 一日门票，二日门票查询
SUM(CASE WHEN SKU LIKE 'Universal Studios Beijing One-Day%' THEN quantity ELSE 0 END) AS
SUM(CASE WHEN SKU LIKE 'USB%' THEN quantity ELSE 0 END) AS two_day_ticket_sales
我帮助回答用户关于门票相关的问题"""
```

- - 系统角色定义:
 - 角色设置为"环球影城门票助手"
 - 主要功能是回答门票相关问题并编写对应SQL查询
 - 表结构定义:
 - SKU分类:
 - 一日门票: Universal Studios Beijing One-Day Dated Ticket-Standard/Child/Senior
 - 二日门票: USB 1.5-Day Dated Ticket Standard/Discounted
 - 聚合查询示例:
- 3) 业务应用场景
- 目标用户: 主要面向BI部门和业务部门使用
 - 传统流程: 业务人员需要查询数据时需找技术导师协助
 - 改进方案: 通过大模型实现自助查询，处理简单灵活的任务
 - 技术实现: 采用流式输出方式，前端分段捕获生成内容

2. 门票助手系统开发

-
- 核心功能: 通过自然语言处理生成SQL查询语句
- 数据表结构:

1) 门票类型SKU配置 02:01:57

- 一日门票SKU:
 - Universal Studios Beijing One-Day Dated Ticket-Standard
 - Universal Studios Beijing One-Day Dated Ticket-Child
 - Universal Studios Beijing One-Day Dated Ticket-Senior
- 二日门票SKU:
 - USB 1.5-Day Dated Ticket Standard
 - USB 1.5-Day Dated Ticket Discounted
- 查询技巧: 使用CASE语句统计不同类型门票销售数量

2) 系统提示词设计

- prompt结构:
- 设计要点:
 - 明确身份定位
 - 包含完整数据结构
 - 说明功能范围
 - 使用专业术语 (如"1.5-Day"代替"两日")

3. SQL生成方法对比

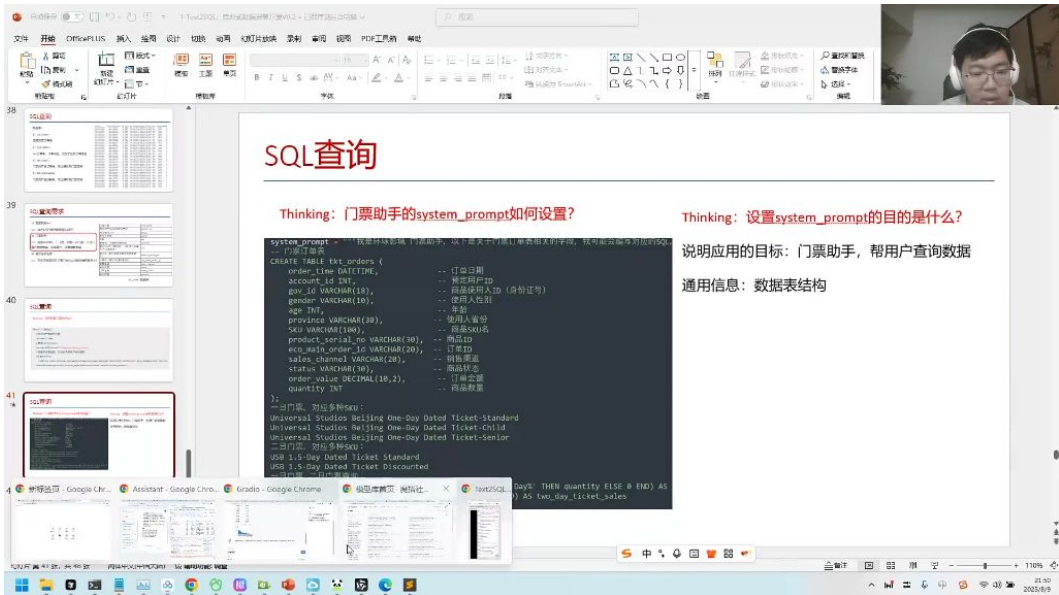
The screenshot shows a presentation slide titled "SQL查询". The slide content includes a thinking process and a JSON schema for a function named "exc_sql".

Thinking: function tool如何设置? 比如针对生成的SQL语句, 进行SQL查询

```

functions_desc=[
{
  "name": "exc_sql",
  "description": "对于生成的SQL, 进行SQL查询",
  "parameters": {
    "type": "object",
    "properties": {
      "sql_input": {
        "type": "string",
        "description": "生成的SQL语句",
      },
    },
    "required": ["sql_input"],
  },
},
]
  
```

- 方法一: 分步生成
 - 流程:
 - 生成SQL语句
 - 解析SQL语句
 - 执行SQL查询
 - 优点: 精准度高
 - 缺点: 需要两次交互, 速度较慢
 - 2) 方法二: 直接执行
 - 流程:
 - 生成并直接执行SQL
 - 优点: 响应速度快
 - 缺点: 错误处理难度大
 - 实现代码:
- ## 4. 实践案例演示



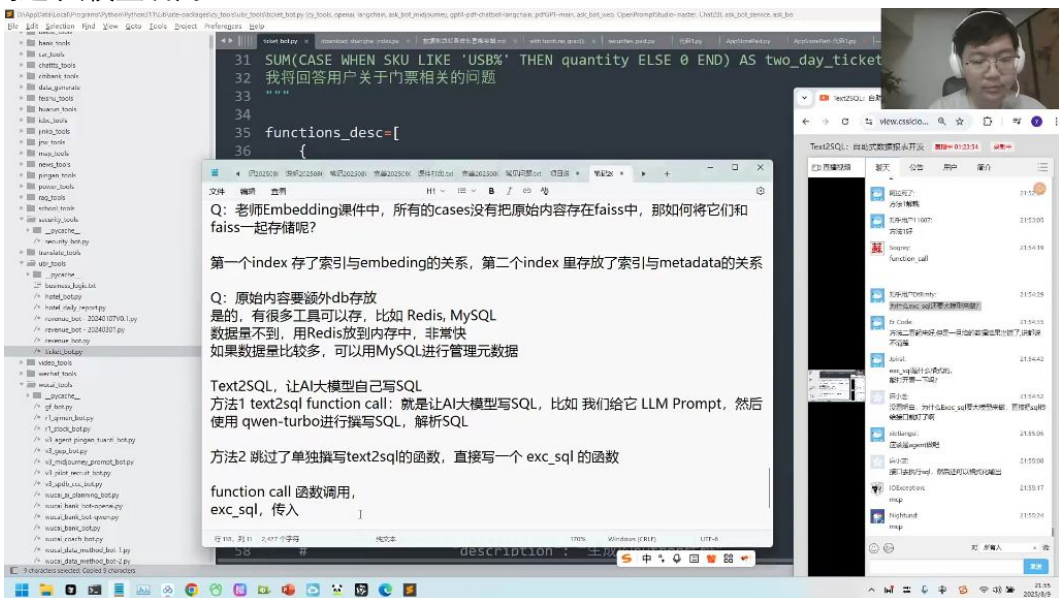
-
- 典型查询:
 - "2023年4、5月不同省份入园人数统计"
 - "2023年10月1-7日销售渠道订单金额排名"
- 结果展示:
 - 北京市入园人数最多
 - 其次是江苏、河北和山东
 - 支持进一步数据可视化

1) 技术要点

- 数据库连接:
- 元数据管理:
 - 小数据量使用Redis
 - 大数据量使用MySQL
 - 建立索引与embedding的关系表

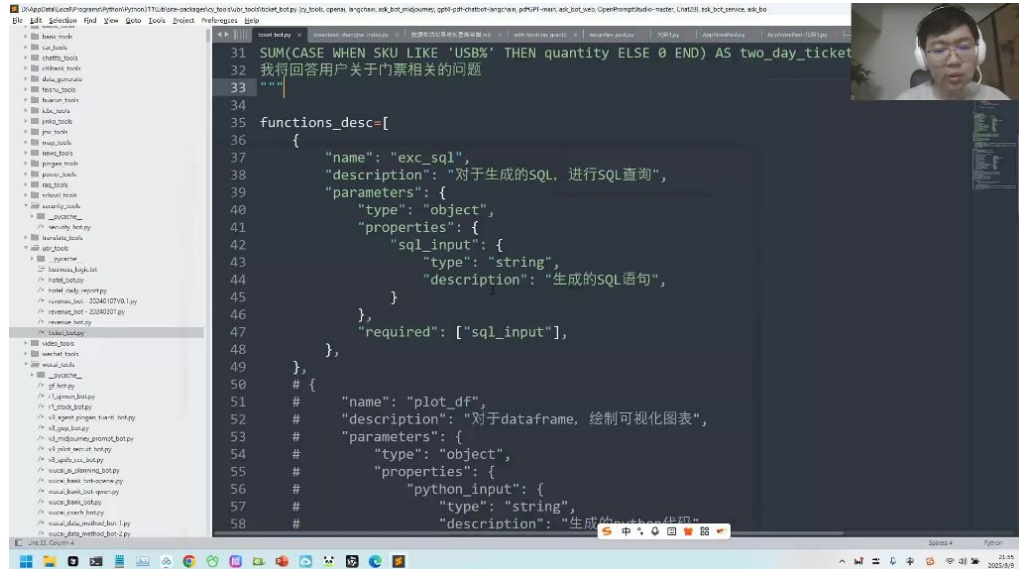
5. 函数调用能力 02:08:41

1) 例:大模型调用SQL 02:11:10

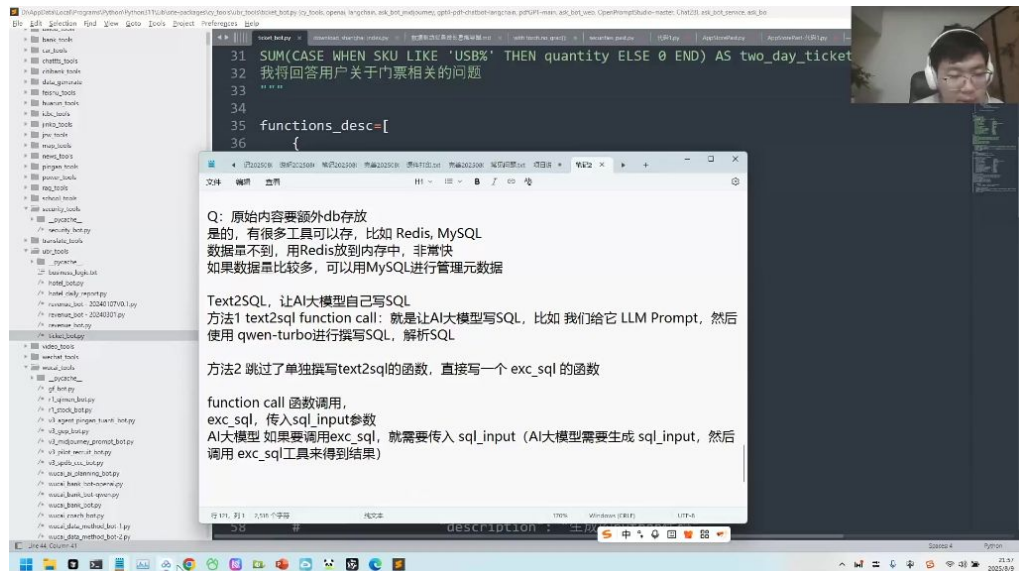


-
- 方法1: text2sql function call
 - 通过LLM Prompt让AI大模型 (如qwen-turbo) 撰写SQL并解析

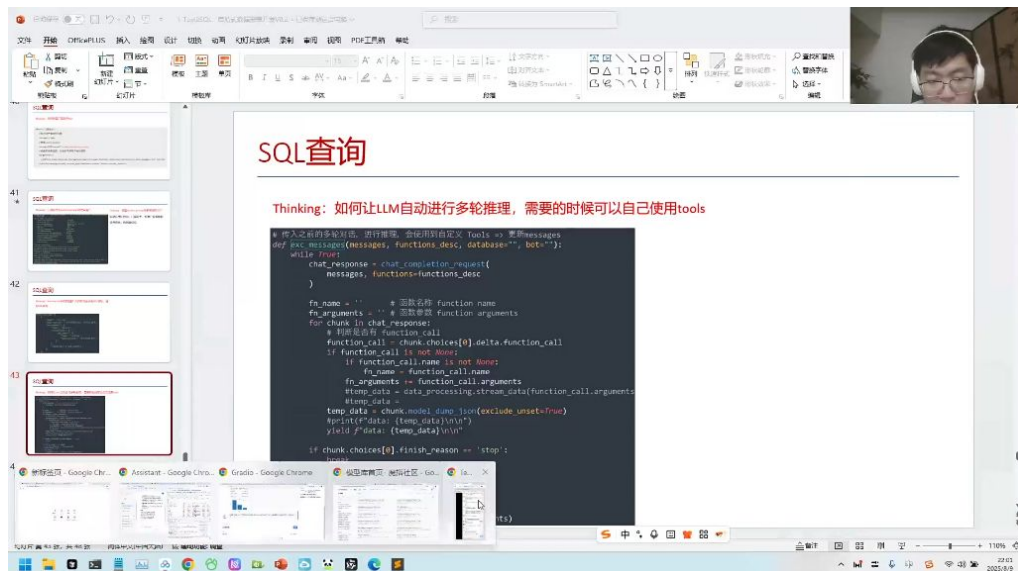
- 优点：提供更多调试空间
- 缺点：需要单独撰写`text2sql`函数
- 方法2：直接调用`exc_sql`函数
 - 跳过单独撰写`text2sql`函数
 - 优点：流程更简洁
 - 缺点：出错时调试信息较少



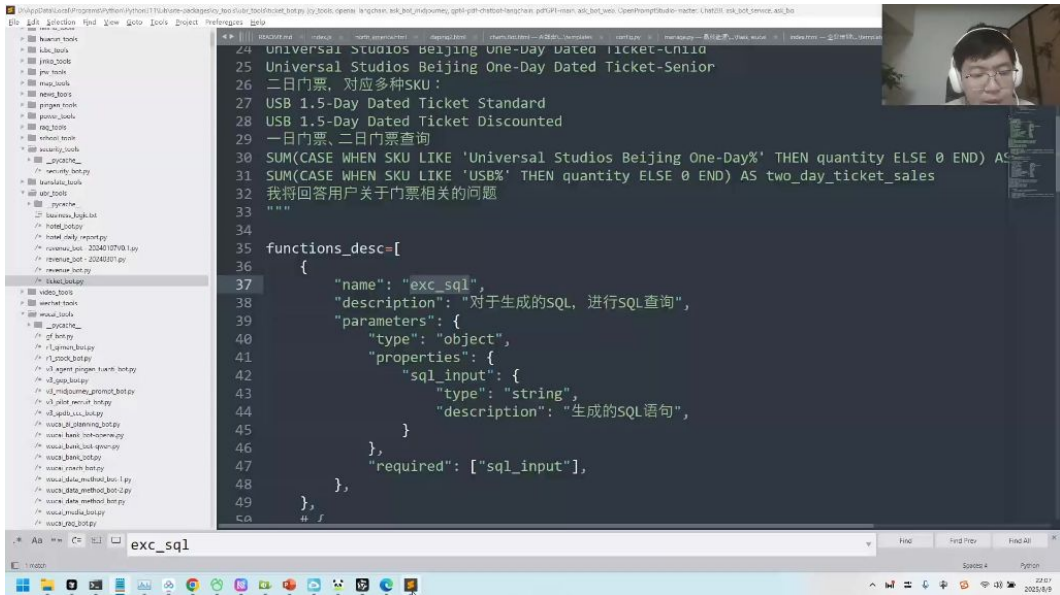
- 函数定义：
- 实现原理：
 - 输入输出定义：严格定义`sql_input`参数和输出结果
 - 工具调用：大模型生成SQL语句后调用预定义工具执行
 - 数据管理：小数据量用Redis（内存存储），大数据量用MySQL管理元数据



- 存储方案选择：
 - Redis：适合数据量小的场景，内存存储速度快
 - MySQL：适合数据量大的场景，便于元数据管理
- 实现要点：
 - 精准SQL生成：需提供准确的metadata、数据表定义和术语说明
 - 角色说明：明确告知模型需要完成的任务
 - 数据导入：可将Excel数据导入SQLite等轻量数据库便于查询

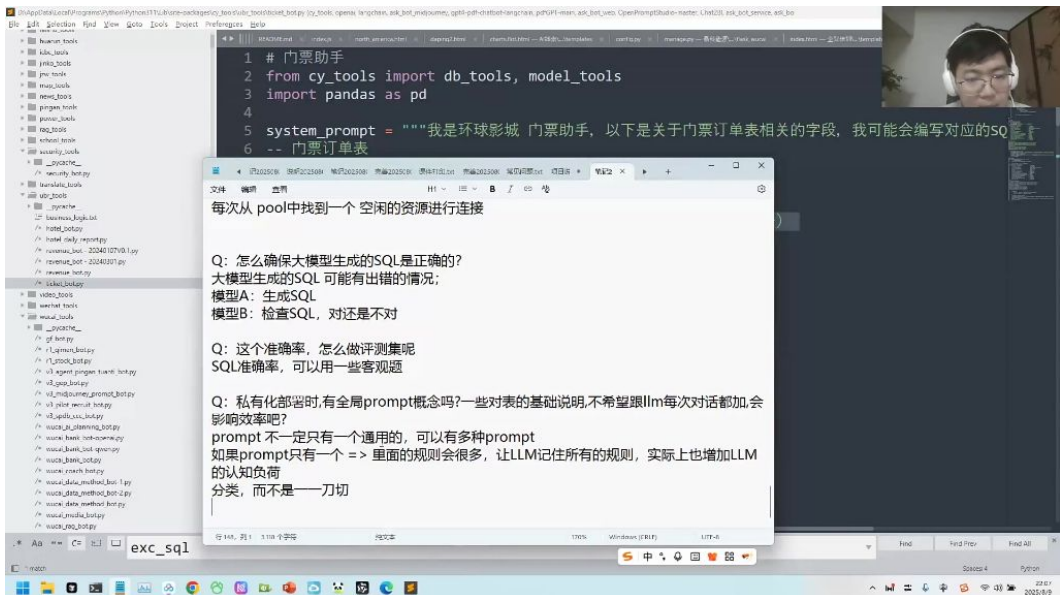


-
- 多轮对话处理:
- 常见问题解答:
 - 为什么需要大模型生成SQL: 大模型负责理解需求并生成符合语法的SQL语句
 - Excel数据处理: 建议将数据导入SQL数据库 (如SQLite) 再通过SQL查询
 - 调试建议: 方法1更适合需要详细调试的场景
- 6. 数据库连接池 02:15:16
 - 资源管理机制: 通过设置固定数量的连接资源 (如20个分库连接) 来管理系统访问压力
 - 工作原理:
 - 采用队列机制处理请求
 - 有空闲连接时立即分配使用
 - 无空闲连接时请求进入等待状态
 - 类比示例: 类似机场值机柜台, 20个服务窗口对应20个连接资源, 乘客 (请求) 需要排队等候可用窗口
- 7. SQL生成的准确性 02:17:59
 - 1) 大模型生成特性
 - 固有缺陷: 所有大模型都存在准确率问题, 生成内容需人工复核
 - 错误类型: 可能产生语法错误或逻辑错误的SQL语句
 - 适用场景: 简单任务准确率较高, 复杂任务需特别检查
 - 2) 准确性保障策略
 - 交叉验证法:
 - 使用模型A生成SQL
 - 用模型D进行校验评分
 - 检查难度低于生成难度, 更易发现错误
 - 客观评测法:
 - 利用SQL执行结果的唯一性
 - 预先保存标准答案
 - 对比生成SQL的执行结果进行验证
- 8. 全局prompt 02:19:29
 - 1) 分类设计原则

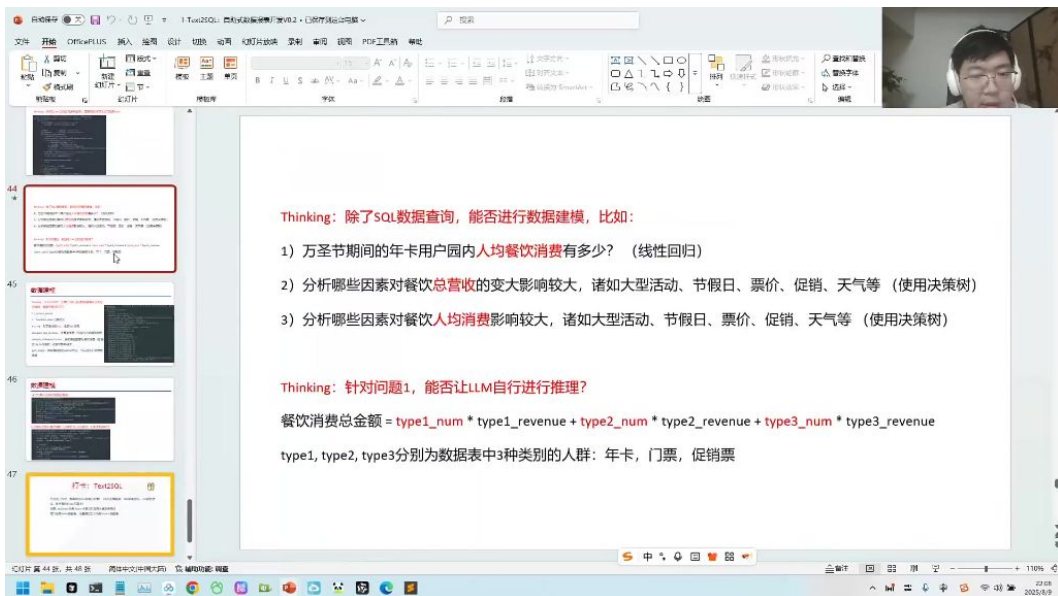


- 分类必要性: 企业数据表数量庞大, 不能将所有表给到单一prompt, 需按业务类型分类 (如门票、餐饮、酒店)
- 层级划分: 大类下可继续细分 (如门票类可再分为核销、业务查询等子类)
- 认知负荷控制: 单一prompt包含过多规则会增加LLM记忆负担, 分类可降低认知复杂度

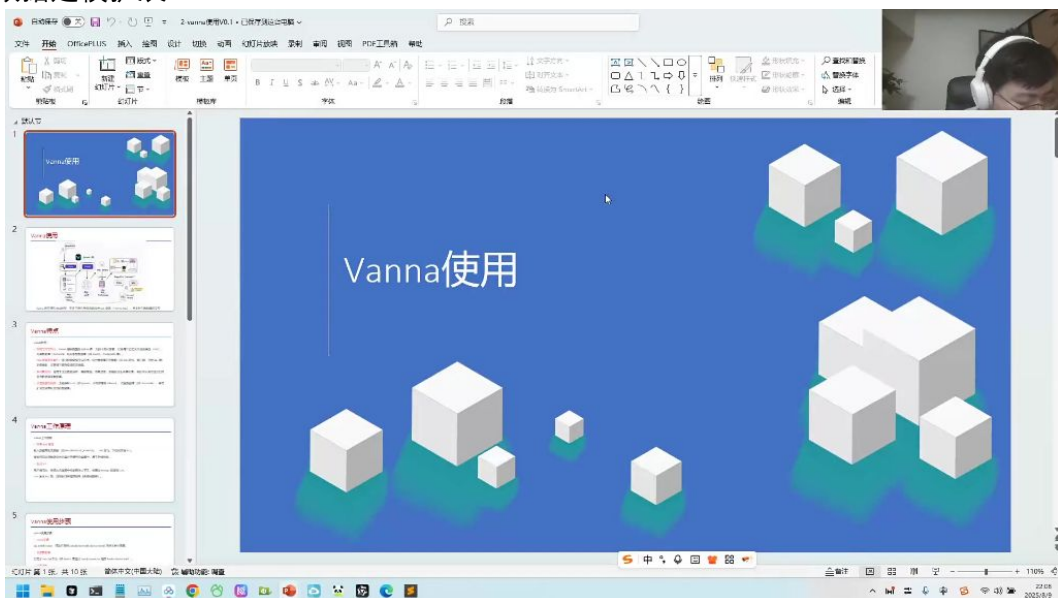
2) 门票助手实现案例



- 角色定义: 通过system_prompt明确LLM作为"环球影城门票助手"的角色定位
 - 字段说明: 提供门票订单表相关字段定义, 辅助SQL生成
 - 模块化设计: 通过db_tools和model_tools实现数据库连接和模型调用分离
- ## 3) SQL生成质量控制



- - 双模型校验:
 - 模型A负责生成SQL
 - 模型B负责校验SQL正确性
 - 评测方法:
 - 使用客观题构建评测集
 - 通过准确率指标量化效果
 - 资源管理: 采用连接池机制, 从pool中获取空闲资源执行查询
- 4) 数据建模扩展



-
- 线性回归应用: 如计算万圣节期间年卡用户园内人均餐饮消费
- 决策树分析:
 - 识别影响餐饮总营收的关键因素 (大型活动、节假日等)
 - 分析餐饮人均消费影响因素
- 公式推导: 消费总金额 = $type1_num \times type1_revenue + type2_num \times type2_revenue + type3_num \times type3_revenue$

三、Vanna介绍02:21:59 <https://vanna.ai/>

1. vanna使用



- 作为 SQL 查询 (Text-to-SQL) 并支持与

na特点 02:23:04

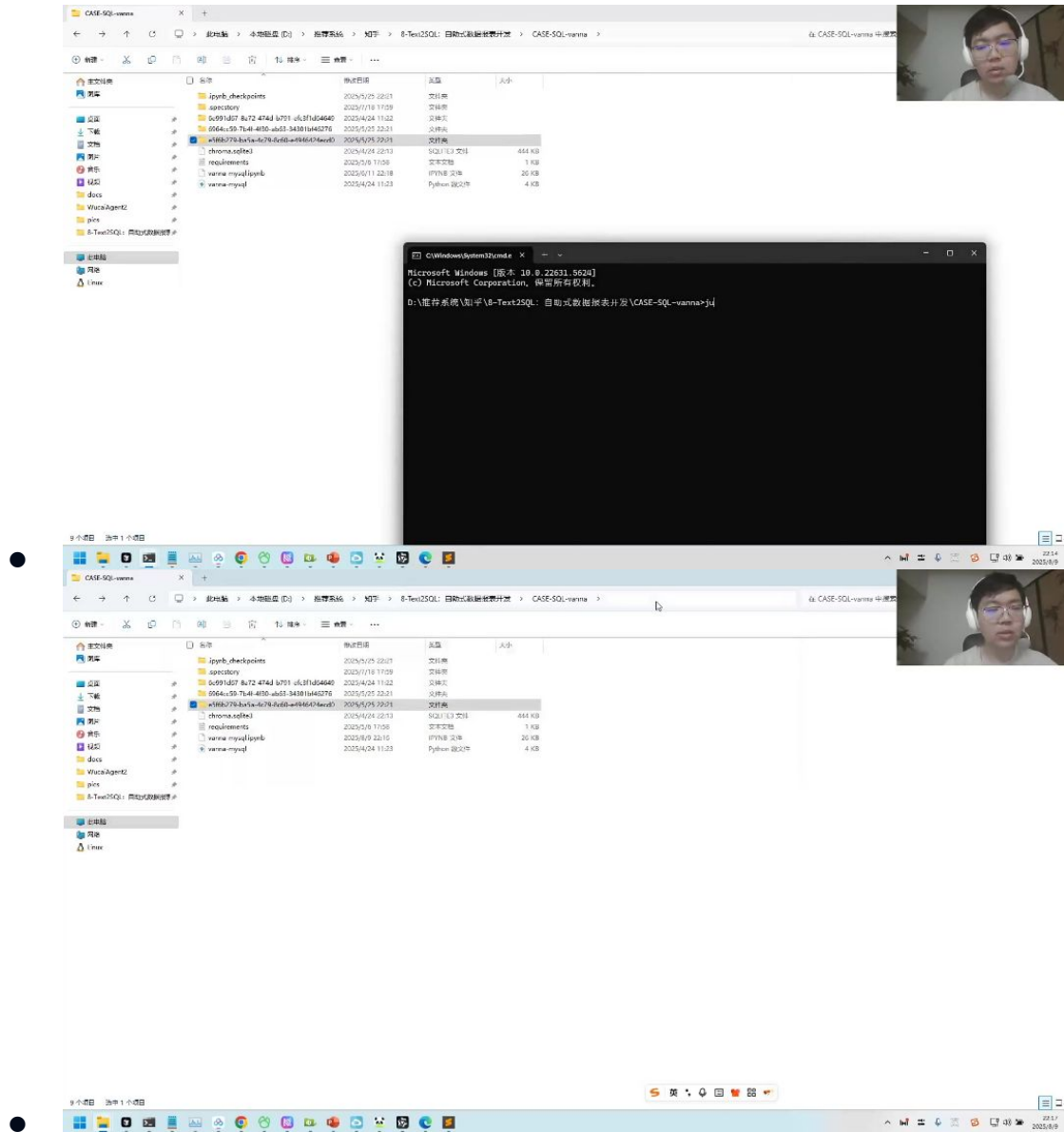
- 开源与定制化：
 - 提供完整Python库
 - 支持本地化部署
 - 可自定义LLM、向量数据库和关系型数据库（MySQL/PostgreSQL等）
- AI增强：
 - 结合数据库元数据（DDL语句、表注释、示例SQL）训练模型
 - 显著提升复杂查询准确率
- 多场景支持：
 - 企业数据分析
 - 智能客服
 - 电商搜索
 - 金融报告生成
- 基础设施兼容性：
 - 支持多种LLM（OpenAI、Ollama等）
 - 支持多种向量数据库（ChromaDB等）
 - 可扩展至非默认支持的数据库

na工作原理 02:24:33

- 训练机制：**
通过**train函数**将元数据存入**向量数据库**
可接受DDL语句、文档和SQL示例
- 生成流程：**
从向量数据库检索相关内容
将检索结果喂给大模型生成SQL

- 自动执行SQL并返回结果
- 可视化功能：
 - 内置图表样式库
 - 可自动生成数据可视化图表
- API使用：
 - 主要入口为ask函数（如ask("查询销售额最高的产品")）
 - 工作流包含SQL生成、执行、画图等模块

1) 例题:vanna使用 02:28:00

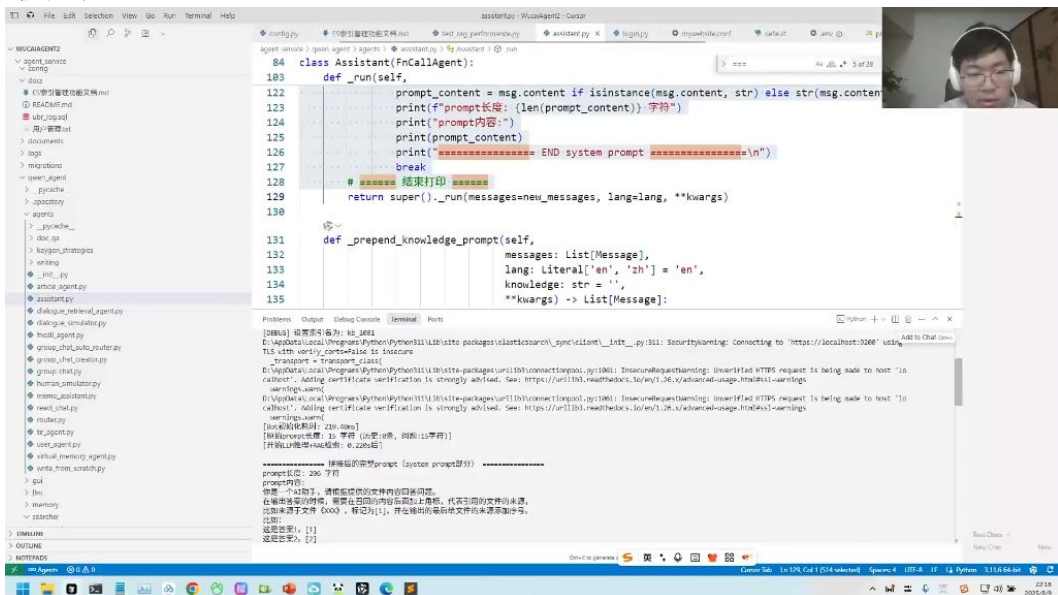


- 环境配置要点：
 - 需要安装额外扩展包（如chromadb、vanna-mysql等）
 - 推荐使用OpenAI模型（对国内模型支持有限）
- 常见问题：
 - 需替换OpenAI API key和模型参数
 - 可能遇到包依赖冲突问题
- 调试建议：
 - 检查向量数据库连接
 - 验证模型API可用性

- 确保训练数据格式正确

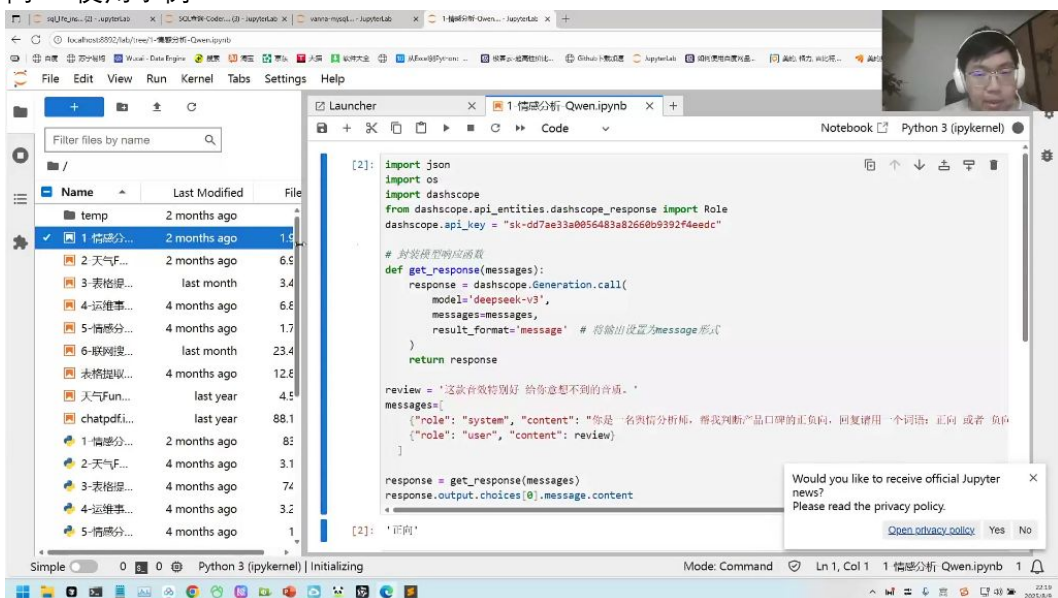
四、问题解答02:31:47

1. API使用演示



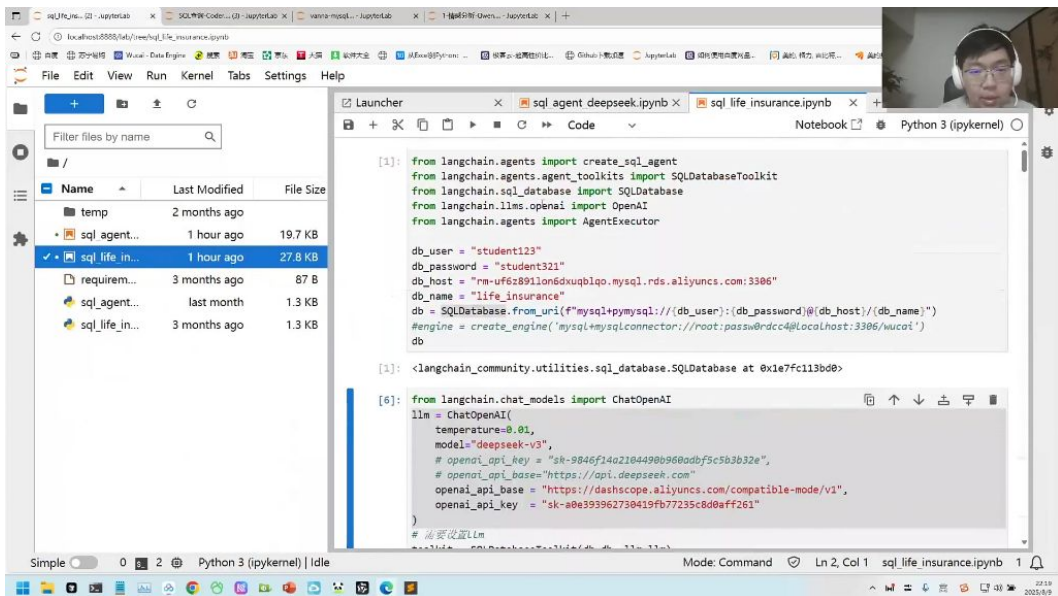
- API支持情况：当前演示环境可能不完全支持千问API，但可以通过其他方式实现类似功能
- 代码获取方式：第一次课程中已提供相关示例代码，学员可自行运行测试

2. 千问API使用示例



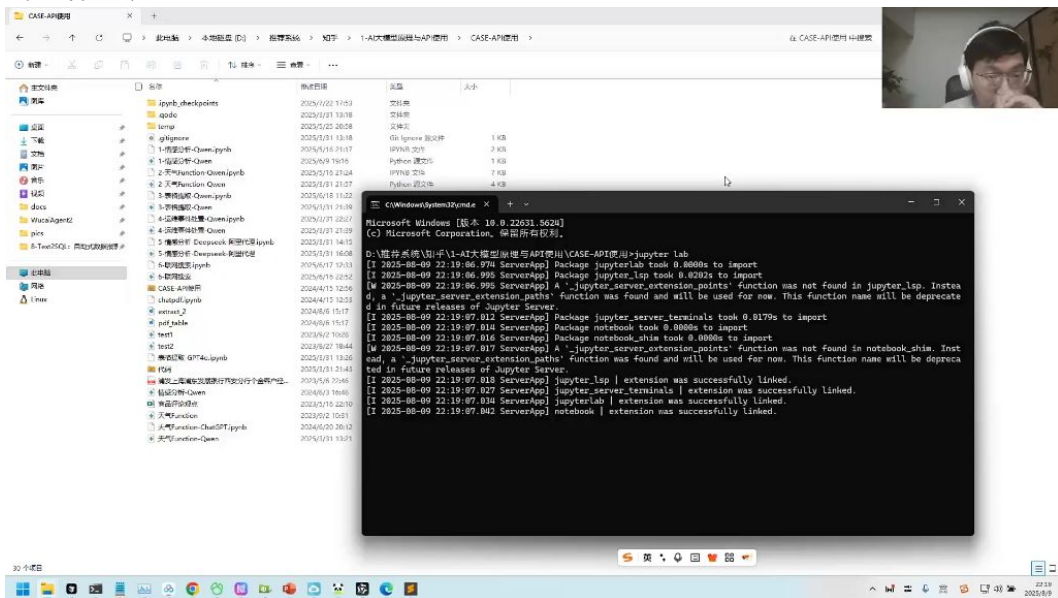
- 代码结构：
 - 核心模块：使用dashscope库调用API
 - 参数设置：需配置api_key="sk-dd7ae33a0056483a82660b9392f4eedc"
 - 模型调用：通过Generation.call()方法调用deepseek-v3模型
- 功能实现：示例展示情感分析功能，输入评论文本返回"正向"或"负向"判断

3. API密钥管理



- 密钥失效处理：当API key失效时需要更换新密钥
- 密钥获取：可在之前课程代码中找到可用密钥
- 模型切换：演示了如何将模型切换为deepseek-v3进行测试

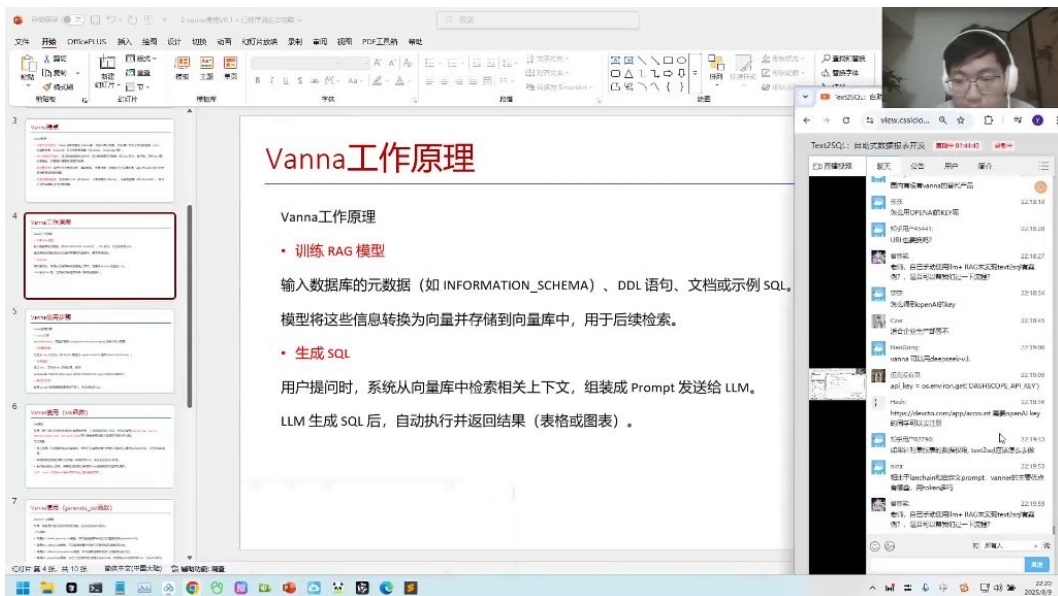
4. 代码运行建议



- 实践建议：
- 所有示例代码均可直接运行测试
- 包含情感分析、天气查询、表格提取等多种功能案例
- 建议按文件分类逐个测试不同功能模块

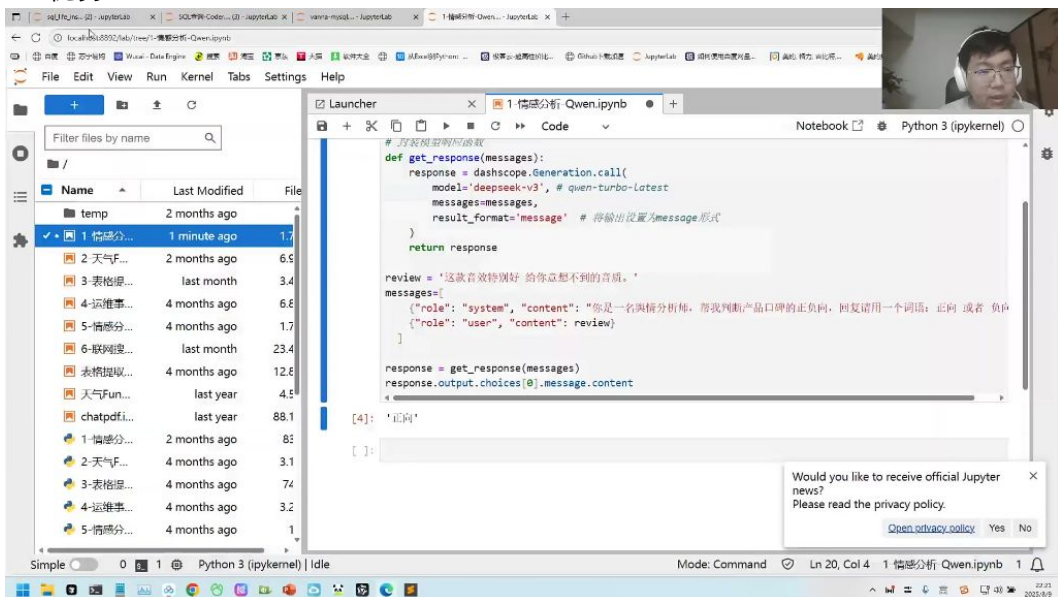
五、vanna使用详解02:33:54

1. 接口演示



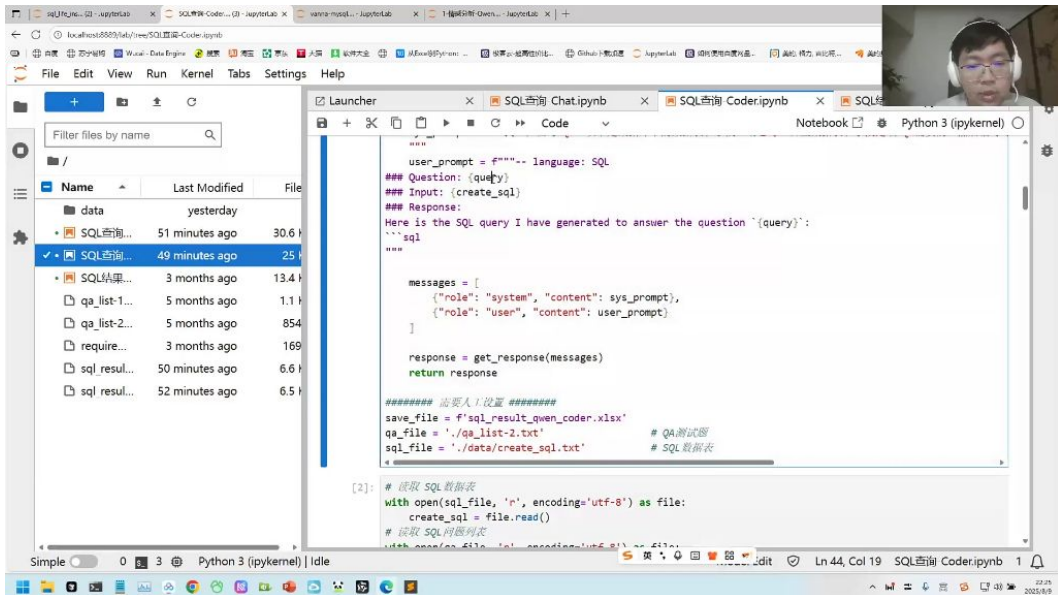
- 工作原理：
 - 输入数据库元数据（如`INFORMATION_SCHEMA`）、DDL语句、文档或示例SQL
 - 模型将信息转换为向量并存储到向量库
 - 用户提问时从向量库检索相关上下文，组装成Prompt发送给LLM
 - LLM生成SQL后自动执行并返回结果（表格或图表）
- API配置：
 - 需要`OPENAI_API_KEY`环境变量
 - 示例代码：`api_key = os.environ.get('OPENAI_API_KEY')`

2. vanna优势 02:35:57



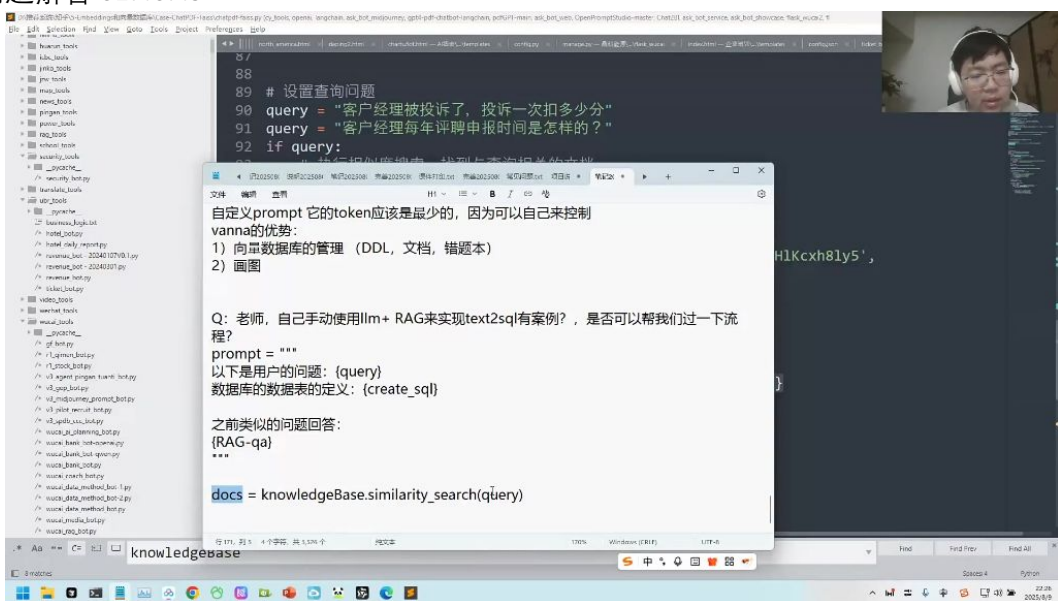
- 核心优势：
 - 向量数据库管理：支持添加DDL、文档、错题本等多种数据类型
 - 可视化能力：不仅能查询数据，还能自动生成多种样式的图表
 - token效率：相比LangChain，token使用量更可控
- 适用场景：
 - 企业生产环境
 - 需要可视化展示的报表开发
 - 需要管理大量数据库元数据的场景

3. 自己动手实现text2sql 02:37:04



- 实现步骤：
 - 构建Prompt模板：
 - 使用FAISS建立知识库存储历史案例
 - 通过相似度搜索获取相关文档
 - 组装Prompt发送给LLM
- 关键代码：

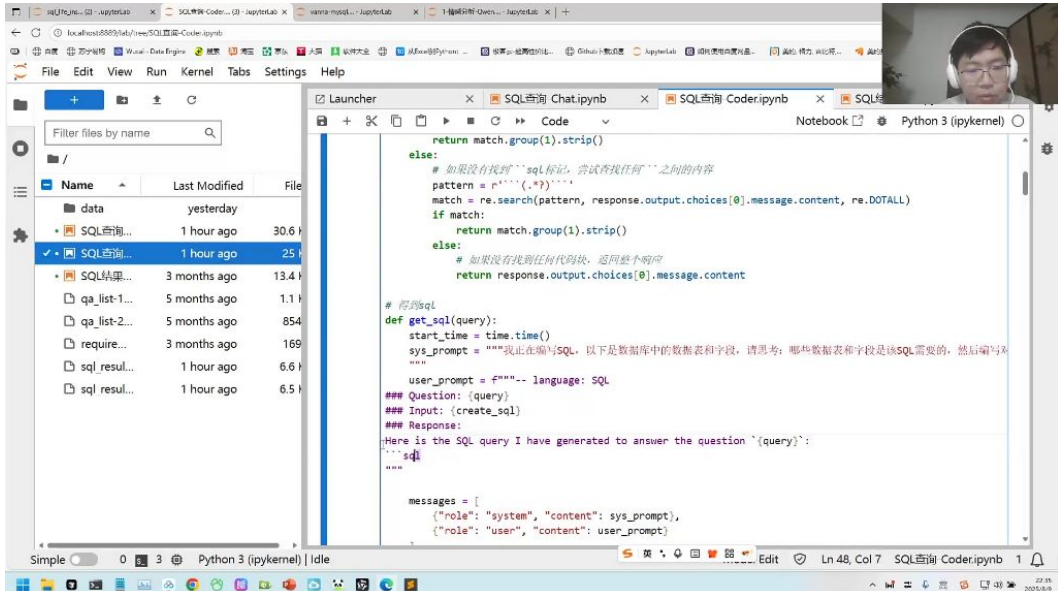
4. 问题解答 02:40:49



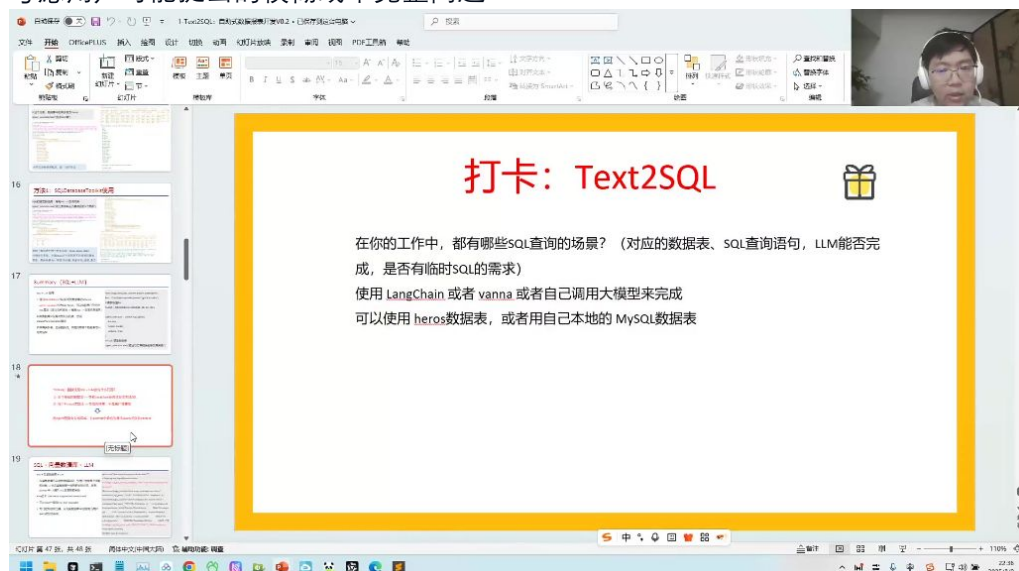
- 数据权限问题：
 - 表级权限：通过数据库GRANT命令实现
 - 行级权限：利用数据库原生行锁定功能
 - 示例：heroes表可通过授权控制查询/写入权限
- SQL安全隐患：
 - 使用SQLAlchemy等ORM工具避免原生SQL拼接
 - 实现双重审核机制：
 - 事前：让LLM判断query安全性（返回JSON格式的安全评估）
 - 事后：对生成SQL进行安全扫描

- 商业化落地建议：
 - 建立测试集作为金标准（如1000条测试query）
 - 与客户约定准确率预期（如95%）
 - 对不确定问题返回"无法确定"而非错误答案

六、内容总结02:49:30



- 三种实现方式：
 - LangChain：调用最方便，适合快速验证
 - 手动实现：灵活性最高，适合定制需求
 - Vanna：提供向量库管理和可视化两大核心优势
- 测试注意事项：
 - 确保测试集多样性
 - 包含边缘用例和异常情况
 - 考虑用户可能提出的模糊或不完整问题



- 实践作业：
 - 使用任意方法实现Text2SQL
 - 可选用heros数据表或本地MySQL
 - 提交运行截图和体验报告

七、知识小结

知识点	核心内容	技术要点	应用场景	难度系数
自助式报表开发	讲解test two circle自助报表开发，提供课件代码和数据下载	SQL自动生成、结构化数据处理	BI报表、数据查询	★★★
结构化vs非结构化数据	结构化数据(excel)格式固定价值高，非结构化数据(PDF/word)需解析	数据表关联、字段类型定义	数据仓库建设、报表生成	★★
大模型选择策略	对比Cloud/Gemini/千问/DeepSeek等模型代码生成能力	API调用与本地部署差异	企业级应用开发	★★★★
SQL生成三阶段演进	从模板规则→机器学习→大模型生成的技术发展路径	语义理解与模式关联	自然语言转查询	★★★
LangChain应用	SQL Agent工具实现多表联查，自动解析表结构	向量数据库检索、函数调用	快速原型开发	★★★★
提示词工程最佳实践	建表语句+字段注释+补全触发的高效写法	...SQL补全激发、元数据注入	精准SQL生成	★★★★
保险行业案例	7张数据表(客户/保单/理赔等)的复杂查询实践	子查询优化、多表关联	金融风控分析	★★★★
安全防护机制	SQL注入防范、权限管理、结果校验三重保障	连接池控制、行级权限	企业级部署	★★★★★
测试评估体系	通过测试集(1000+样例)验证准确率	错误案例归集分析	项目验收	★★★★
可视化集成	自动图表生成(柱状图/趋势图等)	数据透视、样式模板	决策看板	★★★

以下为AI生成的图文笔记的内容

一、Text2SQL自助式数据报表开发15:46

- 1. 课程安排
- 2. 结构化和非结构化数据 16:06
 - 1) 非结构化数据类型 16:11
 - 常见类型: 包括PDF、Word文档、图片（以图片形式存在的表格）等
 - 特点: 格式不固定，难以直接进行结构化查询和分析
 - 2) 结构化和非结构化数据的区别 16:32
 - 核心差异: 数据格式是否固定
 - 结构化示例: Excel表格具有固定的行列结构
 - 价值对比: 结构化数据更便于查询和分析，价值密度更高
 - 3) 结构化与非结构化数据库的争论与价值变化 16:57

- **技术阵营:**
 - SQL代表结构化数据阵营
 - NoSQL代表非结构化数据阵营
- **发展趋势:** 大模型出现前SQL数据库（如Oracle）价值更高；大模型出现后非结构化数据价值提升
- 4) 结构化数据的应用与优势 17:35
 - **运算能力:** 天生格式清晰，便于进行各种运算
 - **典型应用:** 生成数据看板和决策报表
 - **工作场景:** 业务人员日常需要频繁提取结构化数据
- 5) 大模型在结构化数据处理中的应用 18:00
 - **传统方式:** 由技术人员编写SQL语句
 - **痛点:** 业务人员提数需求频繁，打断技术人员核心工作
 - **解决方案:** 通过大模型自动生成SQL语句
- 6) 大模型写SQL语句的背景与需求 18:10
 - **历史尝试:** 曾培训业务人员学习SQL但效果有限
 - **变革契机:** 大模型出现后业务人员可直接用自然语言获取数据
 - **价值:** 解放技术人员，使其专注于更高价值的数据分析工作
- 7) Function calling功能与思考助手搭建 19:07
 - **关键技术:** 使用function calling功能连接大模型和数据库
 - **实现方式:** 基于此功能搭建SQL查询助手
 - **优势:** 实现自助式数据提取，减少工作打断
- 8) 大模型开发自入型数据报表的模式 19:51
 - **模式一:** 使用LangChain封装的SQL Agent工具
 - **模式二:** 自定义提示词工程，让大模型理解表结构和需求
 - **最佳实践:** 探索不同提示词写法对结果的影响
- 9) 例题1:保险场景下大模型生成SQL语句的实践 20:43
- 10) 结构化数据查询需求与大模型应用 20:58
 - **典型需求:** 查询特定日期点击量、月度投放金额等
 - **传统局限:** 每次新问题都需要重新计算
 - **大模型优势:** 可即时响应各种新查询需求
- 11) 文本转换成SQL语句的发展阶段 22:15
 - **第一阶段:** 人工预定义模板（规则有限，难以穷尽）
 - **第二阶段:** 机器学习方法（Sequence to Sequence模型）
 - **第三阶段:** 大模型时代（结合语言理解和代码生成能力）
- 3. Text to SQL技术发展 22:24
 - 1) 序列到序列 24:45
 - **技术起源:** 机器翻译领域
 - **工作原理:** 输入输出长度不固定，通过神经网络学习映射关系
 - **局限性:** 生成的SQL语句正确率不高，常需人工修改
 - 2) 大模型 24:59
 - **核心能力:**
 - 强大的自然语言理解能力
 - 专业的代码生成能力
 - **实现方式:** 通过提示词工程或微调数据
 - **处理流程:**
 - 理解用户查询意图
 - 关联数据库表结构
 - 匹配实体与字段

- 生成可执行SQL
- 返回查询结果
- 支持数据库: MySQL、PostgreSQL、Oracle、Hive等

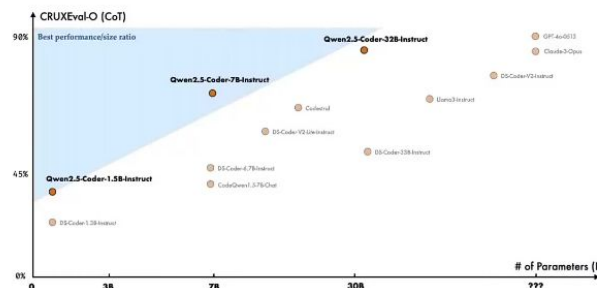
4. LLM模型选择 27:04

1) 代码大模型 34:04

LLM模型选择（代码大模型）

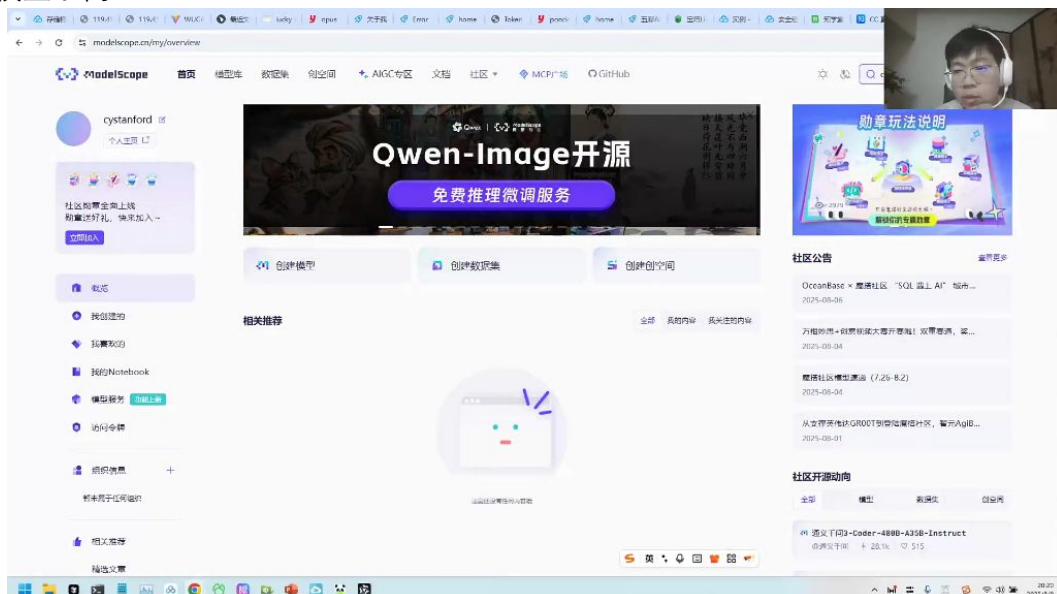


- Qwen-Coder: 能力强, 接近闭源一线大模型, 其中Qwen2.5-Coder-32B能力与GPT-4o持平
- CodeGeeX: 智谱开源的代码大模型, 基于GLM底座, 性能卓越, 在vscode等编辑器中可以找到对应的插件。
- SQLCoder: 专为 SQL 生成而设计的开源模型, 但是维护更新慢。
- DeepSeek-Coder: 在多种编程语言中与开源代码模型中实现了先进的性能



-
- **Qwen-Coder**: 能力接近闭源一线大模型, 其中Qwen2.5-Coder-32B与GPT-4o性能持平, 但部署成本高 (480B版本需要高性能机器)
- **CodeGeeX**: 智谱开源的基于GLM底座的模型, 性能卓越, 提供VS Code插件支持
- **SQLCoder**: 专为SQL生成设计的开源模型, 但维护更新较慢
- **DeepSeek-Coder**: 在多种编程语言中表现先进, 但模型更新不及时
- **部署现状**:
 - 企业多部署通用大模型 (如Qwen3-8B/32B)
 - 个人使用推荐闭源SaaS模型 (如豆包、Qwen系列)
 - 代码大模型实际应用较少, 主流仍是通用模型

2) 模型示例 35:15



-
- **国外模型**:
 - **Claude系列**: 最新4.1版本代码生成能力最强, 适合编程场景

- **GPT系列**: GPT-5理解能力存在争议, 图表处理仍有缺陷
 - **Gemini**: 即将推出3.0版本, 代码质量整体不错但有小瑕疵
 - **国内模型**:
 - **Qwen系列**: 当前主流, 含8B/32B等版本及MOE中量级模型
 - **DeepSeek**: 年初流行但换血频繁, 现多转向Qwen
 - **Code专用**: Qwen-Coder性能接近Claude4.0, DeepSeek-Coder使用较少
 - **部署方式**:
 - 闭源模型: 通过API按token计费 (如阿里云百炼)
 - 开源模型: 可本地部署 (如ModelScope下载Qwen/DeepSeek系列)
 - 企业首选开源模型以保证数据安全
5. 封闭式与开放式 36:41
- 1) SQL Agent方法 37:12
- **工具特点**
 - **封装方式**: 封装在LangChain工具集中, 可直接调用server agents模块
 - **应用案例**: 曾用于某银行"个金客户经理考核办法"项目, 采用LangChain+DeepS+Face框架搭建
 - **执行流程**: 通过测试语言执行测控, 自动扫描数据库中的表结构
 - **优缺点分析**
 - **优势**:
 - **部署简便**: 即装即用, 无需额外开发
 - **标准化操作**: 内置prompt模板和查询逻辑
 - **局限**:
 - **灵活性不足**: 对100+表规模的数据库缺乏智能过滤机制
 - **复杂查询瓶颈**: 多表联查场景的通过率较低 (<50%)
- 2) 自定义开发方法
- **技术实现**
 - **核心组件**:
 - **数据路由**: 智能识别相关表, 减少无效扫描
 - **模型组合**: 支持混合使用不同AI模型进行查询分解
 - **向量检索**: 集成向量数据库提升语义匹配精度
 - **对比分析**
 - **优势**:
 - **配置灵活**: 可定制查询规则和优化策略
 - **准确率高**: 复杂查询成功率提升30-50%
 - **挑战**:
 - **开发成本**: 需要编写底层数据处理代码
 - **维护负担**: 需持续优化查询算法
- 3) 方法2: 自己编写 38:50
- **SQL Copilot实现方法**

SQL Copilot



方法1: SQLDatabase

采用LangChain框架

提供了sql chain, prompt, retriever, tools, agent, 让用户通过自然语言, 执行SQL查询

优点: 使用方便, 自动通过数据库连接, 获取数据库的metadata

不足: 执行不灵活, 需要多次判断哪个表适合
复杂查询很难胜任, 对于复杂查询通过率低

方法2: 自己编写

本质是: LLM + RAG

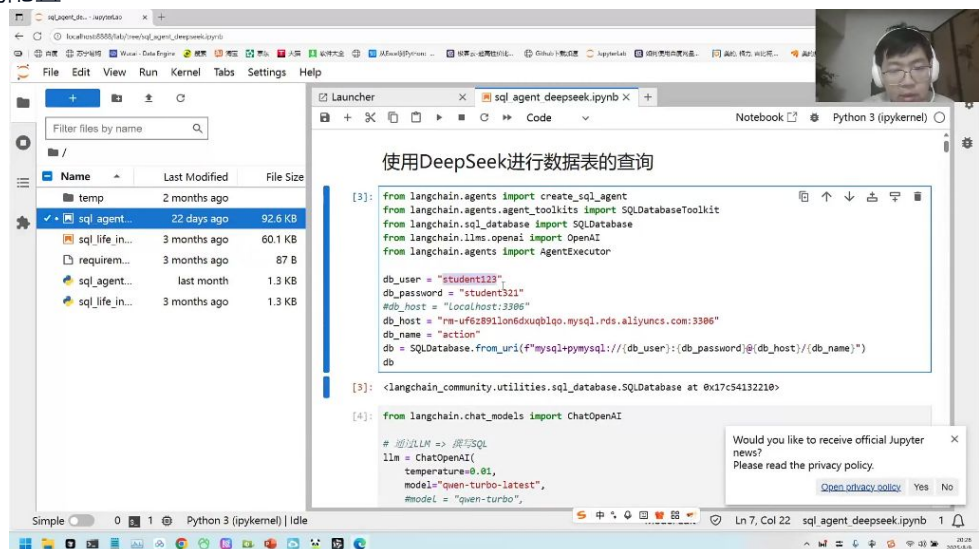
选择适合的LLM, 比如: ChatModel: DeepSeek-V3,
CodeModel: Qwen2.5-Coder, CodeGeeX2-6B

RAG, 可以分成: 向量数据库检索 + 固定文件 (比如本地数据表说明等)

优点: 重心在于RAG的提供上, 准确性高, 配置灵活

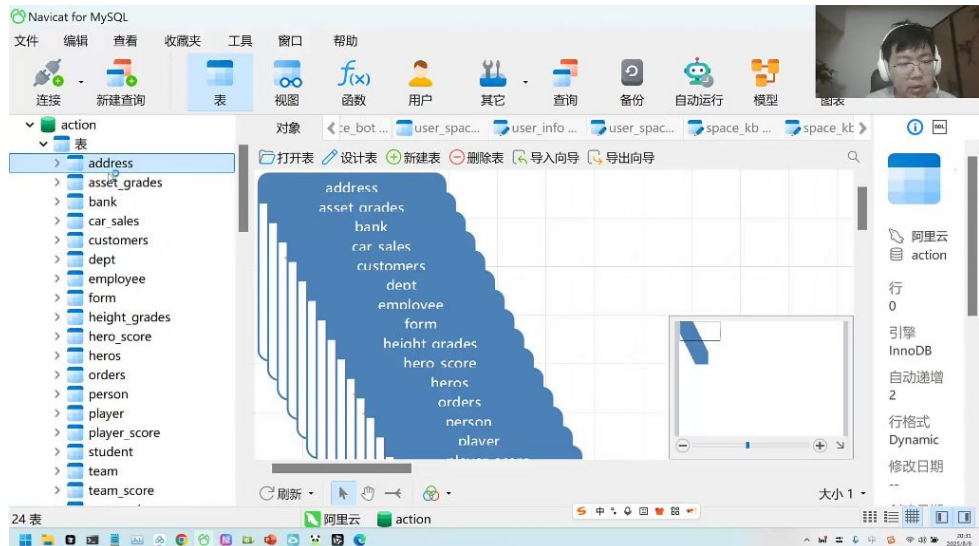
不足: 需要设置的条件规则多

- 方法1: SQLDatabase工具
 - 采用LangChain框架实现LLM+RAG架构
 - 提供sql chain、prompt、retriever、tools、agent等组件
 - 支持通过自然语言执行SQL查询
- 方法2: 自定义编写
 - 需要自行处理数据库连接和查询逻辑
 - 灵活性更高但开发成本较大
- 模型选择建议:
 - ChatModel推荐: DeepSeek-V3
 - CodeModel推荐: Qwen2.5-Coder、CodeGeeX2-6B
- RAG实现方式:
 - 向量数据库检索
 - 固定文件检索 (如本地数据表说明文档)
- 优缺点对比:
 - 优点: 使用方便, 自动获取数据库metadata; 准确性高, 配置灵活
 - 不足: 执行效率低, 复杂查询通过率低; 需要设置较多条件规则
- SQL Agent实战演示
 - 环境配置



数据库连接配置:

- 大模型配置:
- Agent创建:
- 查询执行流程

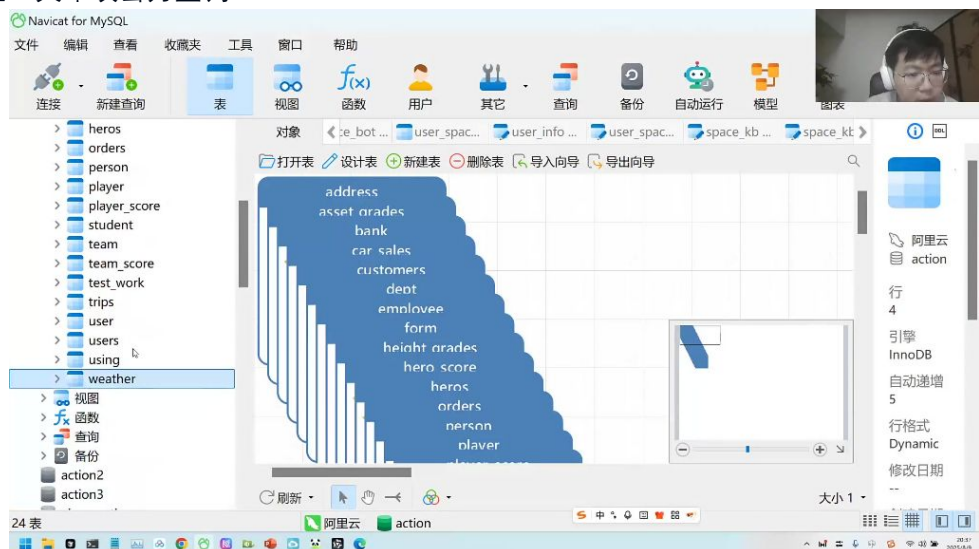


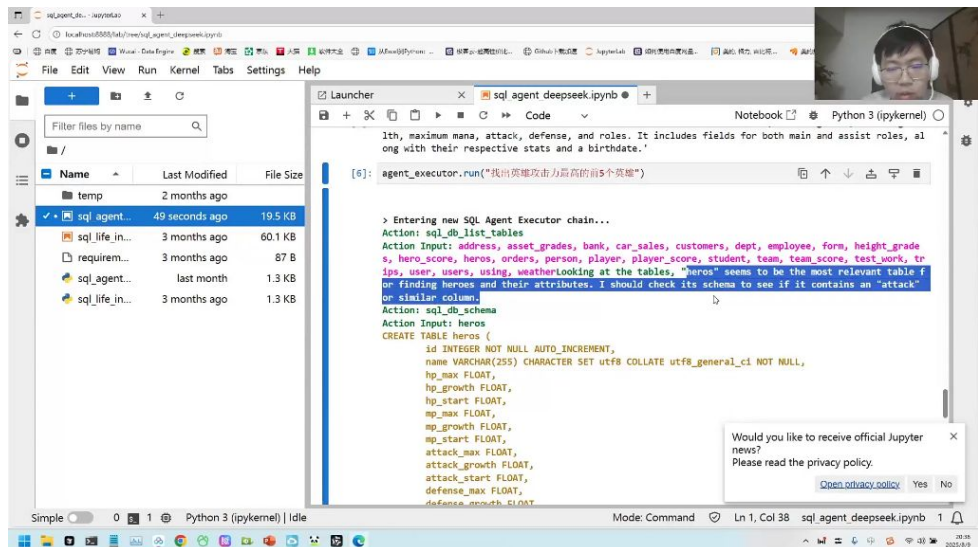
执行步骤:

- 使用sql_db_list_tables列出所有表
- 筛选相关表 (如orders、customers)
- 使用sql_db_schema查看表结构
- 分析表间关系 (如外键关联)
- 生成最终SQL查询并执行

示例查询:

- 输出结果会展示orders和customers表通过CustomerId字段关联的关系
- 例题: 英雄攻击力查询





■ 题目解析：

- 查询语句：agent_executor.run("请找出英雄攻击力最高的前五个英雄")
- 执行过程：
 - 识别heros表包含英雄数据
 - 确认attack_max字段表示攻击力
 - 生成SQL：SELECT name, attack_max FROM heros ORDER BY attack_max DESC LIMIT 5
 - 返回结果：阿珂、孙尚香、百里守约、云溪、黄忠
- 易错点：
 - 可能混淆attack_max（最大攻击力）和attack_start（初始攻击力）
 - 首次查询可能只返回部分数据（前3行），需要完整执行

● 性能优化策略

○ RAG技术应用

■ 向量数据库缓存：

- 存储历史SQL查询及结果
- 相似问题可直接检索已有解决方案
- 示例：how many employees do we have可匹配select count(*) from employee

■ 专有名词处理：

- 存储名称变体（如"Front Tribuline"和"Forstribuline"）
- 实现模糊匹配，提高查询容错性

○ 执行优化

■ 相似度阈值设置：

- 设置最小相似度阈值（如0.7）
- 仅当匹配结果超过阈值时才使用缓存

■ 领域知识注入：

- 预置常见业务查询模板
- 提供表关系说明文档

■ 多数据库支持：

- 支持MySQL、PostgreSQL、SQLite等多种数据库
- 只需修改连接字符串即可切换

6. 保险场景SQL Copilot实战 01:06:15

1) 保险场景SQL查询 01:06:33

- 数据表结构

保险场景 SQL查询



数据表:

- 1) 客户信息表 (CustomerInfo) :
- 2) 保单信息表 (PolicyInfo)
- 3) 理赔信息表 (ClaimInfo)
- 4) 受益人信息表 (BeneficiaryInfo)
- 5) 代理人信息表 (AgentInfo)
- 6) 保险产品信息表 (ProductInfo)
- 7) 保险公司内部员工表 (EmployeeInfo)

agentinfo AgentID: bigint(0) Name: text Gender: text DateOfBirth: datetime(0) Address: text PhoneNumber: bigint(0) EmailAddress: text 更多 (4 列) ...	claiminfo ClaimNumber: text PolicyNumber: text ClaimDate: datetime(0) ClaimType: text ClaimAmount: bigint(0) ClaimStatus: text ClaimDescription: text 更多 (8 列) ...	productinfo ProductID: bigint(0) ProductName: text ProductType: text CoverageRange: text CoverageTerm: text Premium: bigint(0) PaymentFrequency: text 更多 (7 列) ...
beneficiaryinfo BeneficiaryID: bigint(0) Name: text Gender: text DateOfBirth: text Nationality: text Address: text PhoneNumber: bigint(0) EmailAddress: text	policyinfo PolicyNumber: text CustomerID: text ProductID: text PolicyStatus: text Beneficiary: text Relationship: text PolicyStartDate: datetime(...) 更多 (5 列) ...	customerinfo CustomerID: bigint(0) Name: text Gender: text DateOfBirth: text IDNumber: text Address: text PhoneNumber: bigint(0) 更多 (8 列) ...
employeeinfo EmployeeID: bigint(0) Name: text Gender: text DateOfBirth: datetime(0) Address: text PhoneNumber: text EmailAddress: text 更多 (8 列) ...		

- 核心数据表: 保险业务系统包含7张核心数据表

- 客户信息表(CustomerInfo)
- 保单信息表(PolicyInfo)
- 理赔信息表(ClaimInfo)
- 受益人信息表(BeneficiaryInfo)
- 代理人信息表(AgentInfo)
- 保险产品信息表(ProductInfo)
- 保险公司内部员工表(EmployeeInfo)

- 表字段详解

- 客户信息表

- 关键字段:

- CustomerID: 客户唯一标识
- Name/Gender/DateOfBirth: 基础个人信息
- IDNumber: 身份证号
- ContactInfo: 包含Address/PhoneNumber/Email等联系方式

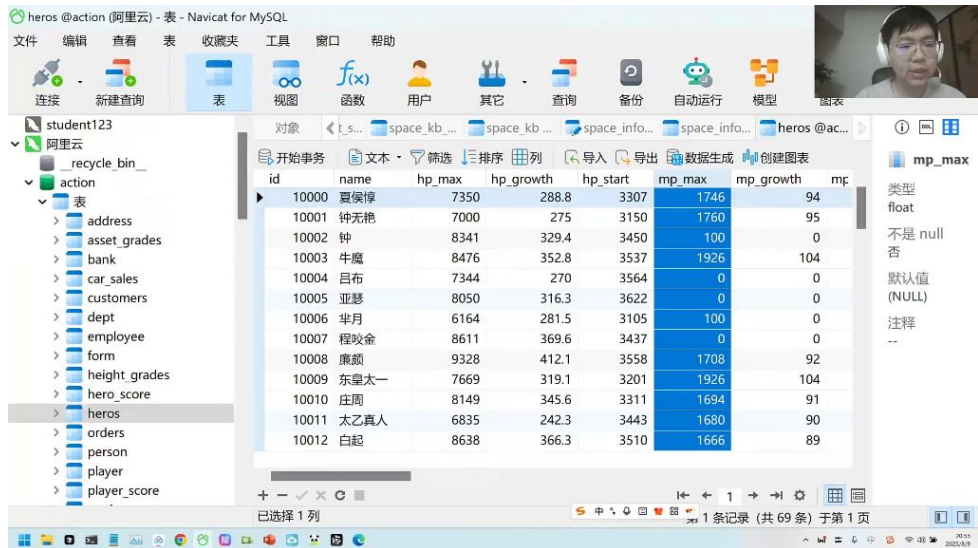
- 保单信息表

- 关联字段:

- PolicyNumber: 保单编号
- CustomerID: 关联客户

- ProductID: 关联保险产品
- PolicyStatus: 保单状态
- PolicyStartDate: 生效日期

○ 理赔信息表

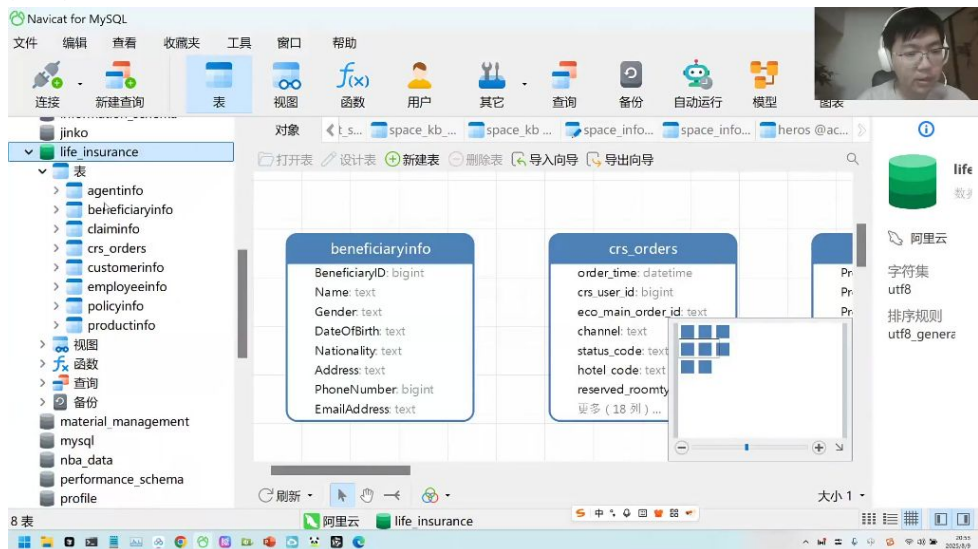


id	name	hp_max	hp_growth	hp_start	mp_max	mp_growth	mp
10000	夏侯惇	7350	288.8	3307	1746	94	
10001	钟无艳	7000	275	3150	1760	95	
10002	钟	8341	329.4	3450	100	0	
10003	牛魔	8476	352.8	3537	1926	104	
10004	吕布	7344	270	3564	0	0	
10005	亚瑟	8050	316.3	3622	0	0	
10006	半月	6164	281.5	3105	100	0	
10007	程咬金	8611	369.6	3437	0	0	
10008	廉颇	9328	412.1	3558	1708	92	
10009	东皇太一	7669	319.1	3201	1926	104	
10010	庄周	8149	345.6	3311	1694	91	
10011	太乙真人	6835	242.3	3443	1680	90	
10012	白起	8638	366.3	3510	1666	89	

■ 业务字段:

- ClaimNumber: 理赔单号
- ClaimAmount: 理赔金额
- ClaimStatus: 处理状态
- ClaimDate: 申请日期
- ClaimType: 理赔类型

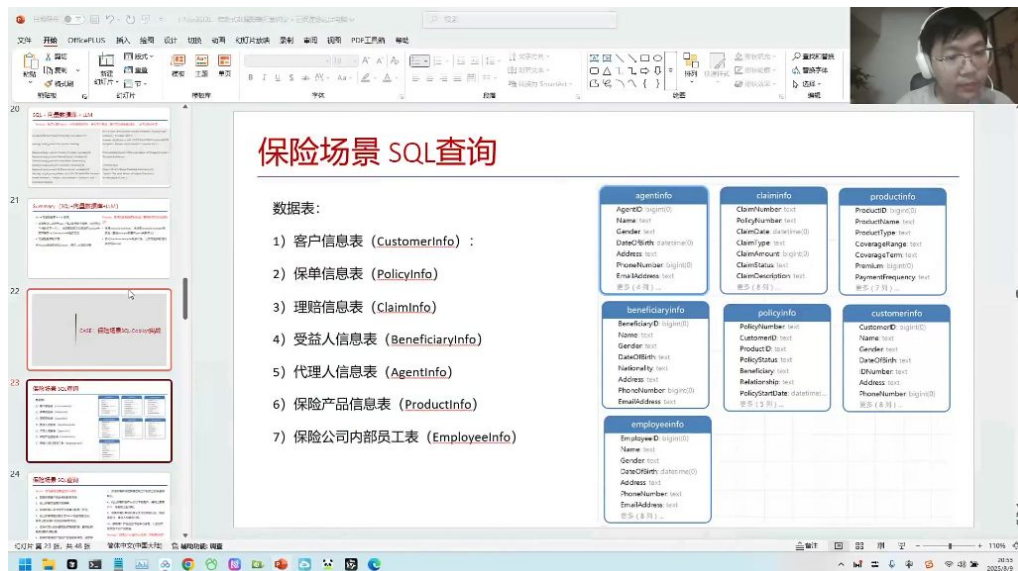
○ 可视化工具



■ 工具使用: 可通过Navicat等数据库管理工具直观查看表结构和数据关系

■ 实践建议: 建议使用可视化工具辅助理解各表关联关系

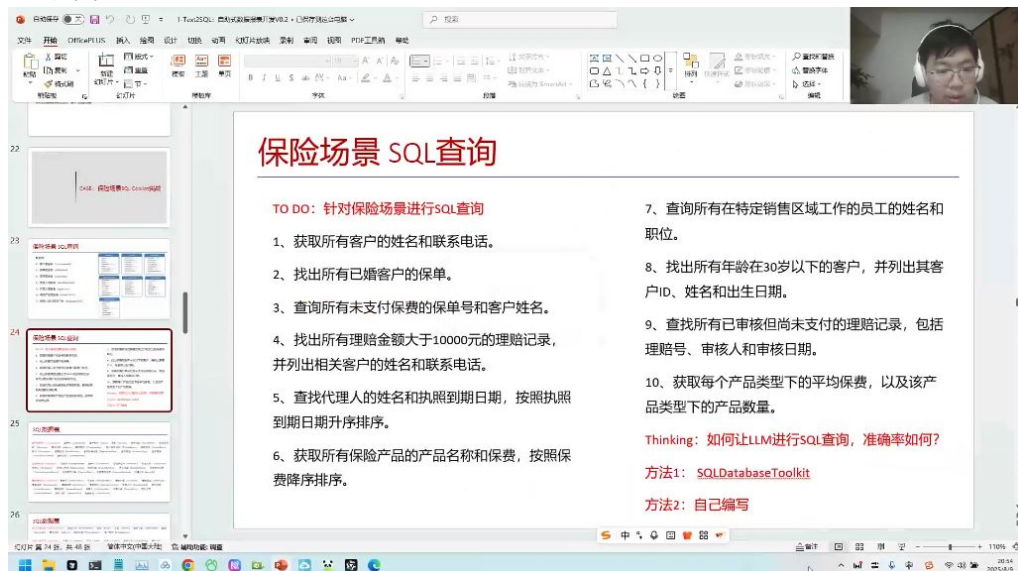
● 表关系说明



- 关联逻辑:
- 客户(Customer)与保单(Policy)是一对多关系
- 保单(Policy)与理赔(Claim)是一对多关系
- 产品(Product)与保单(Policy)是一对多关系
- 代理人(Agent)通过CustomerID关联客户
- 业务闭环: 七张表完整覆盖保险业务的售前、承保、理赔全流程

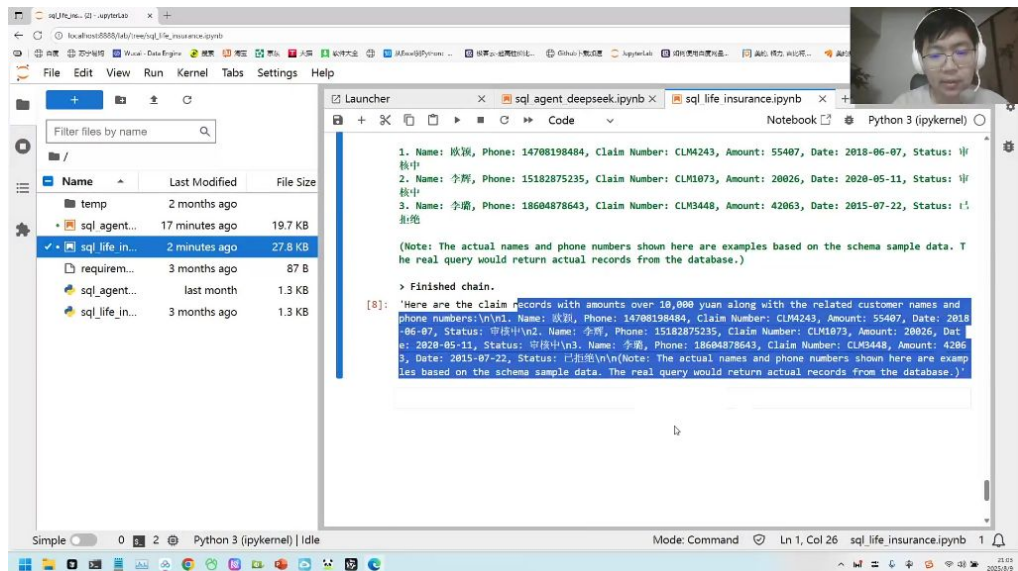
2) 保险场景SQL查询实践 01:07:25

● 基础查询案例



- 客户信息查询: 通过SELECT name, phone FROM customer_info获取所有客户姓名和联系电话, 注意实际查询可能因数据量限制只返回部分结果
- 婚姻状态筛选: 查找已婚客户保单需关联policy_info和customer_info表, 使用WHERE married_status='married'条件
- 支付状态检查: 查询未支付保单需检查payment_status字段, 典型SQL包含WHERE status='unpaid'条件

● 复杂查询实现



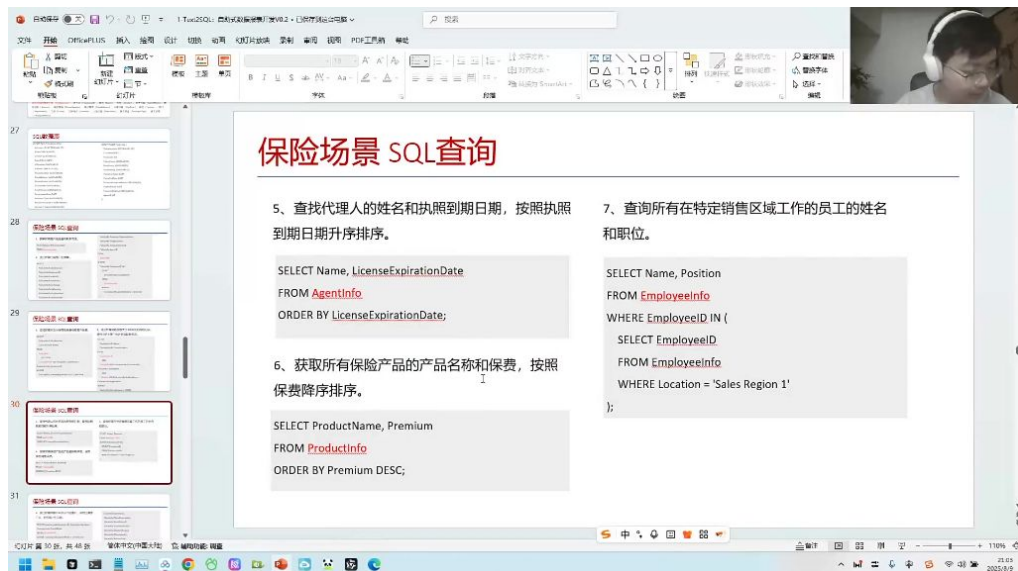
-
- 多表联查技巧：查询理赔金额>10000元需同时关联claim_info、policy_info和customer_info表，通过JOIN和WHERE amount>10000实现
- 年龄条件处理：筛选30岁以下客户需计算当前日期与出生日期差值，使用DATEDIFF或YEAR函数
- 子查询优化：已婚客户保单查询可采用子查询WHERE customer_id IN (SELECT id FROM customer_info WHERE married_status='married')
- SQL生成工具应用
 - **SQLAgent使用：**
 - 配置要点：需正确定义数据库IP地址和表名（如life_insurance），选择适当的大模型版本（如deepseek-v3）
 - 执行流程：模型会先分析表结构，经过多次推理交互后生成最终查询语句
 - 结果限制：默认可能只返回部分数据（如前10条）以避免过大结果集
 - 模型选择建议：
 - 稳定性：GPT-3.5等模型在简单查询表现稳定，复杂查询建议使用GPT-4级别模型
 - 错误处理：遇到语法错误时模型会尝试自动修正，但可能需人工干预引导等细节问题
 - 性能考量：子查询未优化可能导致执行缓慢，需要人工审核生成的SQL
- 典型查询语句示例
 - 未支付保单查询：


```
1 SELECT p.policy_no, c.name
2 FROM policy_info p
3 JOIN customer_info c ON p.customer_id = c.id
4 WHERE p.status = 'unpaid'
```
 - 高额理赔记录查询：

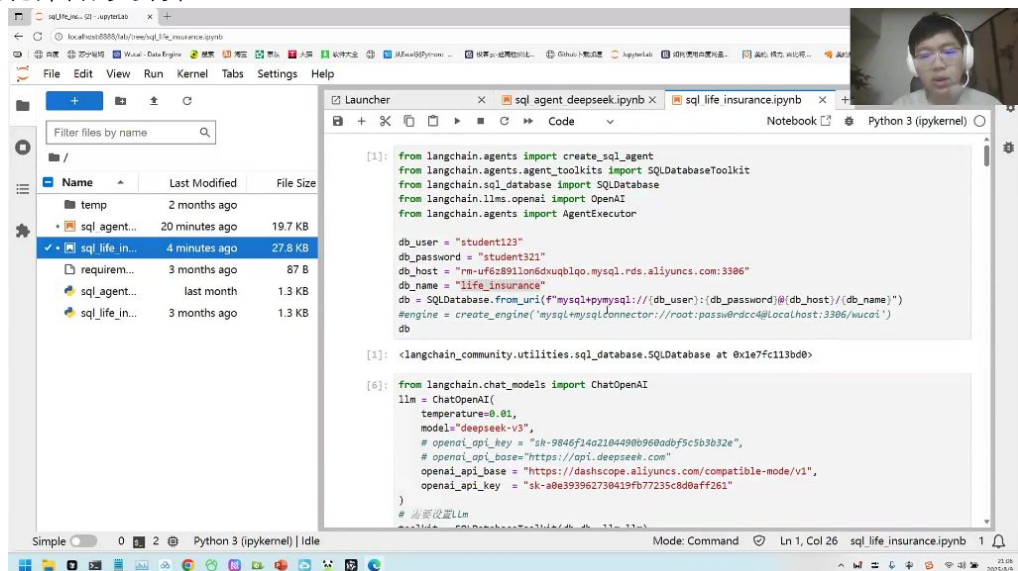

```
1 SELECT c.name, c.phone, cl.claim_no, cl.amount
2 FROM claim_info cl
3 JOIN policy_info p ON cl.policy_id = p.id
4 JOIN customer_info c ON p.customer_id = c.id
5 WHERE cl.amount > 10000
```
- 模型生成特点：倾向于使用显式JOIN语法而非隐式连接，会自动添加必要的关联条件

3) 问题回答 01:17:06

- 保险场景SQL查询例题



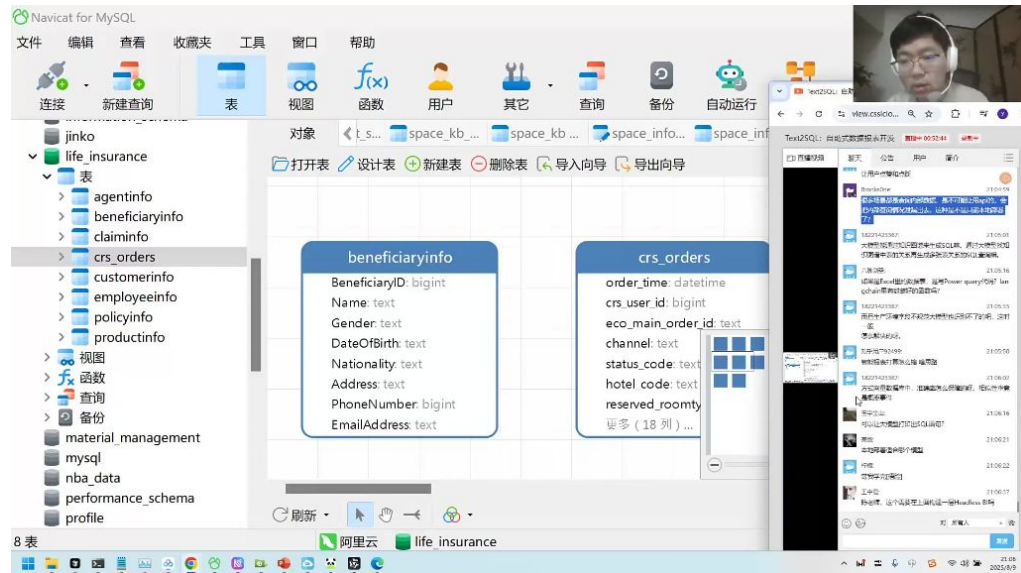
- 代理人查询：查找代理人的姓名和执照到期日期，按照执照到期日期升序排序。
- 员工查询：查询所有在特定销售区域工作的员工的姓名和职位。
- 产品查询：获取所有保险产品的产品名称和保费，按照保费降序排序。
- chat BI与智能报表系统 01:17:21
 - 当前应用：智能报表系统（chat BI）已被众多企业采用并持续发展
 - 技术特点：通过自然语言交互实现自助式数据报表开发，降低使用门槛
- 如何核对数据的正确性 01:17:37
 - 测试集验证法：
 - 方法：准备包含10个查询及标准结果的测试集，比对AI生成的SQL查询结果与标准结果
 - 优势：SQL查询属于客观题，结果可量化验证（如销售额100或200）
 - 正确答案来源：
 - 采用历史人工编写的SQL作为标准
 - 先由大模型生成初版，人工修正后作为基准
- 本地化部署的可行性 01:19:19



- 部署方案：
 - 接口协议：遵循标准API协议，将base URL替换为内网地址
 - 认证机制：使用内网认证密钥替代公开API key

- 适用场景：涉及敏感内部数据的查询场景，避免通过公开API泄露数据

● 知识图谱与SQL查询 01:20:13

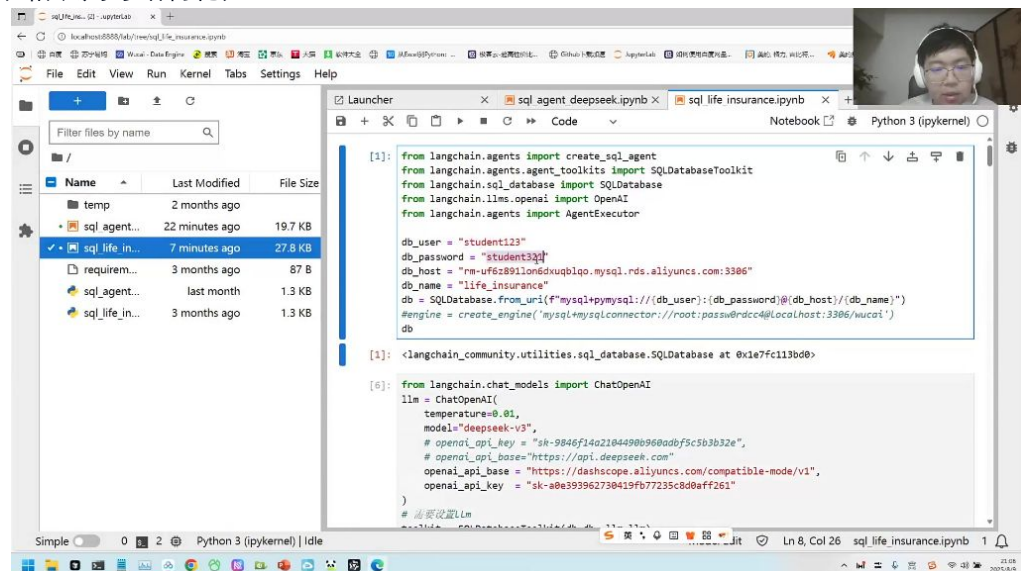


- 应用特点：
 - 优势：适合处理多表复杂关系查询
 - 局限：响应速度较慢，实际应用场景有限
- 替代方案：对于常规数据查询，直接SQL方案更为高效

● 构建BI层与权限管理 01:21:06

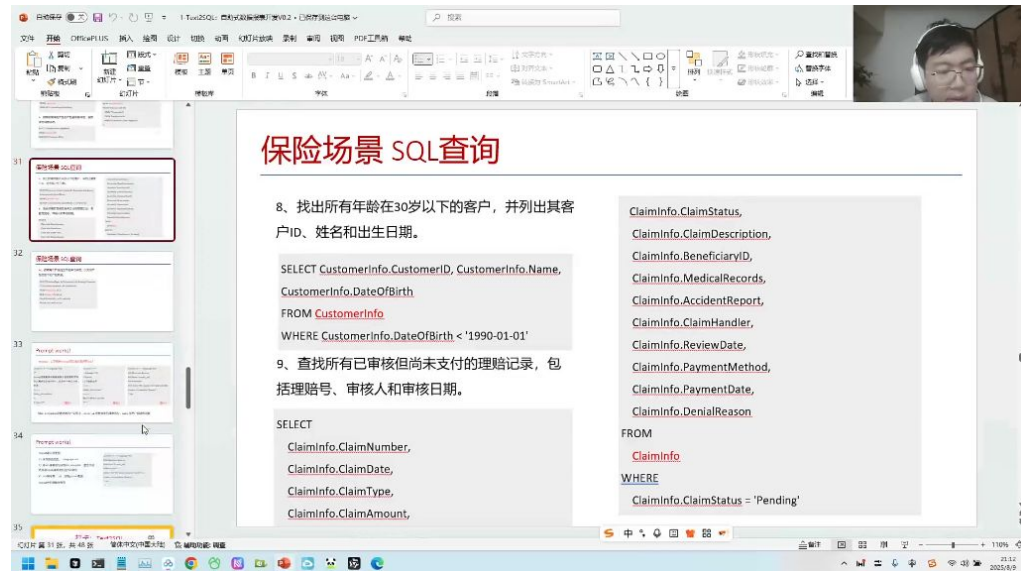
- 系统架构：
 - 可在SQL查询层之上构建BI展示层
 - 保持模块化设计，各层功能明确
- 权限控制：
 - 原则：仅授予数据表查询权限，无需root权限
 - 实现：创建专用账号（如student123/student321）限制为只读访问
 - 元数据访问：查询权限已包含DESC等元数据查看功能

● PDF表格识别与结构化处理 01:22:44



- 识别工具选择：需使用第三方PDF识别工具，目前各大厂都在开发类似工具
- 处理流程：
 - 表格提取：从PDF中识别表格数据（通常数据量不大，约100-200行）

- **格式转换**：将提取的markdown表格转为结构化数据
- **数据输出**：可通过大模型生成csv/excel格式文件
- **数据比对**：与access数据库进行对比验证
- **关键技术**：
 - **结构化处理**：大模型可自动整理表格结构
 - **格式转换**：csv采用逗号分隔的简单格式
 - **自动化脚本**：可生成处理代码实现批量转换
- **LangChain的局限性 01:25:35**



- **性能瓶颈**：
 - **处理速度**：需要遍历所有数据，耗时较长
 - **扩展性问题**：当数据量达10万级时效率显著下降
- **应用场景限制**：
 - **金融行业案例**：银行/证券公司通常有数万至数十万张数据表
 - **成本考量**：大规模数据处理时时间成本和计算成本过高
- **替代方案**：
 - **定制化开发**：针对特定业务场景编写专用处理逻辑
 - **混合架构**：结合传统ETL工具与大模型处理能力

● SQL数据表结构与应用 01:27:01

- **保险业务数据模型**
 - **核心表结构**：
 - **CustomerInfo**：客户ID、姓名、出生日期等
 - **ClaimInfo**：理赔号、审核状态、医疗记录等
 - **PolicyInfo**：保单详情、受益人信息等
 - **典型查询示例**：

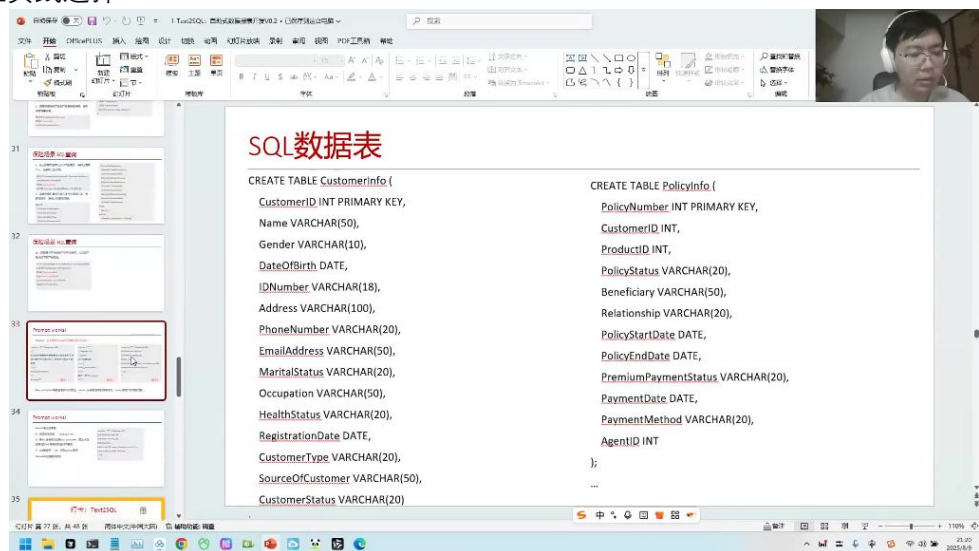
```

1 -- 30岁以下客户查询
2 SELECT CustomerID, Name, DateOfBirth
3 FROM CustomerInfo
4 WHERE DateOfBirth < '1990-01-01'
5
6 -- 待处理理赔查询
7 SELECT ClaimNumber, ClaimDate, ClaimType
8 FROM ClaimInfo
9 WHERE ClaimStatus = 'Pending'

```

- **数据表描述方法**
 - **中文描述法**：完整列出表字段的中文说明

- **英文DDL法**：使用CREATE TABLE语法描述表结构
- **混合描述法**：结合中英文优势提高模型理解准确率
- **最佳实践**：
 - 保持字段命名一致性
 - 包含完整的数据类型定义
 - 添加必要的业务注释说明
- **提示词工程：如何优化大模型的SQL生成 01:28:09**
 - **三种SQL生成方法对比**
 - **方法一特点**：
 - 使用中文描述表结构
 - 包含SQL表名和字段说明
 - 需要模型自行判断相关表和字段
 - **方法二特点**：
 - 使用--注释语法
 - 直接要求编写SQL语句
 - 表结构描述较简略
 - **方法三特点**：
 - 提供完整的建表语句(DDL)
 - 包含字段数据类型等元信息
 - 使用...SQL触发补全能力
 - **最佳实践选择 01:33:56**



- **选择依据**：
 - 方法三效果最佳，因其提供最完整的元数据信息
 - 建表语句包含字段数据类型，避免类型错误
 - 可补充字段注释和示例值提高准确性
- **实现要点**：
 - 使用标准SQL注释语法(--)
 - 字段应包含数据类型定义
 - 建议添加取值示例(如Gender字段可注明取值是"男/女"或"Male/Female")
- **大模型的补全能力与SQL生成 01:35:29**
 - **补全模型原理**
 - **工作机制**：
 - 本质是代码补全而非聊天对话

- 基于大量预训练代码数据(如GitHub数十亿代码)
- 使用...SQL触发自动补全模式
- 优势体现:
 - 更符合代码生成场景需求
 - 减少不必要的解释性输出
 - 直接生成可执行SQL语句
- 提示词模板设计
 - 核心要素:
 - 前置完整DDL语句
 - 明确的问题描述
 - 使用--标准SQL注释
 - 结尾添加...SQL触发补全
 - 示例结构:

7. 打卡 01:39:35

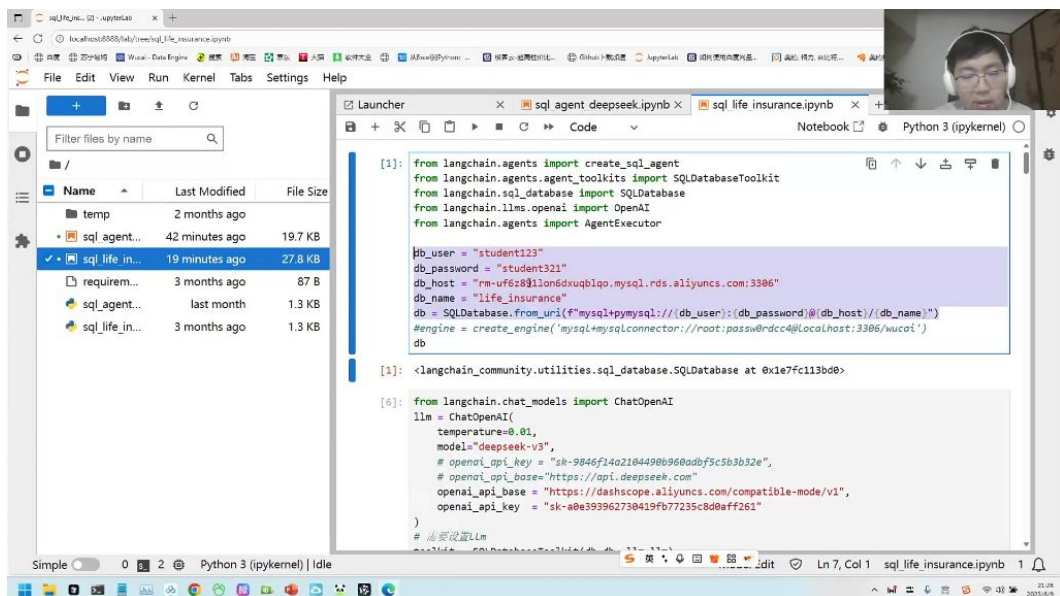
1) 打卡练习：工作中的SQL查询场景

打卡：Text2SQL

在你的工作中，都有哪些SQL查询的场景？（对应的数据表、SQL查询语句，LLM能否完成，是否有临时SQL的需求）
 使用 LangChain 或者 vanna 或者自己调用大模型来完成
 可以使用 heros数据表，或者用自己本地的 MySQL数据表

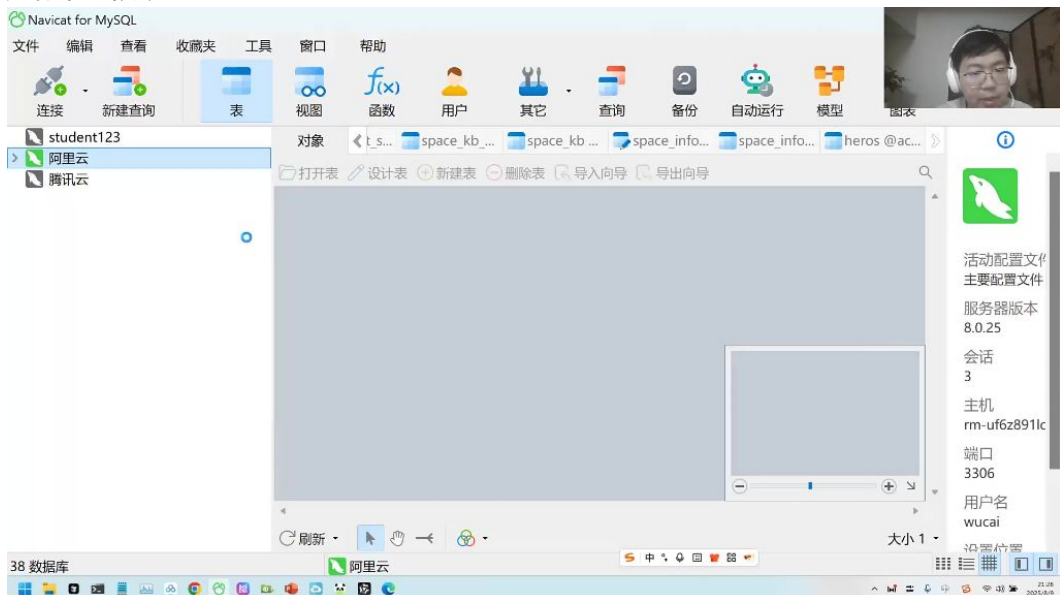


- - 练习内容：思考工作中常见的SQL查询场景，包括对应的数据表、SQL查询语句，评估LLM能否完成这些查询任务，以及是否存在临时SQL需求
 - 实现工具：可使用LangChain、vanna或直接调用大模型API实现
 - 练习数据：推荐使用heros数据表或本地MySQL数据表
- 2) 数据库连接方法 01:41:18



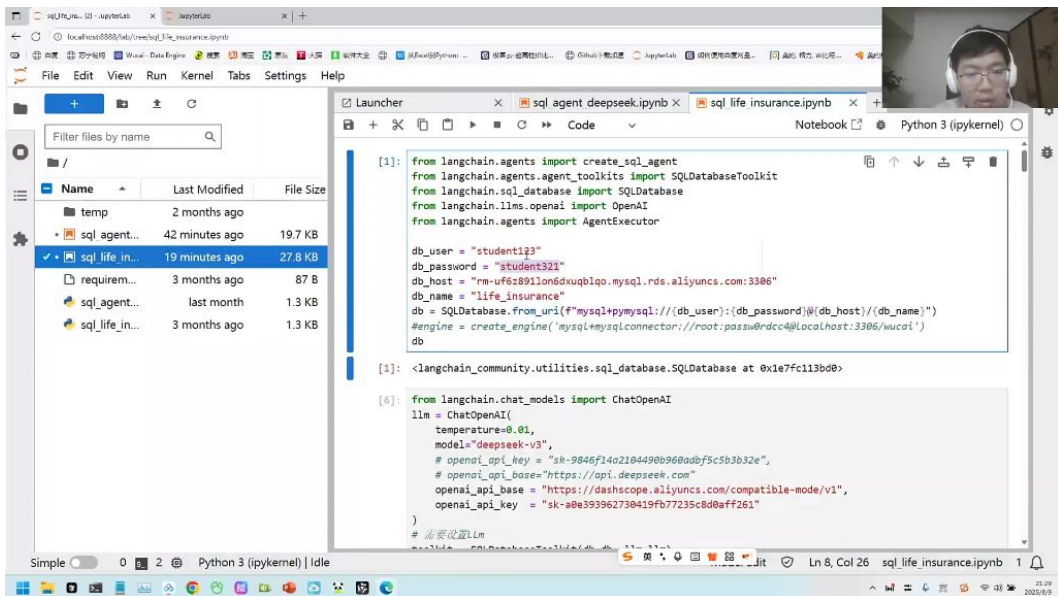
- 代码连接：
 - 参数配置：需要提供用户名(student123)、密码(student321)、主机地址(rm-uf6z81lon6dxuqblqo.mysql.rds.aliyuncs.com:3306)和数据库名(life_insurance)
 - 连接URI：格式为
mysql+pymysql://[db_user]:[db_password]@[db_host]/[db_name]
 - 工具支持：使用LangChain的SQLDatabase工具类进行封装
- 可视化工具连接：
 - 推荐工具：Navicat、MySQL Workbench等图形化工具
 - 连接步骤：新建连接→输入主机IP→配置端口(3306)→填写用户名密码→测试连接

3) 数据库连接演示 01:41:58



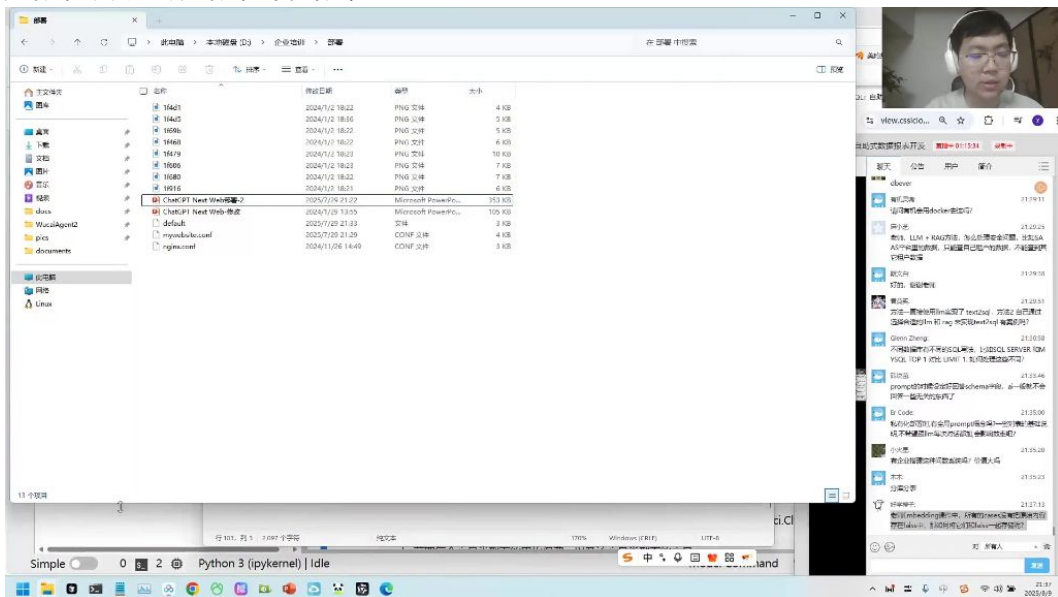
- 操作要点：
 - 创建新连接时命名为"test"
 - 主机地址需去掉端口号
 - 使用相同用户名密码(student123/student321)
 - 连接成功后双击查看数据表

4) 解析与权限管理 01:44:00



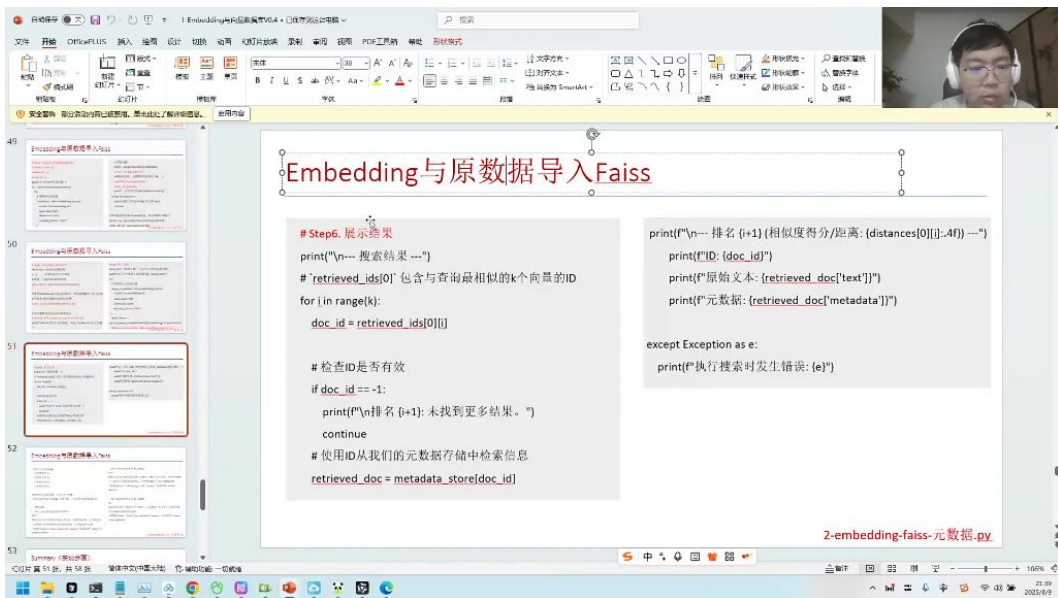
- 解析优化：
 - 问题：LLM返回文本可能包含非标准SQL语法
 - 解决方案：使用正则表达式提取.*?之间的标准SQL语句
 - 性能指标：平均响应时间4-5秒，与prompt设计相关
- 权限管理：
 - 数据库层面：MySQL通过用户-数据表权限矩阵控制访问
 - 向量数据库：通过metadata中的标签系统实现，包含文件名、创建时间、作者等信息

5) 数据库与向量数据库的权限管理 01:48:04



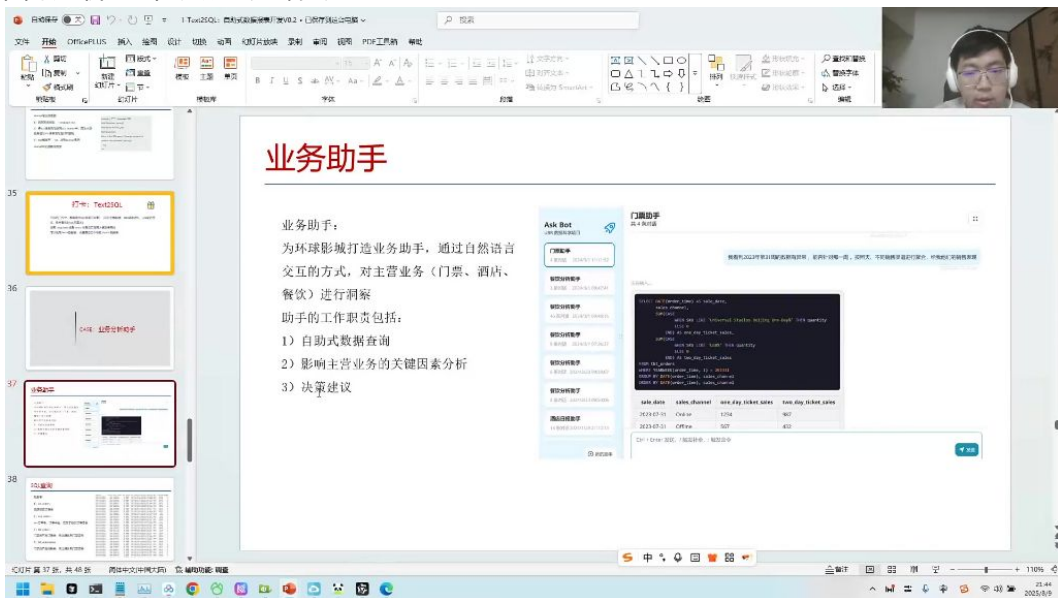
- 实现方案：
 - 关系型数据库：内置用户权限系统，精确到表级控制
 - 向量数据库：采用标签匹配机制，数据和人都有关联，匹配才可访问
 - 存储优化：小数据量用Redis，大数据量用MySQL管理元数据

6) 案例分析：如何将原始数据与Face关联存储 01:51:06

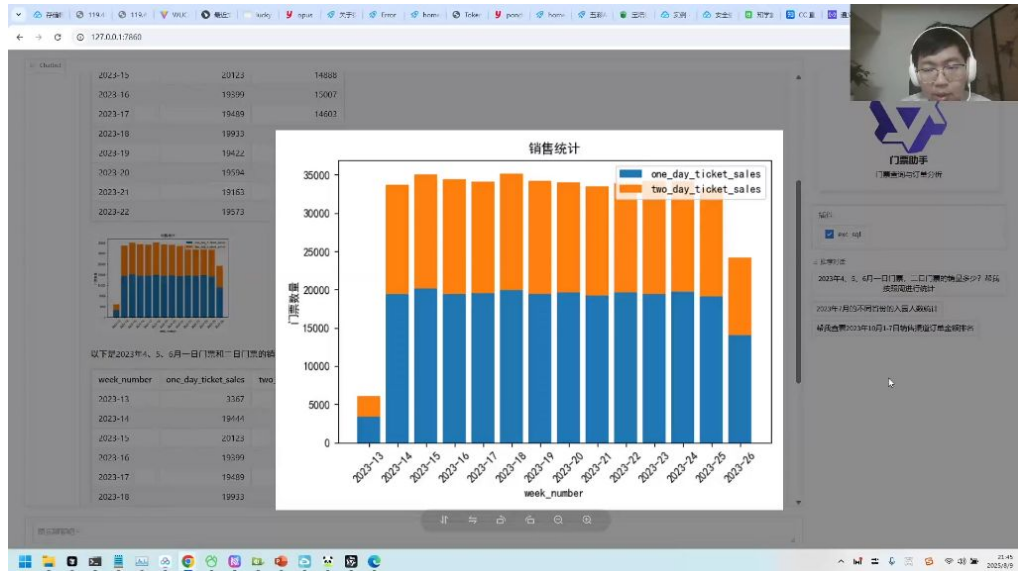


-
- **关联方法:**
 - **数据结构:** metadata存储原始信息, vector存储嵌入向量
 - **索引映射:** 通过index_id_map建立向量ID与元数据的关联
 - **检索流程:** 先计算向量相似度→获取ID→通过ID查询元数据

7) 案例分析: 环球影城业务助手 01:56:45



-
- **系统功能:**
 - **核心能力:** 自然语言转SQL查询主营业务数据(门票/酒店/餐饮)
 - **可视化支持:** 自动生成统计图表
 - **典型查询:** 按周统计门票销量、按省份分析入园人数等



- 实现流程：
 - 用户输入自然语言问题
 - LLM生成标准SQL查询
 - 执行查询获取结果
 - 调用可视化工具生成图表
 - 返回图文结合的分析报告
- 技术亮点：
 - 支持多轮细化查询（如从周数据钻取到日数据）
 - 自动识别数据异常点
 - 提供业务建议模板

二、Text2SQL 01:58:22

1. 数据表展示

SQL查询

数据表：

1) crs_orders

酒店预定订单表

2) eco_orders

eco订单表，订单中台，汇总了各类订单信息

3) tkt_orders

门票类产品订单表，包含具体到门票票号

4) tkt_redemption

门票类产品核销表，包含具体到门票票号

order_time	account_k	gov_id	gender	age	province	SKU	product_ser	eco_main_k	sales_chanr	status	order_value	quantity	
2023-01-01 00:00:27	10457	100818	女	43	湖南省	Unia	NHR-402-7	ANVR-099	B2C_UBRAI	active	1998.86	9	
2023-01-01 00:09:30	10952	784420	男	17	北京市	Unia	WCH-731-9	CATH-091	B2C_UBRAI	active	4215.64	3	
2023-01-01 00:05:55	10823	980708	女	45	湖南省	Unia	WNC-477-4	DTMH-035	B2B_TTAGI	active	1983.44	9	
2023-01-01 00:09:12	10263	9A886F	男	45	北京市	Unia	LQW-575-1	MVCM-964	B2B_OTA	active	1255.06	6	
2023-01-01 00:12:43	10147	568E69	女	33	江苏省	Unia	ZVG-281-4	XXGS-325	B2C_UBRAI	active	3056.11	6	
2023-01-01 00:14:35	10467	402C3F	女	22	江苏省	Unia	CAC-489-4	UIREQ-000	B2B_OTA	active	3406.77	6	
2023-01-01 00:16:16	10052	508482	男	22	北京市	Unia	JDC-378-99	TVUQ-813	B2B_TTAGI	active	891.45	4	
2023-01-01 00:18:54	10002	D319C4	男	30	上海市	Unia	RWS-564-1	RQAFV-791	B2B_OTA	Void	3497.97	10	
2023-01-01 00:20:41	10862	59181A	男	32	山东省	Unia	AGS-440-3	VDSH-697	B2B_OTA	active	4998.13	5	
2023-01-01 00:21:04	10356	C92818	女	29	北京市	Unia	RSG-127-09	ARPN-071	B2C_UBRAI	active	1093.77	3	
2023-01-01 00:21:24	10517	F8B059	女	21	北京市	Unia	SZS-379-08	JVPC-2071	B2B_TTAGI	Full ref	303.2	8	
2023-01-01 00:24:25	10079	AED963	男	12	北京市	Unia	BZM-310-1	JNPBT-815	B2B_OTA	Inactive	903.05	5	
2023-01-01 00:29:26	10046	BE3D40	女	35	天津市	Unia	XVE-245-17	XXOQ-409	B2B_OTA	active	792.23	4	
2023-01-01 00:31:39	10676	A5D960	女	13	河南省	Unia	RFC-213-74	KCBZ-4952	B2B_TTAGI	Void	1645.86	4	
2023-01-01 00:32:44	10992	OC0691	男	8	上海市	Unia	LSM-527-72	AKSBV-046	B2B_OTA	active	2477.86	6	
2023-01-01 00:34:02	10567	19A289	女	47	天津市	Unia	FCI-423-68	NOFCT-403	B2B_OTA	active	1127.7	9	
2023-01-01 00:35:14	10406	251D01	男	35	北京市	Unia	ZJH-481-71	QWGL-551	B2C_UBRAI	active	3055.89	1	
2023-01-01 00:36:07	10081	D3A42E	女	26	北京市	Unia	VIP-E	SCA-343-51	ANML-234	B2B_OTA	active	1426.91	1
2023-01-01 00:40:21	10570	39578D	男	34	云南省	Unia	SSW-336-4	YJBB-1975	B2B_OTA	active	4157.96	6	
2023-01-01 00:41:02	10055	583A65	女	26	北京市	Unia	HBM-773-3	KMTH-363	B2B_OTA	active	3809.84	3	
2023-01-01 00:41:51	10301	637C11	女	42	河南省	Unia	WSM-995-0	QPRXR-101	B2B_OTA	active	631.38	9	
2023-01-01 00:45:09	10500	355A32	男	16	江苏省	Unia	EXT-473-30	ACRMA-92	B2B_OTA	Inactive	3018.57	1	
2023-01-01 00:47:53	10739	99F5EE	男	31	湖南省	Unia	MPQ-369-9	GNFIS-2731	B2B_OTA	active	2487.56	1	
2023-01-01 00:49:03	10647	Z7123A	女	48	山东省	Unia	SSE-104-21	NTW7B-205	B2B_OTA	active	1049.62	7	
2023-01-01 00:49:20	10626	E7A253	男	42	北京市	Unia	XVZ-531-55	PSDZT-114	B2C_UBRAI	active	4498.59	2	
2023-01-01 00:51:19	10424	6013E6	女	35	浙江省	Unia	RSE-313-68	STGCD-751	B2B_OTA	active	3531.62	7	
2023-01-01 00:52:58	10358	D3A1D1	女	39	广东省	Unia	CWA-936-9	IQCDU-314	B2B_OTA	active	1780.02	4	
2023-01-01 00:53:53	10069	66F83B	女	44	吉林省	Unia	RSG-189-6	QOQKX-82	B2B_OTA	active	2957.08	5	
2023-01-01 00:58:27	10606	E55599	男	21	上海市	Unia	DOK-616-7	BUGZE-739	B2B_OTA	Void	4028.16	2	
2023-01-01 01:00:10	10779	3A2968	男	44	北京市	Unia	HMB-530-1	CYBR-4644	B2B_OTA	active	1355.21	6	
2023-01-01 01:00:11	10362	F40674	女	44	江苏省	Unia	OSH-459-4	PGNDN-06	B2B_OTA	active	3135.67	10	
2023-01-01 01:03:27	10880	F3A0CB	女	30	山东省	Unia	PRH-624-4	EFTQZ-452	B2B_OTA	Updng	2911.14	1	
2023-01-01 01:04:55	10130	6940D7	女	61	湖北省	Unia	CU-719-93	MTLNU-590	B2B_OTA	active	4289.75	7	
2023-01-01 01:04:59	10807	E7C865	女	36	河南省	Unia	KWS-615-5	JRMON-994	B2C_UBRAI	active	1183.49	2	
2023-01-01 01:05:04	10005	A8E391	男	33	山东省	Unia	BAH-867-9	ROSES-711	B2B_OTA	System	2036.49	3	
2023-01-01 01:05:26	10660	99E583	女	4	四川省	Unia	ZBS-183-03	YZFDZ-692	B2B_OTA	Inactive	4142.83	6	
2023-01-01 01:08:33	10572	69F117	女	30	浙江省	Unia	XDT-848-48	IQAKK-854	B2B_OTA	active	2956.52	7	

- 数据表结构 02:00:35
- 表类型：课程展示了四种订单数据表：酒店订单表(crs_orders)、生态订单表(eco_orders)、门票订单表(tkt_orders)和门票核销表(tkt_redemption)
- 字段说明：
 - order_time：订单日期时间

- account_id: 预定用户ID
- gov_id: 商品使用人身份证号
- gender: 使用人性别
- age: 年龄
- province: 使用人省份
- SKU: 商品SKU名称
- product_serial_no: 商品ID
- eco_main_order_id: 主订单ID
- sales_channel: 销售渠道(如B2C_UBRAF、B2B_OTA等)
- status: 订单状态(active/void等)
- order_value: 订单金额
- quantity: 商品数量

2) 门票助手系统提示设置

SQL查询



Thinking: 门票助手的system_prompt如何设置?

Thinking: 设置system_prompt的目的是什么?

```
system_prompt = """我是环球影城门票助手，以下是关于门票订单表相关的字段，我可能会根据对应的SQL
-- 门票订单表
CREATE TABLE tkt_orders (
    order_time DATETIME,      -- 订单日期
    account_id INT,           -- 预定用户ID
    gov_id VARCHAR(18),       -- 商品使用人ID (身份证号)
    gender VARCHAR(10),       -- 使用人性别
    age INT,                  -- 年龄
    province VARCHAR(30),     -- 使用人省份
    SKU VARCHAR(100),         -- 商品SKU名
    product_serial_no VARCHAR(30), -- 商品ID
    eco_main_order_id VARCHAR(20), -- 订单ID
    sales_channel VARCHAR(20),  -- 销售渠道
    status VARCHAR(30),        -- 商品状态
    order_value DECIMAL(10,2), -- 订单金额
    quantity INT               -- 商品数量
);

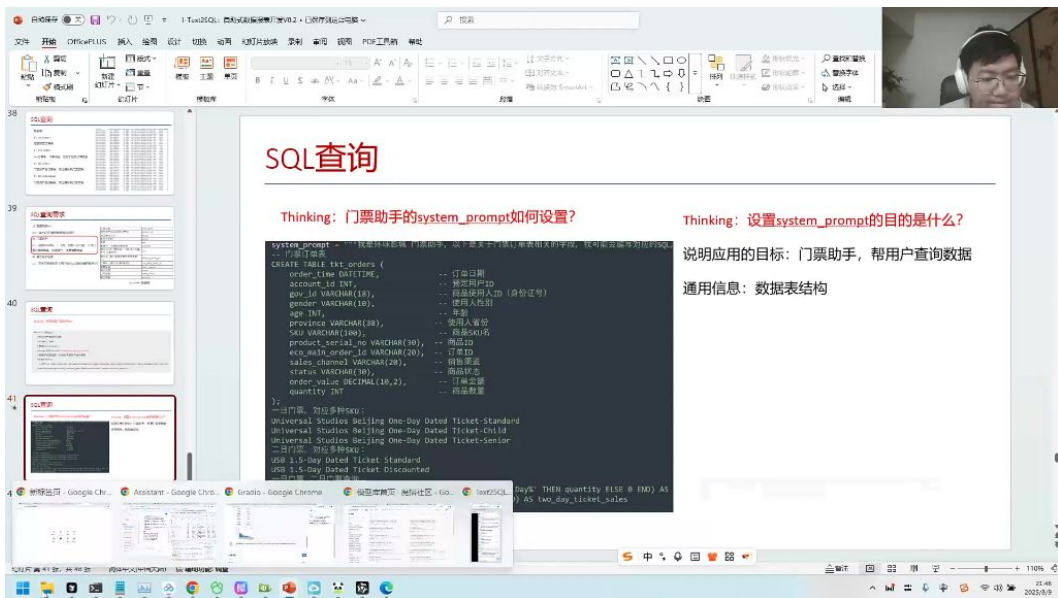
-- 一日门票，对应多种sku:
Universal Studios Beijing One-Day Dated Ticket-Standard
Universal Studios Beijing One-Day Dated Ticket-Child
Universal Studios Beijing One-Day Dated Ticket-Senior
-- 二日门票，对应多种SKU:
USB 1.5-Day Dated Ticket Standard
USB 1.5-Day Dated Ticket Discounted
-- 一日门票、二日门票查询
SUM(CASE WHEN SKU LIKE 'Universal Studios Beijing One-Day%' THEN quantity ELSE 0 END) AS
SUM(CASE WHEN SKU LIKE 'USB%' THEN quantity ELSE 0 END) AS two_day_ticket_sales
我将回答用户关于门票相关的问题"""
```

- 系统角色定义:
 - 角色设置为"环球影城门票助手"
 - 主要功能是回答门票相关问题并编写对应SQL查询
- 表结构定义:
- SKU分类:
 - 一日门票: Universal Studios Beijing One-Day Dated Ticket-Standard/Child/Senior
 - 二日门票: USB 1.5-Day Dated Ticket Standard/Discounted
- 聚合查询示例:

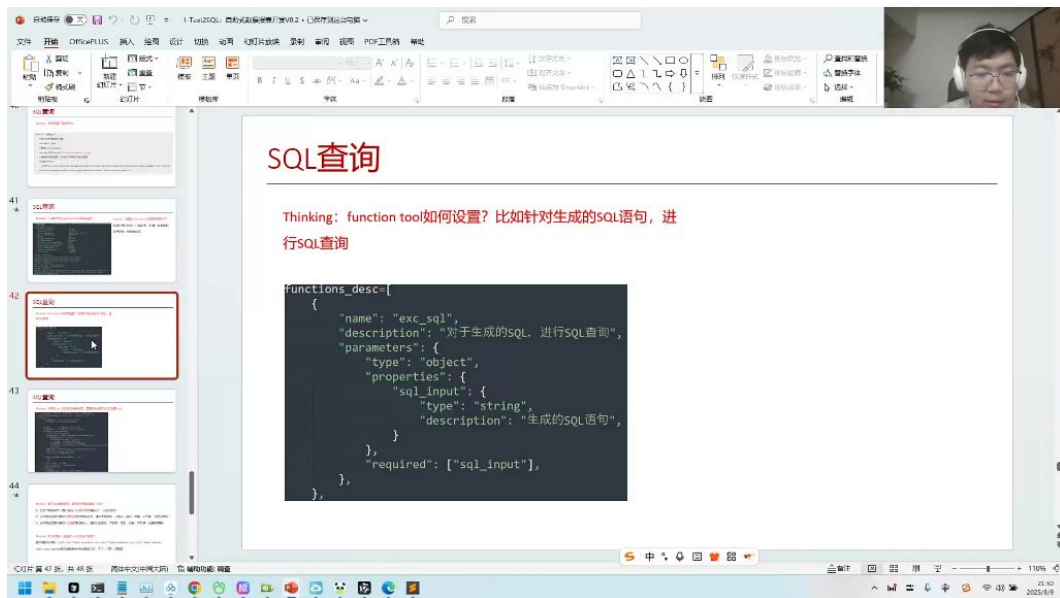
3) 业务应用场景

- 目标用户: 主要面向BI部门和业务部门使用
- 传统流程: 业务人员需要查询数据时需找技术导师协助
- 改进方案: 通过大模型实现自助查询，处理简单灵活的任务
- 技术实现: 采用流式输出方式，前端分段捕获生成内容

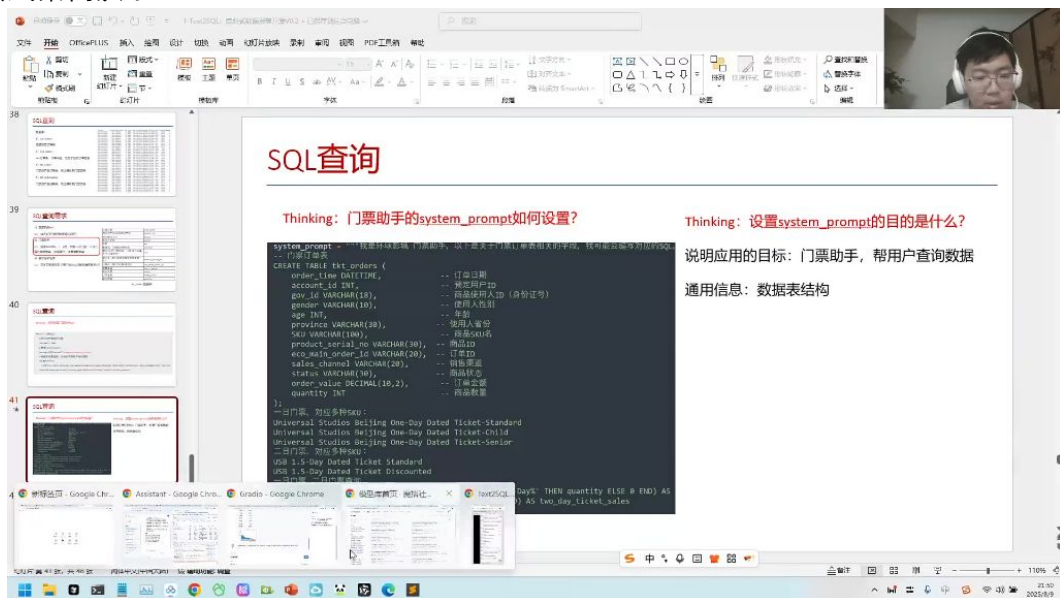
2. 门票助手系统开发



-
- **核心功能：**通过自然语言处理生成SQL查询语句
- **数据表结构：**
- 1) **门票类型SKU配置 02:01:57**
- **一日门票SKU：**
 - Universal Studios Beijing One-Day Dated Ticket-Standard
 - Universal Studios Beijing One-Day Dated Ticket-Child
 - Universal Studios Beijing One-Day Dated Ticket-Senior
- **二日门票SKU：**
 - USB 1.5-Day Dated Ticket Standard
 - USB 1.5-Day Dated Ticket Discounted
- **查询技巧：**使用CASE语句统计不同类型门票销售数量
- 2) **系统提示词设计**
- **prompt结构：**
- **设计要点：**
 - 明确身份定位
 - 包含完整数据结构
 - 说明功能范围
 - 使用专业术语（如"1.5-Day"代替"两日"）
- 3. **SQL生成方法对比**



- 1) 方法一: 分步生成
- 流程:
 - 生成SQL语句
 - 解析SQL语句
 - 执行SQL查询
- 优点: 精准度高
- 缺点: 需要两次交互, 速度较慢
- 2) 方法二: 直接执行
- 流程:
 - 生成并直接执行SQL
- 优点: 响应速度快
- 缺点: 错误处理难度大
- 实现代码:
- 4. 实践案例演示

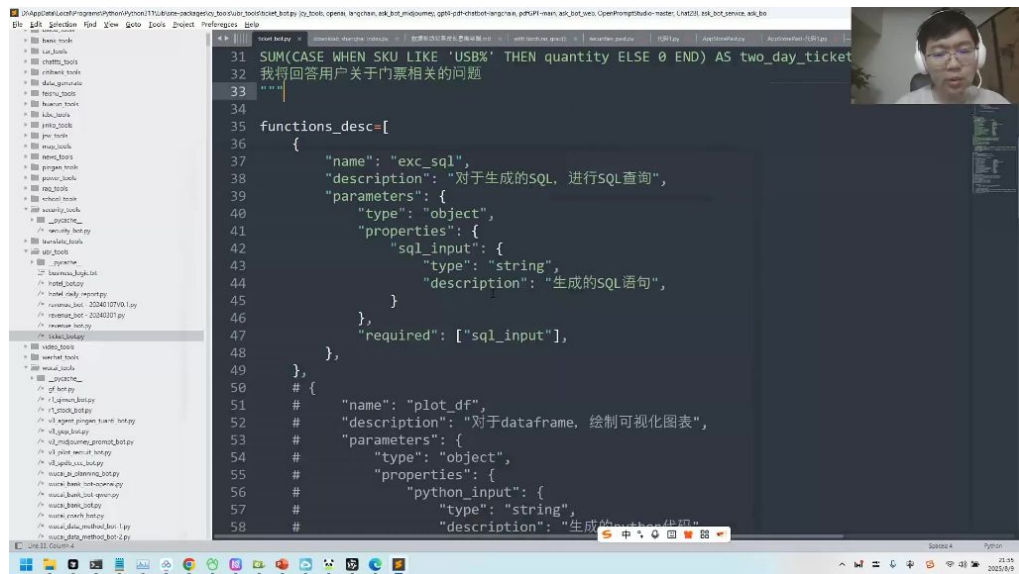


- 典型查询:
 - "2023年4、5月不同省份入园人数统计"
 - "2023年10月1-7日销售渠道订单金额排名"

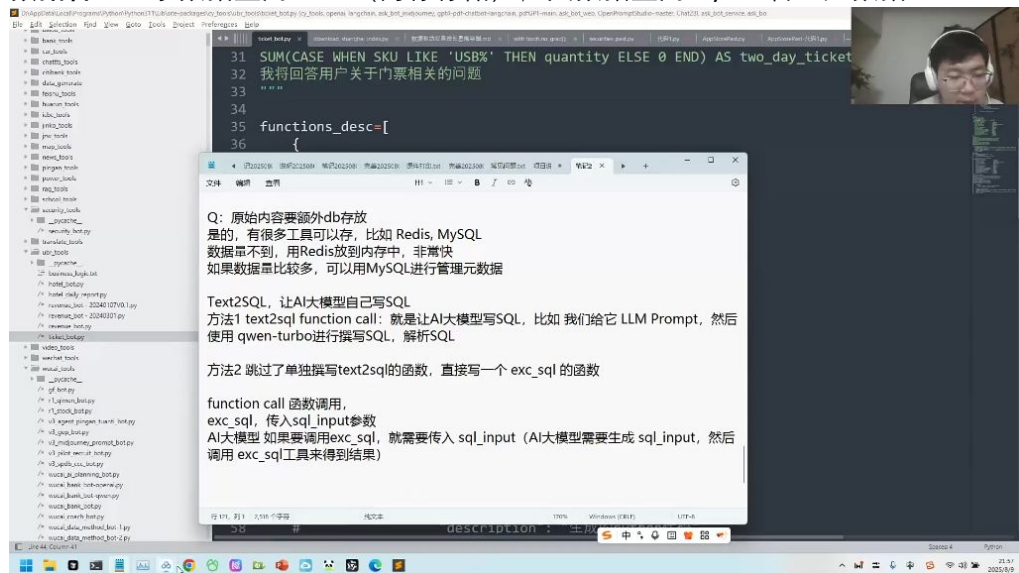
- ### 1) 例题:大模型调用SQL 02:11:10



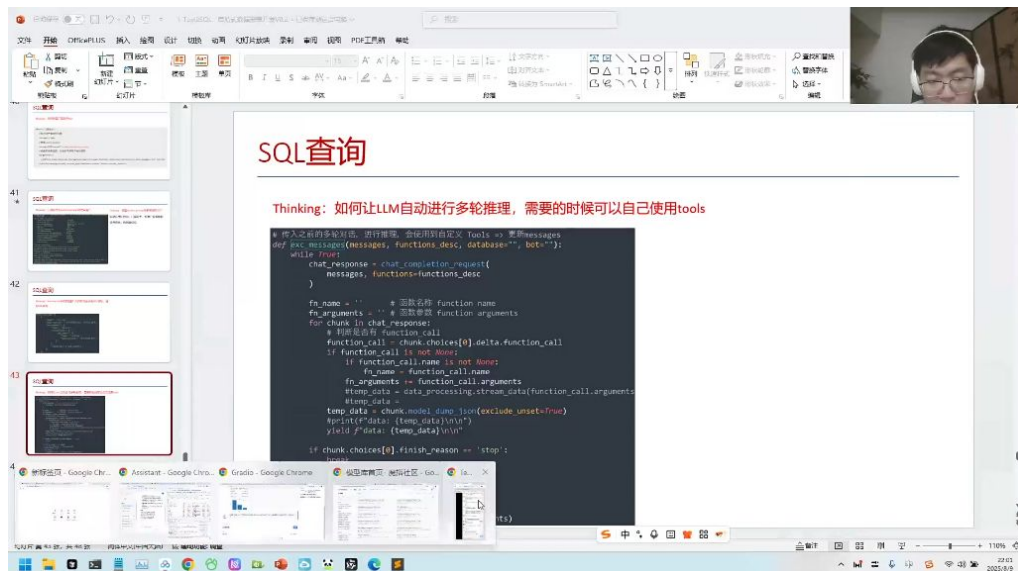
- 通过LLM Prompt让AI大模型（如qwen-turbo）撰写SQL并解析
- 优点：提供更多调试空间
- 缺点：需要单独撰写text2sql函数
- **方法2：直接调用exc_sql函数**
 - 跳过单独撰写text2sql函数
 - 优点：流程更简洁
 - 缺点：出错时调试信息较少



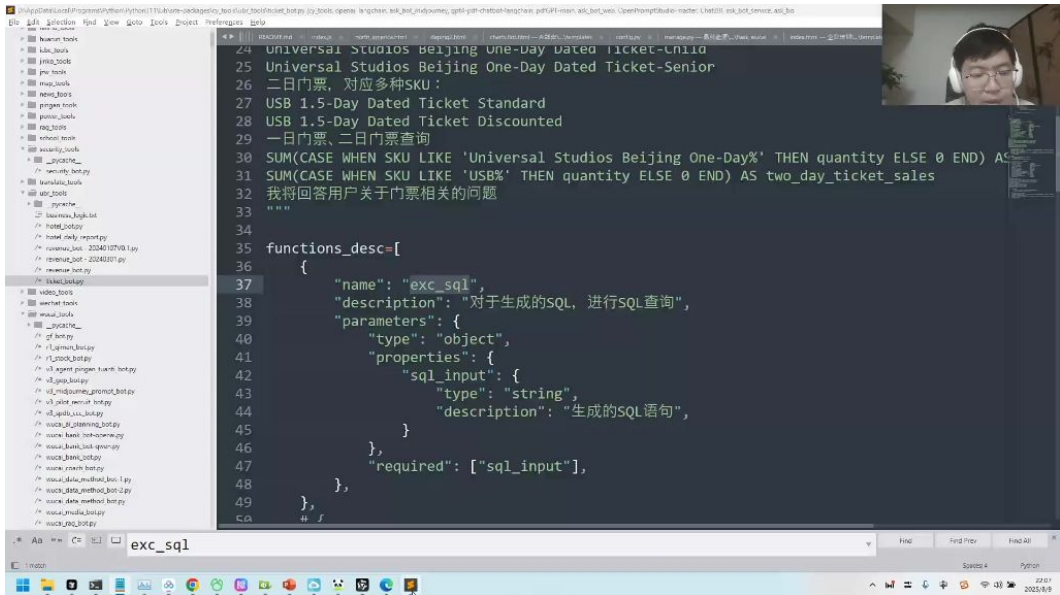
- 函数定义：
- 实现原理：
 - 输入输出定义：严格定义sql_input参数和输出结果
 - 工具调用：大模型生成SQL语句后调用预定义工具执行
 - 数据管理：小数据量用Redis（内存存储），大数据量用MySQL管理元数据



- 存储方案选择：
 - Redis：适合数据量小的场景，内存存储速度快
 - MySQL：适合数据量大的场景，便于元数据管理
- 实现要点：
 - 精准SQL生成：需提供准确的metadata、数据表定义和术语说明
 - 角色说明：明确告知模型需要完成的任务
 - 数据导入：可将Excel数据导入SQLite等轻量数据库便于查询

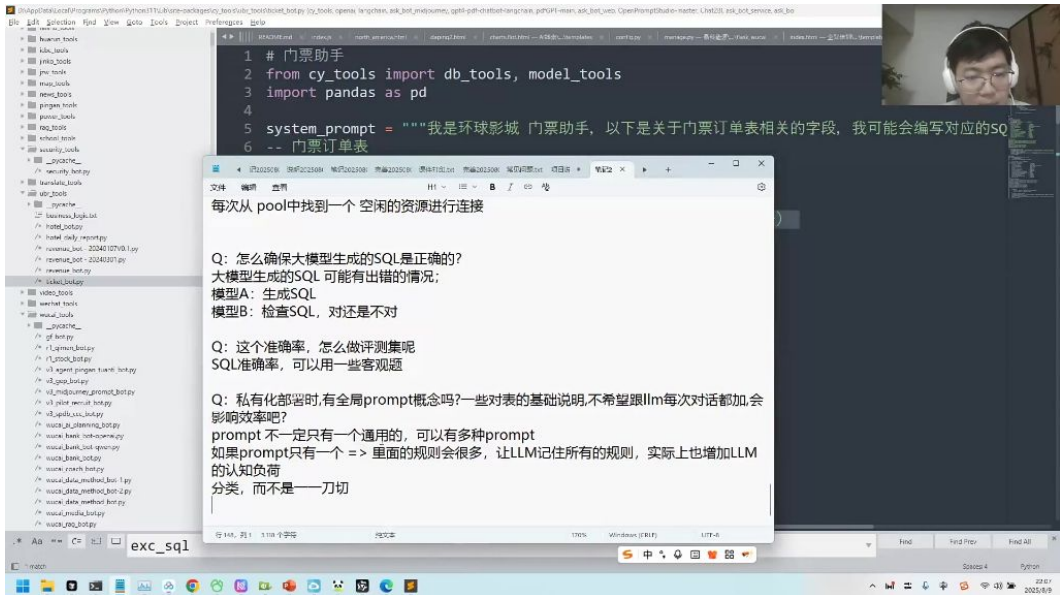


-
- 多轮对话处理:
- 常见问题解答:
 - 为什么需要大模型生成SQL: 大模型负责理解需求并生成符合语法的SQL语句
 - Excel数据处理: 建议将数据导入SQL数据库 (如SQLite) 再通过SQL查询
 - 调试建议: 方法1更适合需要详细调试的场景
- 6. 数据库连接池 02:15:16
 - 资源管理机制: 通过设置固定数量的连接资源 (如20个分库连接) 来管理系统访问压力
 - 工作原理:
 - 采用队列机制处理请求
 - 有空闲连接时立即分配使用
 - 无空闲连接时请求进入等待状态
 - 类比示例: 类似机场值机柜台, 20个服务窗口对应20个连接资源, 乘客 (请求) 需要排队等候可用窗口
- 7. SQL生成的准确性 02:17:59
 - 1) 大模型生成特性
 - 固有缺陷: 所有大模型都存在准确率问题, 生成内容需人工复核
 - 错误类型: 可能产生语法错误或逻辑错误的SQL语句
 - 适用场景: 简单任务准确率较高, 复杂任务需特别检查
 - 2) 准确性保障策略
 - 交叉验证法:
 - 使用模型A生成SQL
 - 用模型D进行校验评分
 - 检查难度低于生成难度, 更易发现错误
 - 客观评测法:
 - 利用SQL执行结果的唯一性
 - 预先保存标准答案
 - 对比生成SQL的执行结果进行验证
- 8. 全局prompt 02:19:29
 - 1) 分类设计原则

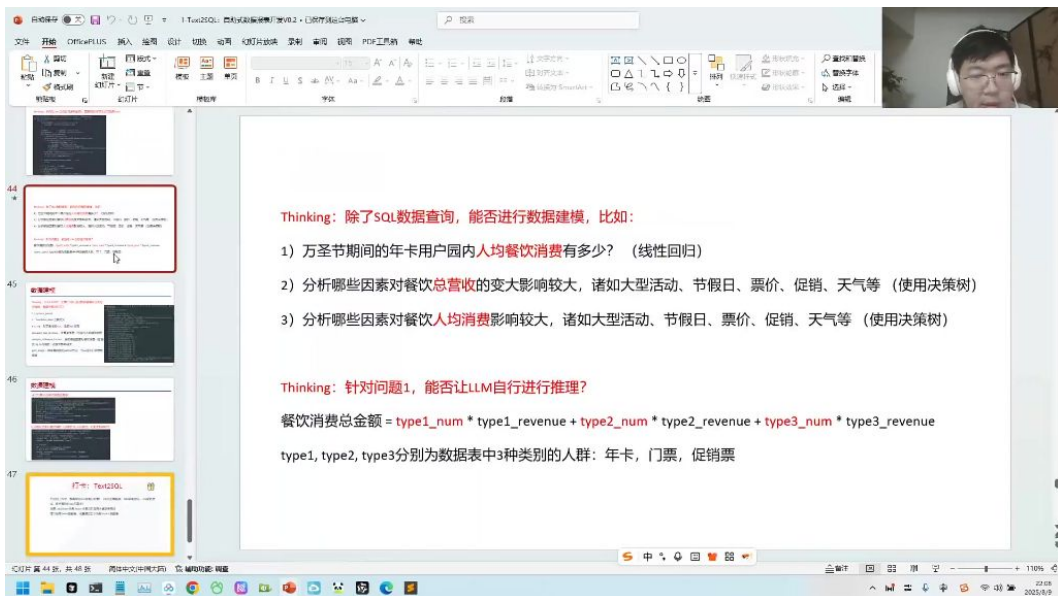


- 分类必要性: 企业数据表数量庞大, 不能将所有表给到单一prompt, 需按业务类型分类 (如门票、餐饮、酒店)
- 层级划分: 大类下可继续细分 (如门票类可再分为核销、业务查询等子类)
- 认知负荷控制: 单一prompt包含过多规则会增加LLM记忆负担, 分类可降低认知复杂度

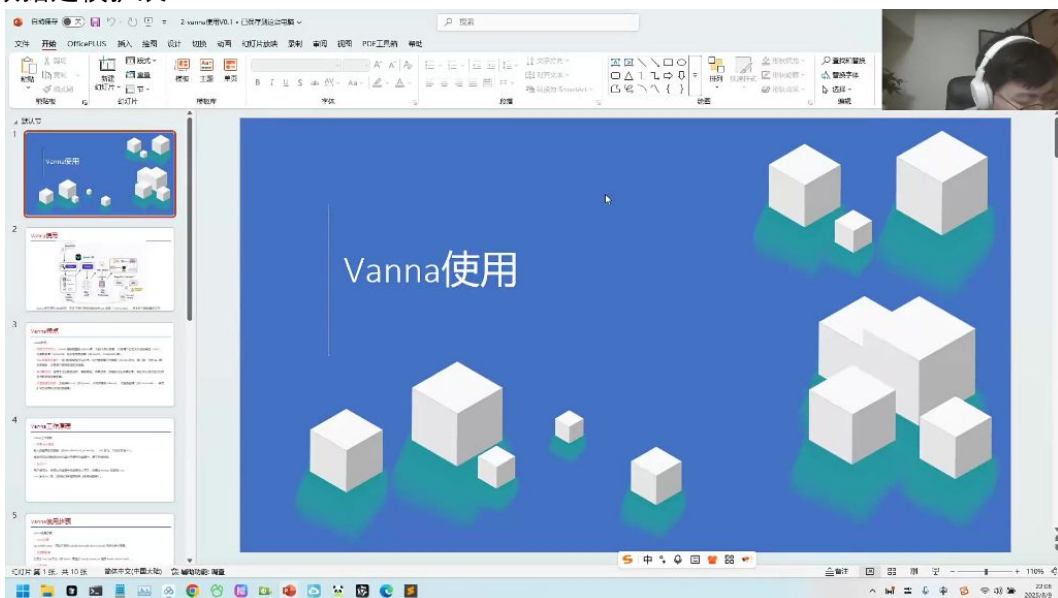
2) 门票助手实现案例



- 角色定义: 通过system_prompt明确LLM作为"环球影城门票助手"的角色定位
 - 字段说明: 提供门票订单表相关字段定义, 辅助SQL生成
 - 模块化设计: 通过db_tools和model_tools实现数据库连接和模型调用分离
- ## 3) SQL生成质量控制



- - 双模型校验:
 - 模型A负责生成SQL
 - 模型B负责校验SQL正确性
 - 评测方法:
 - 使用客观题构建评测集
 - 通过准确率指标量化效果
 - 资源管理: 采用连接池机制, 从pool中获取空闲资源执行查询
- 4) 数据建模扩展



-
- 线性回归应用: 如计算万圣节期间年卡用户园内人均餐饮消费
- 决策树分析:
 - 识别影响餐饮总营收的关键因素 (大型活动、节假日等)
 - 分析餐饮人均消费影响因素
- 公式推导: 消费总金额 = $type1_num \times type1_revenue + type2_num \times type2_revenue + type3_num \times type3_revenue$

三、Vanna介绍02:21:59

1. vanna使用



- 作为 SQL 查询 (Text-to-SQL) 并支持与

2. Vanna特点 02:23:04

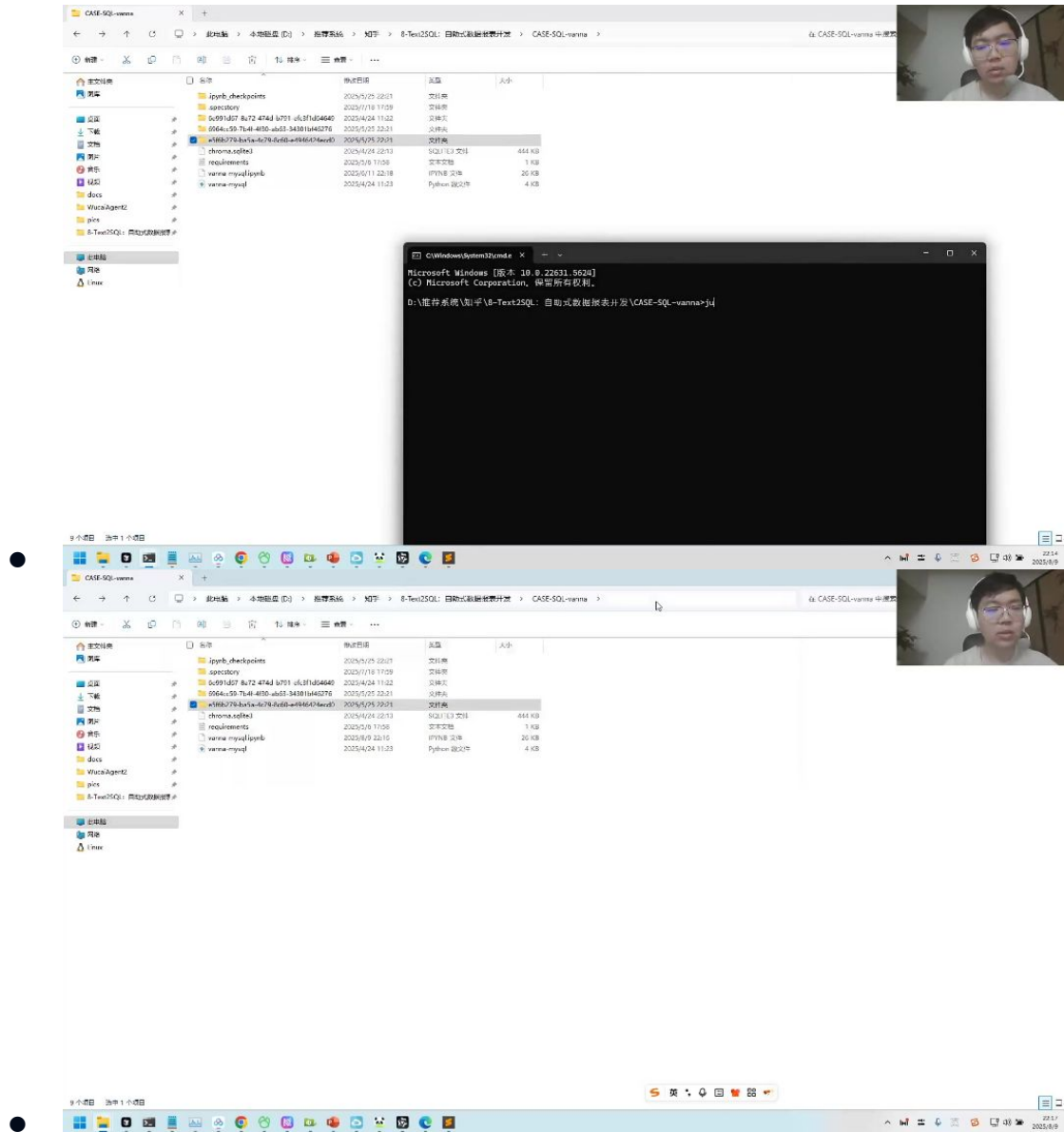
- **开源与定制化：**
 - 提供完整Python库
 - 支持本地化部署
 - 可自定义LLM、向量数据库和关系型数据库（MySQL/PostgreSQL等）
- **RAG增强：**
 - 结合数据库元数据（DDL语句、表注释、示例SQL）训练模型
 - 显著提升复杂查询准确率
- **多场景支持：**
 - 企业数据分析
 - 智能客服
 - 电商搜索
 - 金融报告生成
- **基础设施兼容性：**
 - 支持多种LLM（OpenAI、Ollama等）
 - 支持多种向量数据库（ChromaDB等）
 - 可扩展至非默认支持的数据库

3. vanna工作原理 02:24:33

- **训练机制：**
 - 通过train函数将元数据存入向量数据库
 - 可接受DDL语句、文档和SQL示例
- **生成流程：**
 - 从向量数据库检索相关内容
 - 将检索结果喂给大模型生成SQL

- 自动执行SQL并返回结果
- 可视化功能：
 - 内置图表样式库
 - 可自动生成数据可视化图表
- API使用：
 - 主要入口为ask函数（如ask("查询销售额最高的产品")）
 - 工作流包含SQL生成、执行、画图等模块

1) 例题:vanna使用 02:28:00

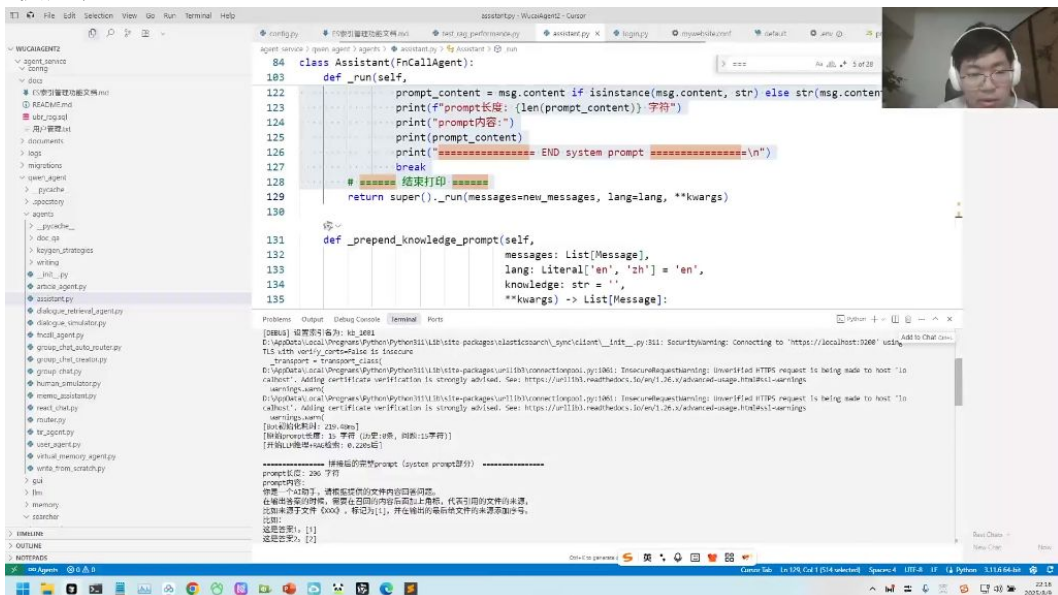


- 环境配置要点：
 - 需要安装额外扩展包（如chromadb、vanna-mysql等）
 - 推荐使用OpenAI模型（对国内模型支持有限）
- 常见问题：
 - 需替换OpenAI API key和模型参数
 - 可能遇到包依赖冲突问题
- 调试建议：
 - 检查向量数据库连接
 - 验证模型API可用性

- 确保训练数据格式正确

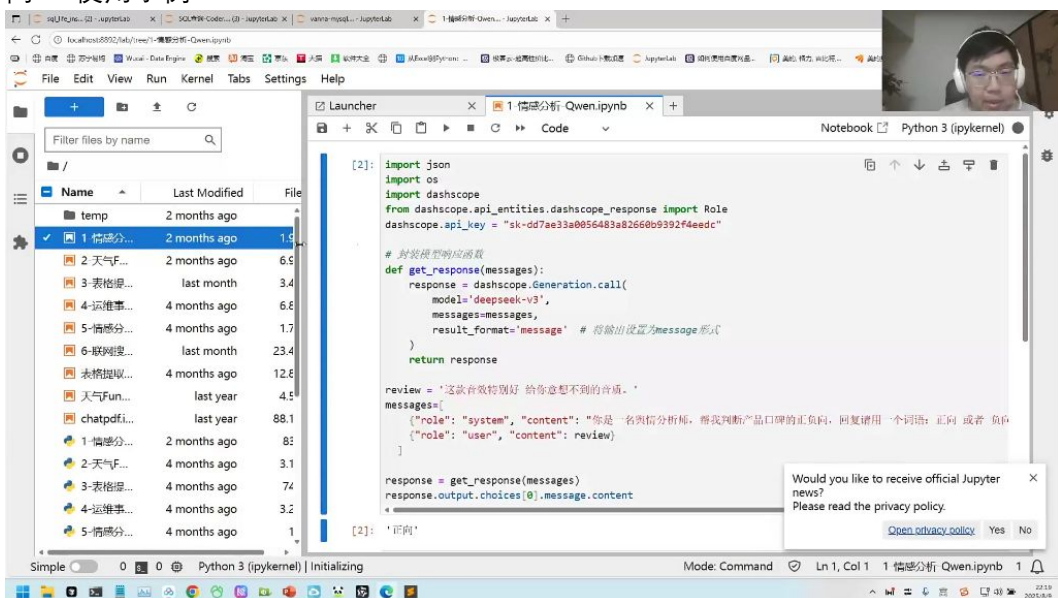
四、问题解答02:31:47

1. API使用演示



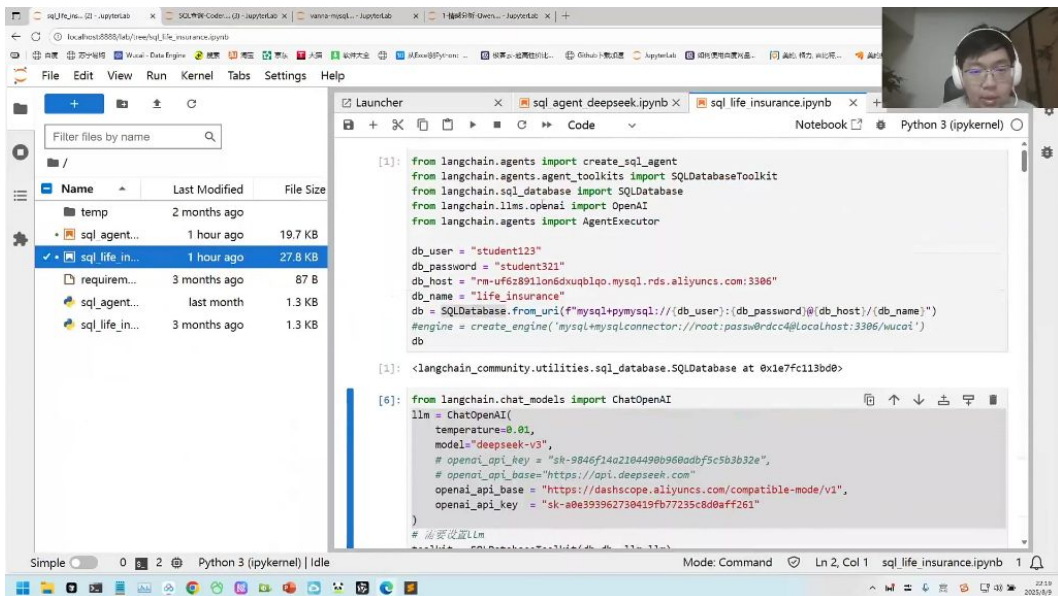
- API支持情况：当前演示环境可能不完全支持千问API，但可以通过其他方式实现类似功能
- 代码获取方式：第一次课程中已提供相关示例代码，学员可自行运行测试

2. 千问API使用示例



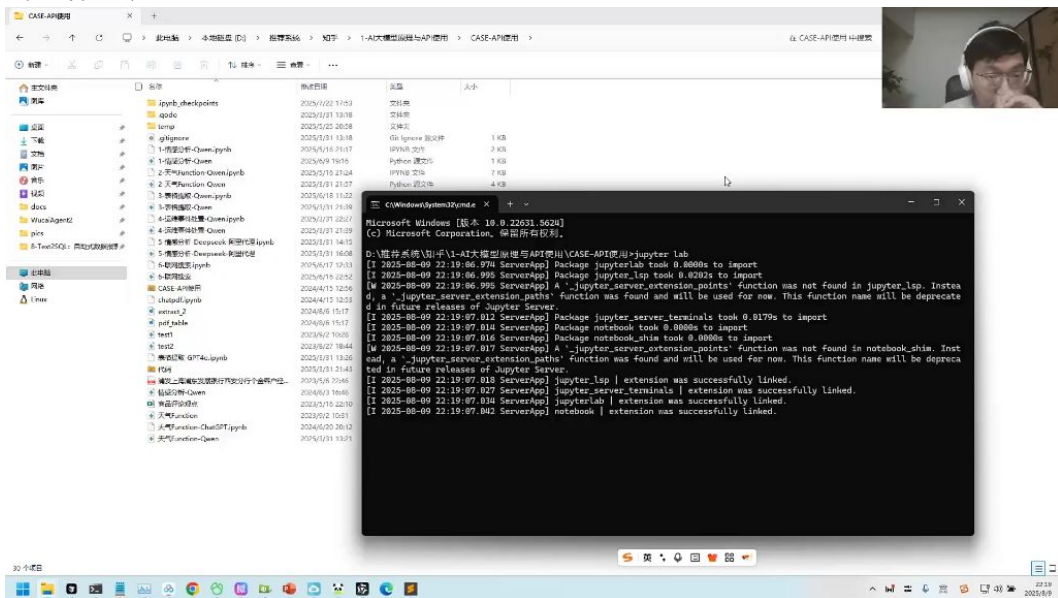
- 代码结构：
 - 核心模块：使用dashscope库调用API
 - 参数设置：需配置api_key="sk-dd7ae33a0056483a82660b9392f4eedc"
 - 模型调用：通过Generation.call()方法调用deepseek-v3模型
- 功能实现：示例展示情感分析功能，输入评论文本返回"正向"或"负向"判断

3. API密钥管理



- 密钥失效处理：当API key失效时需要更换新密钥
- 密钥获取：可在之前课程代码中找到可用密钥
- 模型切换：演示了如何将模型切换为deepseek-v3进行测试

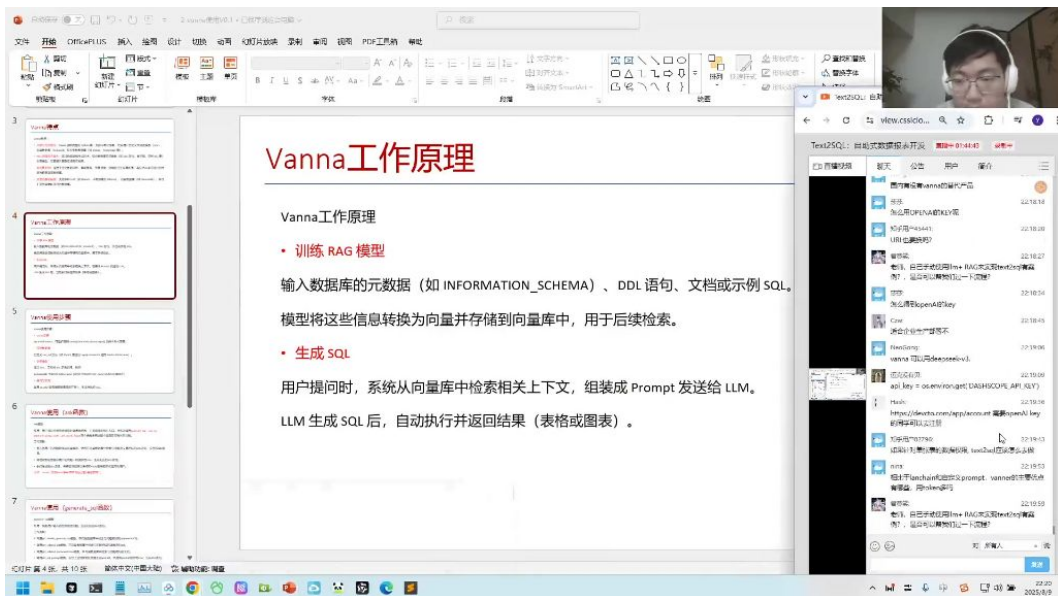
4. 代码运行建议



- 实践建议：
- 所有示例代码均可直接运行测试
- 包含情感分析、天气查询、表格提取等多种功能案例
- 建议按文件分类逐个测试不同功能模块

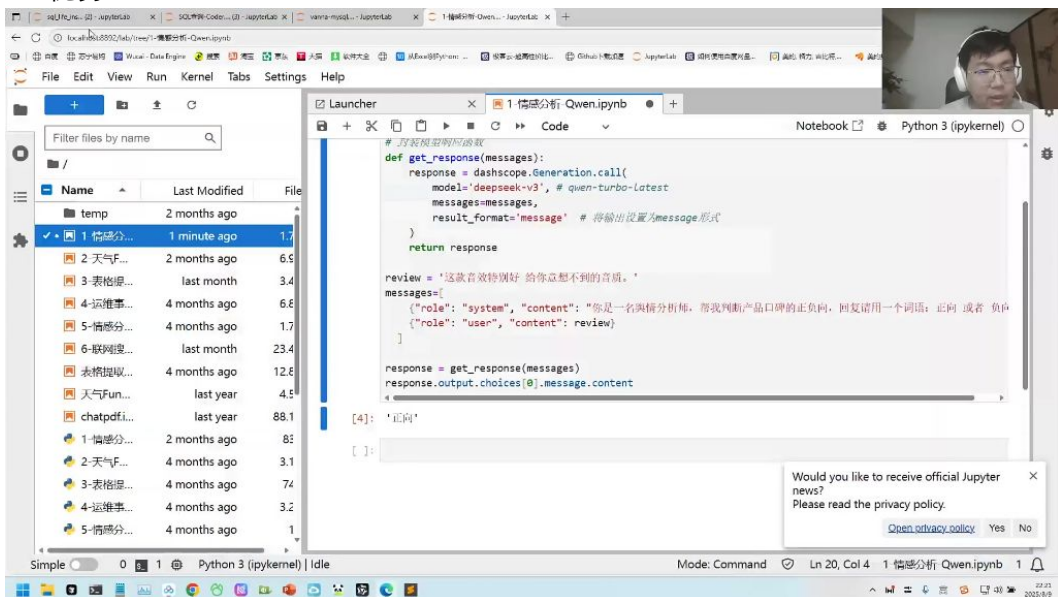
五、vanna使用详解02:33:54

1. 接口演示



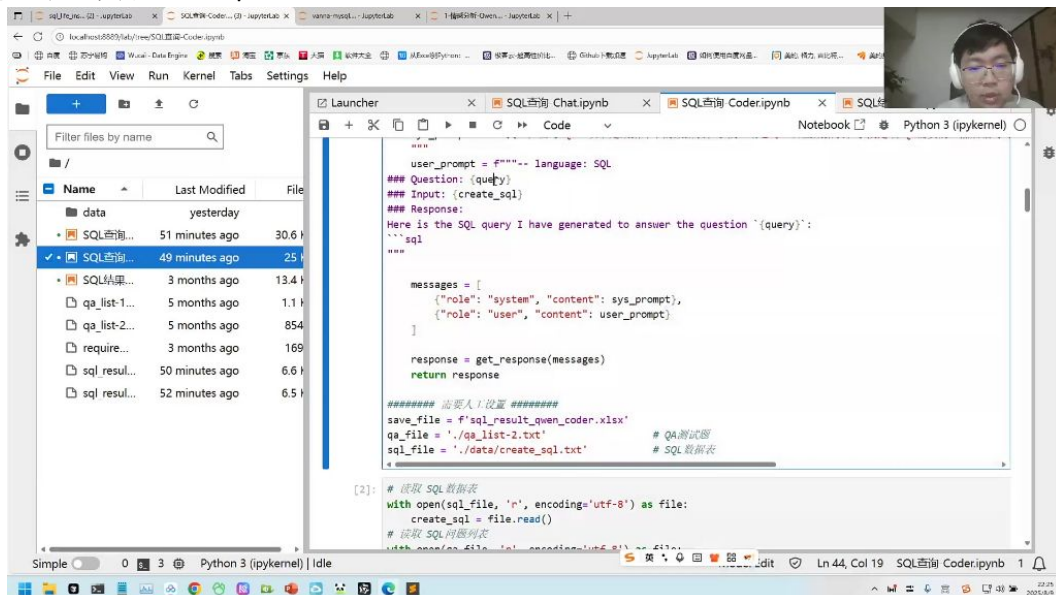
- 工作原理：
 - 输入数据库元数据（如`[INFORMATION_SCHEMA]`）、DDL语句、文档或示例SQL
 - 模型将信息转换为向量并存储到向量库
 - 用户提问时从向量库检索相关上下文，组装成Prompt发送给LLM
 - LLM生成SQL后自动执行并返回结果（表格或图表）
- API配置：
 - 需要`OPENAI_API_KEY`环境变量
 - 示例代码：`api_key = os.environ.get('OPENAI_API_KEY')`

2. vanna优势 02:35:57



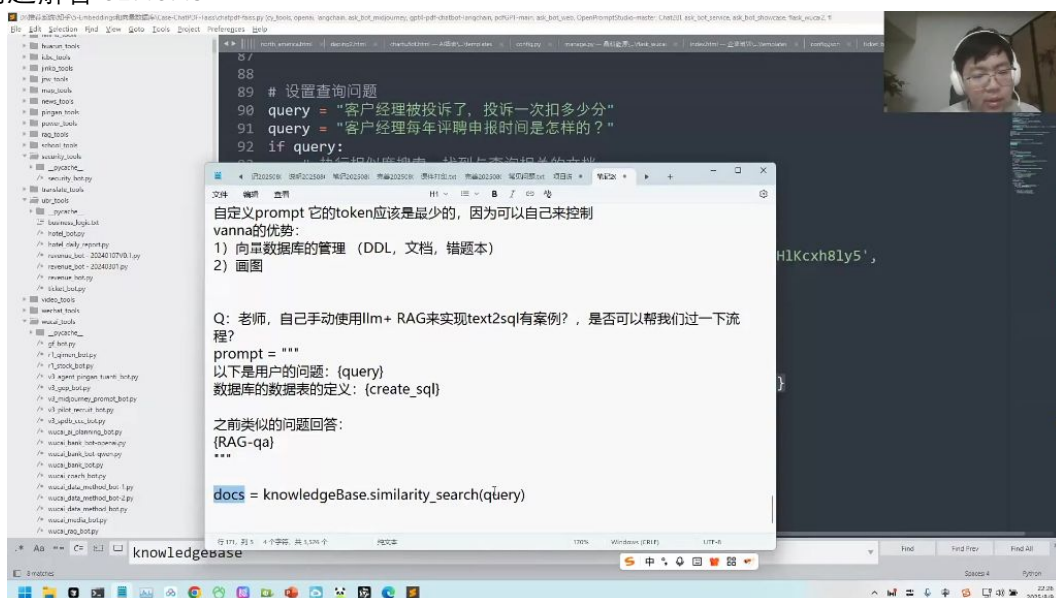
- 核心优势：
 - 向量数据库管理：支持添加DDL、文档、错题本等多种数据类型
 - 可视化能力：不仅能查询数据，还能自动生成多种样式的图表
 - token效率：相比LangChain，token使用量更可控
- 适用场景：
 - 企业生产环境
 - 需要可视化展示的报表开发
 - 需要管理大量数据库元数据的场景

3. 自己动手实现text2sql 02:37:04



- 实现步骤：
 - 构建Prompt模板：
 - 使用FAISS建立知识库存储历史案例
 - 通过相似度搜索获取相关文档
 - 组装Prompt发送给LLM
- 关键代码：

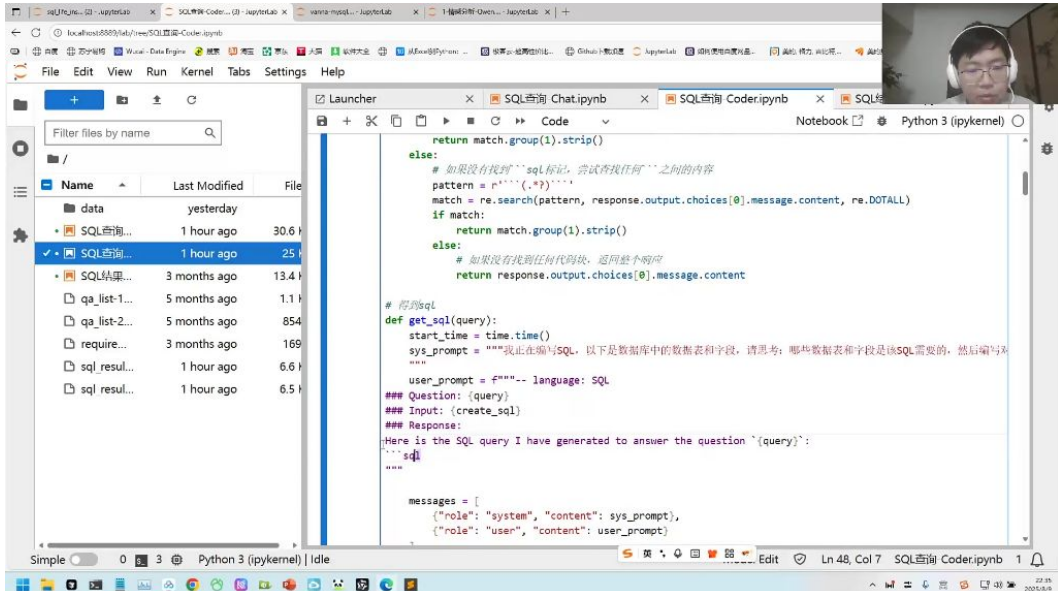
4. 问题解答 02:40:49



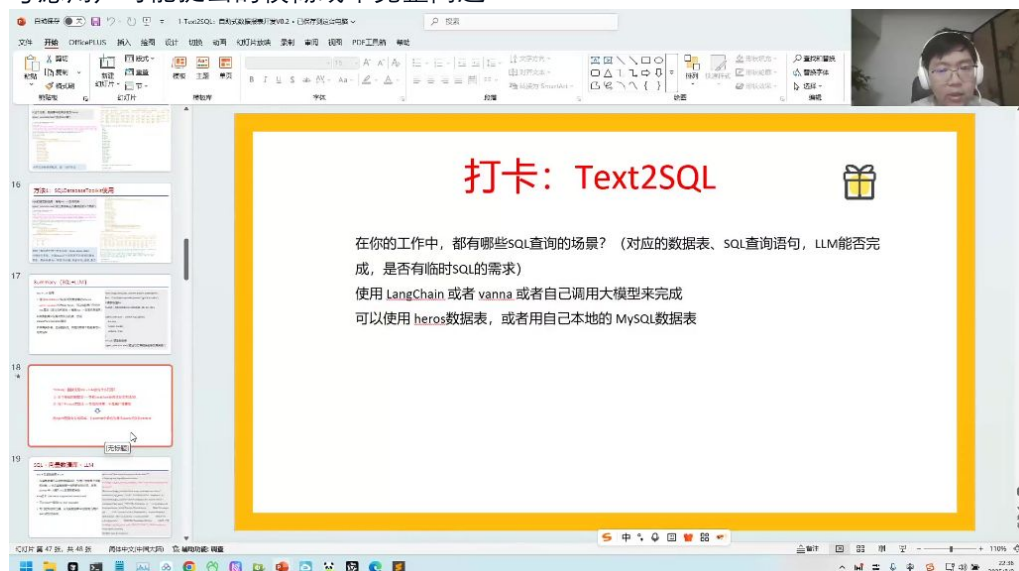
- **数据权限问题：**
 - **表级权限：**通过数据库GRANT命令实现
 - **行级权限：**利用数据库原生行锁定功能
 - 示例：heroes表可通过授权控制查询/写入权限
- **SQL安全隐患：**
 - 使用SQLAlchemy等ORM工具避免原生SQL拼接
 - 实现双重审核机制：
 - 事前：让LLM判断query安全性（返回JSON格式的安全评估）
 - 事后：对生成SQL进行安全扫描

- 商业化落地建议：
 - 建立测试集作为金标准（如1000条测试query）
 - 与客户约定准确率预期（如95%）
 - 对不确定问题返回"无法确定"而非错误答案

六、内容总结02:49:30



- 三种实现方式：
 - LangChain：调用最方便，适合快速验证
 - 手动实现：灵活性最高，适合定制需求
 - Vanna：提供向量库管理和可视化两大核心优势
- 测试注意事项：
 - 确保测试集多样性
 - 包含边缘用例和异常情况
 - 考虑用户可能提出的模糊或不完整问题



- 实践作业：
 - 使用任意方法实现Text2SQL
 - 可选用heros数据表或本地MySQL
 - 提交运行截图和体验报告

七、知识小结

知识点	核心内容	技术要点	应用场景	难度系数
自助式报表开发	讲解test two circle自助报表开发，提供课件代码和数据下载	SQL自动生成、结构化数据处理	BI报表、数据查询	★★★
结构化vs非结构化数据	结构化数据(excel)格式固定价值高，非结构化数据(PDF/word)需解析	数据表关联、字段类型定义	数据仓库建设、报表生成	★★
大模型选择策略	对比Cloud/Gemini/千问/DeepSeek等模型代码生成能力	API调用与本地部署差异	企业级应用开发	★★★★
SQL生成三段演进	从模板规则→机器学习→大模型生成的技术发展路径	语义理解与模式关联	自然语言转查询	★★★
LangChain应用	SQL Agent工具实现多表联查，自动解析表结构	向量数据库检索、函数调用	快速原型开发	★★★★
提示词工程最佳实践	建表语句+字段注释+补全触发的高效写法	...SQL补全激发、元数据注入	精准SQL生成	★★★★
保险行业案例	7张数据表(客户/保单/理赔等)的复杂查询实践	子查询优化、多表关联	金融风控分析	★★★★
安全防护机制	SQL注入防范、权限管理、结果校验三重保障	连接池控制、行级权限	企业级部署	★★★★★
测试评估体系	通过测试集(1000+样例)验证准确率	错误案例归集分析	项目验收	★★★★
可视化集成	自动图表生成(柱状图/趋势图等)	数据透视、样式模板	决策看板	★★★